VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

School of Computer Science and Engineering (SCOPE)

MINI PROJECT REPORT

VaxTrack – Newborn Vaccination Schedule Tracker

Course Code: BCSE302P – Database Systems Laboratory

Submitted by

S. No.	Name	Registration Number
1	Nekita Saras Dhanabal	24BCE2991

Under the guidance of

Dr. G. M. Karthik

Associate Professor, School of Computer Science and Engineering (SCOPE)

Vellore Institute of Technology, Vellore

Date of Submission: 12 / 10 / 2026

CERTIFICATE

This is to certify that the Mini Project Report titled "**VaxTrack – Newborn Vaccination Schedule Tracker**" submitted by **Nekita Saras Dhanabal (24BCE2991)** in partial fulfilment of the requirements of the course BCSE302P – Database Systems Laboratory, is a record of original work carried out under my supervision during the Fall Semester 2026–27.

Signature of the Faculty

Dr. G. M. Karthik

Associate Professor, SCOPE

VIT, Vellore

Place: Vellore

Date: 12 / 10 / 2026

ABSTRACT

Newborns in India must receive vaccines in a fixed sequence at set ages, from birth to the boosters at 16–23 months, while clinics must also keep track of vaccine batches and their expiry dates. Paper cards and spreadsheets make it easy to miss a dose, to give a vaccine from an empty or expired batch, or to lose track of who changed a record. VaxTrack is a newborn vaccination schedule tracker built on Oracle Database 21c Express Edition, in which PL/SQL owns every business rule. The vaccination schedule of the National Immunization Schedule is stored as data and expanded into 24 dose records per baby (26 in districts where Japanese encephalitis is endemic). Four packages, five standalone functions, one standalone procedure, five triggers, explicit, parameterized, FOR-loop and REF cursors, and a DBMS_SCHEDULER job handle scheduling, dose administration with first-expiry-first-out stock deduction, overdue detection, alerts and auditing. A Node.js and Express API passes requests to the packages, and a React web application serves three kinds of users: administrators, nurses and parents. The behaviour was verified with 31 automated PL/SQL test cases, 152 API checks and browser tests. The project covers infants and the 16–23 month boosters.

Keywords: DBMS, Oracle PL/SQL, Packages, Triggers, Cursors, Normalization, Immunization Schedule

TABLE OF CONTENTS

CERTIFICATE 2

ABSTRACT 3

TABLE OF CONTENTS 4

1. INTRODUCTION 6

    1.1 Problem Statement 6

    1.2 Objectives 6

    1.3 Importance of DBMS 7

    1.4 Scope of the Project 7

2. SYSTEM REQUIREMENTS 9

    2.1 Hardware Requirements 9

    2.2 Software Requirements 9

3. DATABASE DESIGN 10

    3.1 Entity–Relationship (ER) Diagram 10

    3.2 Relational Schema 13

    3.3 Schema Definition (DDL) 18

    3.4 Normalization 19

4. IMPLEMENTATION 22

    4.1 Functions 23

    4.2 Procedures 24

    4.3 Packages 25

    4.4 Triggers 28

    4.5 Cursors 30

    4.6 Scheduled Job 33

    4.7 Transactions and Error Handling 34

5. FRONT-END 35

    5.1 User Interface 35

5.2 Integration with the DBMS....45
6. VERIFICATION & TESTING....49
6.1 Test Cases....49
6.2 Output Screenshots....51
6.3 Validation of Functionality....58
7. RESULTS & DISCUSSION....60
7.1 Achievements....60
7.2 Challenges Faced....61
7.3 Possible Improvements....62
8. CONCLUSION....64
8.1 Summary of Learning Outcomes....64
8.2 Future Scope....64
9. REFERENCES....65
10. APPENDICES....66
Appendix A – Additional Code....66
Appendix B – Additional Screenshots....67

1. INTRODUCTION

1.1 Problem Statement

A newborn has to receive a long series of vaccines at fixed ages. In India the National Immunization Schedule (NIS) of the Ministry of Health and Family Welfare lists them: BCG, oral polio and hepatitis B at birth, a first round of five vaccines at 6 weeks, further rounds at 10 and 14 weeks, the measles-rubella, pneumococcal booster, polio and vitamin A doses at 9–11 months, and boosters at 16–23 months [6]. Each dose has an age at which it becomes due and an age after which it counts as late, so a single child has 24 doses to track, and 26 in districts where Japanese encephalitis is endemic.

The people involved are the clinic nurses who give the vaccines, the parents or guardians who must bring the child on time, and an administrator who supervises several clinics. In many clinics the record is a paper card or a spreadsheet. With that approach nobody is told which dose is due next or which one has been missed, a nurse can record a dose from a batch that is empty or past its expiry date, a stock count can drift away from the number of doses actually given, and there is no trustworthy answer to the question of who changed a record, and when. VaxTrack addresses these four problems with a database that enforces the rules itself.

1.2 Objectives

1. Design a normalized relational schema (up to 3NF/BCNF) for guardians, newborns, the vaccine schedule, clinic stock, dose records, alerts and an audit trail, with constraints that reject invalid data.
2. Implement all business rules in Oracle PL/SQL, using standalone functions and a procedure, packages, triggers and the four kinds of cursor, so that scheduling, stock deduction, overdue detection and alerts are decided in the database and not in application code.
3. Provide a web application for three roles (administrator, nurse, parent) that reaches the database only through the PL/SQL packages.
4. Verify the system with an automated test script and with end-to-end runs, and keep the evidence.

1.3 Importance of DBMS

A database management system is the natural home for this application [1], [2], for the following reasons, each of which we use directly.

- **Integrity.** Primary, foreign, unique and check constraints keep the data valid: a dose marked DONE must have an administered date, stock can never go below zero, and a nurse account must belong to a clinic. Triggers cover the rules a constraint cannot express, such as a date of birth that is not in the future.
- **Reduced redundancy.** The schema is normalized, so guardian details, clinic details and vaccine details are stored once and referred to by key. The vaccination schedule itself is data in a table, so changing a due age needs an UPDATE, not a code change.
- **Concurrency control.** Two nurses may give the last dose of a batch at the same moment. The administration procedure locks the dose row with SELECT … FOR UPDATE and guards the stock update, so only one of them succeeds.
- **Atomicity and recovery.** Recording a dose changes the dose row and the stock row. Both changes happen inside one transaction with a savepoint, so a failure leaves neither of them changed.
- **Security.** Passwords are stored as hashes, access depends on the role, and the application reaches the data only through named packages.
- **Auditability.** A trigger writes every insert, update and delete on the dose records, together with the user who caused it, to a log that is not removed when the audited row is deleted.
- **Efficiency.** Indexes support the frequent queries, for example the first-expiry-first-out choice of a batch.

1.4 Scope of the Project

Included. Infants (0–1 year) and the 16–23 month boosters of the NIS: 24 core doses per baby, plus the two Japanese encephalitis doses for babies registered in an endemic district. Registration of babies and guardians, automatic schedule generation, administration of doses with stock deduction, rescheduling, overdue and due-soon detection, low-stock and expiry alerts, clinic stock management, reports, a printable vaccination card, an audit log, and administrator tools to correct or delete a newborn record. Three user roles with their own views.

Out of scope. Td for pregnant women, the DPT booster at 5–6 years, Td at 10 and 16 years, and vitamin A doses 3 to 9, because they are beyond newborn tracking. SMS or WhatsApp reminders, a native mobile application and a progressive web app are also left out; the application is a responsive web application only. These items are listed again as future scope in Section 8.2.

Assumptions we made. We count one month as 30 days throughout. The NIS gives age bands and not exact limits, so the "late after" age (`max_age_days`) is a limit we defined: 14 days for the birth doses, four weeks of grace for the doses due at 6, 10 and 14 weeks, and the end of the stated band for the month-based doses. A dose is allowed from its due age. The route of administration and the disease covered by each vaccine are standard programme facts that are not printed in the NIS document we used.

2. SYSTEM REQUIREMENTS

Values below were read from the machine on which the project was built and tested.

2.1 Hardware Requirements

Component	Specification
Processor	Intel Core Ultra 7 155U, 12 cores, 14 logical processors
RAM	16 GB installed class (15.4 GB usable)
Storage	512 GB class NVMe SSD (KIOXIA, 477 GB formatted)
Other	None. The application runs in a desktop or phone web browser; no mobile device or special hardware is needed

We did not test on a smaller machine, so these figures describe what we used and not a minimum.

2.2 Software Requirements

Category	Tool / Version
Operating System	Windows 11 Home Single Language, version 10.0.26200
DBMS	Oracle Database 21c Express Edition, release 21.3.0.0.0, pluggable database XEPDB1, application schema `vaxapp`
Query / IDE Tool	SQL*Plus 21.3.0.0.0 for running the scripts; Visual Studio Code for editing
Front-End Language	JavaScript (React 18.3.1, Vite 8.3.2, Tailwind CSS 4.3.3, React Router 7.18.4, TanStack Query 5.104.1, Recharts 3.10.1, lucide-react 1.52.0)
Back-End	Node.js 24.13.1 (the project needs 20 or later), npm 11.14.1, Express 5.2.1
Connectivity	node-oracledb 7.0.1 in thin mode (no Oracle Client installation needed)
Security libraries	bcrypt 6.0.0, jsonwebtoken 9.0.3, helmet 8.3.0, cors 2.8.6, zod 4.6.5
Browser	Microsoft Edge 155 (Chromium)
Other Tools	Git 2.53.0, Mermaid for the ER and architecture diagrams, Lighthouse and Playwright (driving Edge) for the accessibility and browser checks

The Oracle features we used are described in [3]–[5]; the driver, server, front-end and style libraries in [7]–[11].

3. DATABASE DESIGN

3.1 Entity–Relationship (ER) Diagram

The schema has ten tables. The entities and their key attributes are as follows.

- **CLINICS** (`clinic_id`): a vaccination centre, with a unique name, a city and a phone number.
- **GUARDIANS** (`guardian_id`): the parent or guardian of a baby, with a unique phone number.
- **NEWBORNS** (`baby_id`): a baby, with date of birth, gender and birth weight. Each baby has exactly one guardian and is registered at exactly one clinic.
- **VACCINE_MASTER** (`vaccine_id`): one row per dose of the national schedule, with the minimum, due and late-after age in days. It is the schedule stored as data.
- **VACCINE_INVENTORY** (`inventory_id`): one row per batch of one vaccine at one clinic, with expiry date, quantity and a low-stock threshold.
- **DOSE_RECORDS** (`record_id`): one row per baby per dose, with the scheduled date, the status (PENDING, DONE or OVERDUE), and, once given, the date, the batch used and the user who gave it.
- **APP_USERS** (`user_id`): a login. A nurse belongs to a clinic, a parent belongs to a guardian, an administrator belongs to neither.
- **ALERTS** (`alert_id`): overdue, due-soon, late-dose, low-stock and expiry messages.
- **AUDIT_LOG** (`audit_id`) and **TEST_LOG** (`test_id`): the audit trail and the record of the automated tests. They have no foreign keys, deliberately: audit rows must survive the deletion of the rows they describe.

Relationship	Cardinality	Participation
Guardian – Newborn	1 : N	Every newborn must have a guardian; a guardian may exist without a newborn
Clinic – Newborn	1 : N	Every newborn is registered at one clinic
Newborn – Dose record	1 : N	A baby has 24 or 26 dose records; each dose record belongs to one baby
Vaccine master – Dose record	1 : N	Each dose record refers to one scheduled dose
Clinic – Vaccine inventory	1 : N	A clinic holds many batches
Vaccine master – Vaccine inventory	1 : N	A batch is of one vaccine
Vaccine inventory – Dose record	1 : N (optional)	A dose record refers to the batch used only after the dose is given
Clinic – App user (nurse)	1 : N (optional)	Required for the NURSE role, empty for others
Guardian – App user (parent)	1 : 1 (optional)	Required for the PARENT role; a guardian has at most one login (enforced by `account_pkg`, not by a key)
App user – Dose record	1 : N (optional)	The user who gave the dose
Dose record – Alert	1 : N (optional)	An alert may point to the dose that caused it

The diagram is Figure 3.1.

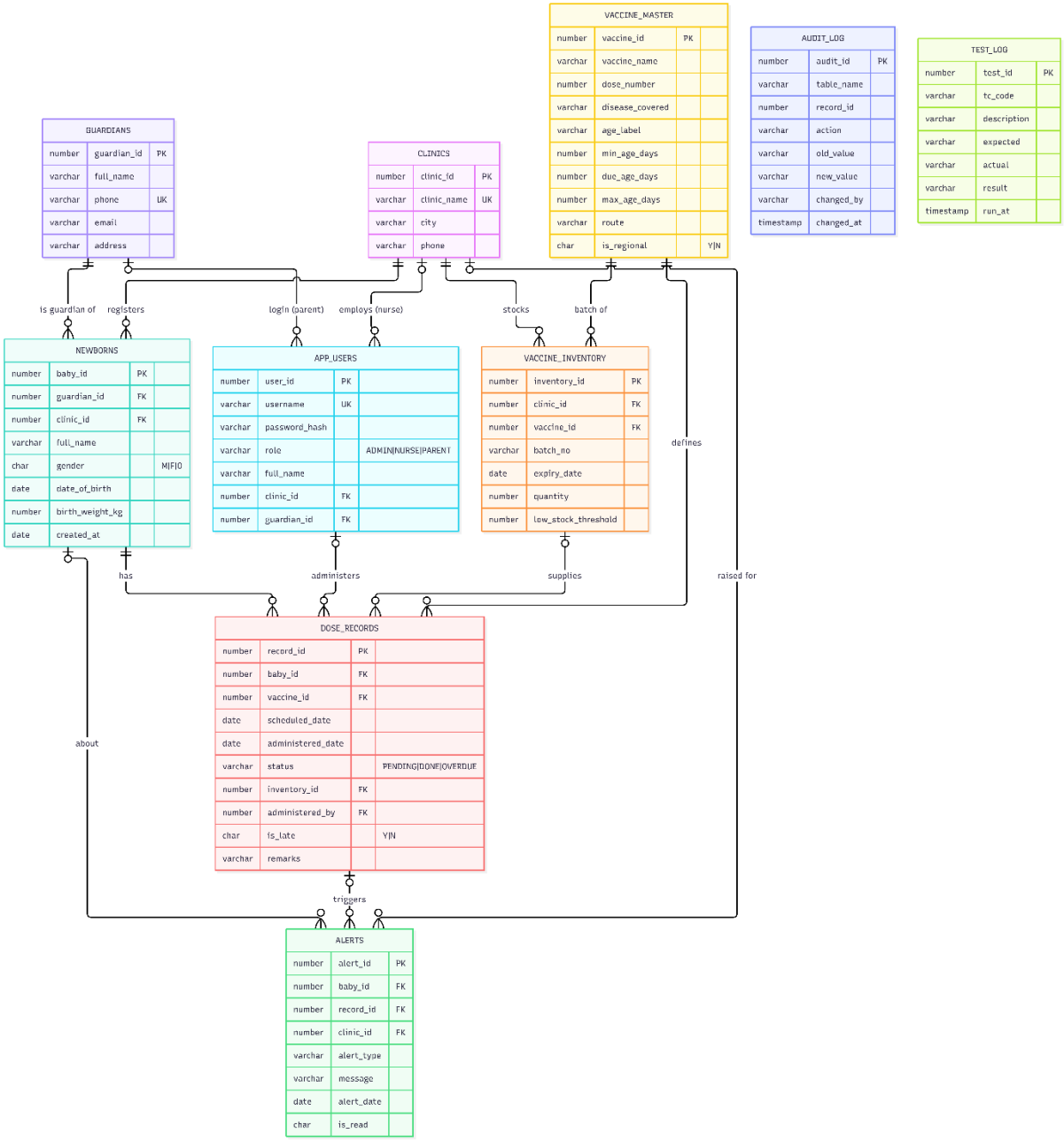


Figure 3.1: Entity-relationship diagram of the VaxTrack schema

3.2 Relational Schema

Each relation is shown below with its constraints. Primary keys are identity columns, so no identifier is typed by hand anywhere in the project.

Table 3.1: CLINICS

Attribute	Data Type	Constraint	Description
clinic_id	NUMBER	PRIMARY KEY (identity)	Clinic identifier
clinic_name	VARCHAR2(100)	NOT NULL, UNIQUE	Name of the clinic
city	VARCHAR2(60)	NOT NULL	City
phone	VARCHAR2(20)		Contact number

Table 3.2: GUARDIANS

Attribute	Data Type	Constraint	Description
guardian_id	NUMBER	PRIMARY KEY (identity)	Guardian identifier
full_name	VARCHAR2(100)	NOT NULL	Name
phone	VARCHAR2(15)	NOT NULL, UNIQUE, CHECK 7–15 digits, spaces, + or -	Phone number; a guardian found by phone is reused
email	VARCHAR2(120)		Optional e-mail address
address	VARCHAR2(250)		Address (required by the registration procedure for a new guardian)

Table 3.3: APP_USERS

Attribute	Data Type	Constraint	Description
user_id	NUMBER	PRIMARY KEY (identity)	Login identifier
username	VARCHAR2(50)	NOT NULL, UNIQUE	Login name
password_hash	VARCHAR2(100)	NOT NULL	bcrypt hash, never the password
role	VARCHAR2(10)	NOT NULL, CHECK IN ('ADMIN','NURSE','PARENT')	Role
full_name	VARCHAR2(100)	NOT NULL	Display name
clinic_id	NUMBER	FOREIGN KEY → CLINICS(clinic_id)	Clinic of a nurse
guardian_id	NUMBER	FOREIGN KEY → GUARDIANS(guardian_id)	Guardian of a parent
(table check)		CHECK: ADMIN, or NURSE with a clinic, or PARENT with a guardian	A login always has the scope its role needs

Table 3.4: NEWBORNS

Attribute	Data Type	Constraint	Description
baby_id	NUMBER	PRIMARY KEY (identity)	Baby identifier
guardian_id	NUMBER	NOT NULL, FOREIGN KEY → GUARDIANS(guardian_id)	Guardian
clinic_id	NUMBER	NOT NULL, FOREIGN KEY → CLINICS(clinic_id)	Registering clinic
full_name	VARCHAR2(100)	NOT NULL	Name
gender	CHAR(1)	NOT NULL, CHECK IN ('M','F','O')	Gender
date_of_birth	DATE	NOT NULL	Date of birth; not in the future (trigger)
birth_weight_kg	NUMBER(4,2)	CHECK between 0.5 and 9.99	Birth weight
created_at	DATE	DEFAULT SYSDATE, NOT NULL	Registration date

Table 3.5: VACCINE_MASTER

Attribute	Data Type	Constraint	Description
vaccine_id	NUMBER	PRIMARY KEY (identity)	Dose identifier
vaccine_name	VARCHAR2(60)	NOT NULL, UNIQUE with dose_number	Vaccine name as in the NIS
disease_covered	VARCHAR2(150)	NOT NULL	Diseases prevented
dose_number	NUMBER(2)	NOT NULL, CHECK >= 0	Dose number (0 for the birth polio dose)
age_label	VARCHAR2(30)	NOT NULL	Age band, e.g. "6 weeks"
min_age_days	NUMBER(5)	NOT NULL	Earliest age at which the dose may be given
due_age_days	NUMBER(5)	NOT NULL	Age at which the dose is due
max_age_days	NUMBER(5)	NOT NULL	Age after which the dose counts as late
route	VARCHAR2(30)	NOT NULL	Route of administration
is_regional	CHAR(1)	DEFAULT 'N', NOT NULL, CHECK IN ('Y','N')	'Y' for Japanese encephalitis (endemic districts only)
(table check)		CHECK min_age_days <= due_age_days <= max_age_days	The three ages are in order

Table 3.6: VACCINE_INVENTORY

Attribute	Data Type	Constraint	Description
inventory_id	NUMBER	PRIMARY KEY (identity)	Batch row identifier
clinic_id	NUMBER	NOT NULL, FOREIGN KEY → CLINICS(clinic_id)	Clinic holding the batch
vaccine_id	NUMBER	NOT NULL, FOREIGN KEY → VACCINE_MASTER(vaccine_id)	Vaccine
batch_no	VARCHAR2(40)	NOT NULL, UNIQUE with clinic_id and vaccine_id	Batch number
expiry_date	DATE	NOT NULL	Expiry date
quantity	NUMBER(6)	DEFAULT 0, NOT NULL, CHECK >= 0	Doses left
low_stock_threshold	NUMBER(6)	DEFAULT 10, NOT NULL, CHECK >= 0	Alert level

Table 3.7: DOSE_RECORDS

Attribute	Data Type	Constraint	Description
record_id	NUMBER	PRIMARY KEY (identity)	Dose record identifier
baby_id	NUMBER	NOT NULL, FOREIGN KEY → NEWBORNS(baby_id), UNIQUE with vaccine_id	Baby
vaccine_id	NUMBER	NOT NULL, FOREIGN KEY → VACCINE_MASTER(vaccine_id)	Scheduled dose
scheduled_date	DATE	NOT NULL	Due date (date of birth + due_age_days, or a rescheduled date)
administered_date	DATE		Date the dose was given
status	VARCHAR2(10)	DEFAULT 'PENDING', NOT NULL, CHECK IN ('PENDING','DONE','OVERDUE')	State of the dose
inventory_id	NUMBER	FOREIGN KEY → VACCINE_INVENTORY(inventory_id)	Batch used
administered_by	NUMBER	FOREIGN KEY → APP_USERS(user_id)	User who gave the dose
is_late	CHAR(1)	DEFAULT 'N', NOT NULL, CHECK IN ('Y','N')	Given after max_age_days
remarks	VARCHAR2(500)		Free text, e.g. a reschedule reason
(table check)		CHECK: DONE if and only if administered_date is not null	A dose is DONE exactly when it has a date

Table 3.8: ALERTS

Attribute	Data Type	Constraint	Description
alert_id	NUMBER	PRIMARY KEY (identity)	Alert identifier
baby_id	NUMBER	FOREIGN KEY → NEWBORNS(baby_id)	Baby concerned
record_id	NUMBER	FOREIGN KEY → DOSE_RECORDS(record_id)	Dose concerned
clinic_id	NUMBER	FOREIGN KEY → CLINICS(clinic_id)	Clinic to be told
alert_type	VARCHAR2(10)	NOT NULL, CHECK IN ('DUE_SOON','OVERDUE','LATE_DOSE','LOW_STOCK','EXPIRY')	Kind of alert
message	VARCHAR2(400)	NOT NULL	Text shown to the user
alert_date	DATE	DEFAULT SYSDATE, NOT NULL	When it was raised
is_read	CHAR(1)	DEFAULT 'N', NOT NULL, CHECK IN ('Y','N')	Read flag

Table 3.9: AUDIT_LOG

Attribute	Data Type	Constraint	Description
audit_id	NUMBER	PRIMARY KEY (identity)	Audit row identifier
table_name	VARCHAR2(30)	NOT NULL	Table that changed
record_id	NUMBER		Key of the changed row
action	VARCHAR2(10)	NOT NULL, CHECK IN ('INSERT','UPDATE','DELETE')	Kind of change
old_value, new_value	VARCHAR2(4000)		Text of the row before and after
changed_by	VARCHAR2(100)		User, taken from the session's client information
changed_at	TIMESTAMP	DEFAULT SYSTIMESTAMP, NOT NULL	Time of the change

Table 3.10: TEST_LOG

Attribute	Data Type	Constraint	Description
test_id	NUMBER	PRIMARY KEY (identity)	Row identifier
tc_code	VARCHAR2(20)	NOT NULL	Test case code, e.g. TC-P4
description	VARCHAR2(300)	NOT NULL	What is tested

Attribute	Data Type	Constraint	Description
expected, actual	VARCHAR2(1000)		Expected and observed output
result	VARCHAR2(4)	NOT NULL, CHECK IN ('PASS','FAIL')	Outcome
run_at	TIMESTAMP	DEFAULT SYSTIMESTAMP, NOT NULL	Time of the run

The tables of the schema are listed in Figure 3.2 and their constraints in Figure 3.3.

```
SQL> SELECT t.table_name,
  2         (SELECT COUNT(*) FROM user_tab_columns c WHERE c.table_name = t.table_name) AS columns_count
  3    FROM user_tables t ORDER BY t.table_name;

TABLE_NAME              COLUMNS_COUNT
----------------------- -------------
ALERTS                              8
APP_USERS                           7
AUDIT_LOG                           8
CLINICS                             4
DOSE_RECORDS                       10
GUARDIANS                           5
NEWBORNS                            8
TEST_LOG                            7
VACCINE_INVENTORY                   7
VACCINE_MASTER                     10

10 rows selected.
```

Figure 3.2: Tables of the schema with column counts

```
SQL> SELECT table_name,
  2         COUNT(CASE WHEN constraint_type = 'P' THEN 1 END) AS pk,
  3         COUNT(CASE WHEN constraint_type = 'R' THEN 1 END) AS fk,
  4         COUNT(CASE WHEN constraint_type = 'U' THEN 1 END) AS uq,
  5         COUNT(CASE WHEN constraint_type = 'C' THEN 1 END) AS chk
  6    FROM user_constraints
  7   WHERE constraint_name NOT LIKE 'SYS\_%' ESCAPE '\'
  8   GROUP BY table_name ORDER BY table_name;

TABLE_NAME                PK         FK         UQ        CHK
------------------- ---------- ---------- ---------- ----------
ALERTS                     1          3          0          2
APP_USERS                  1          2          1          2
AUDIT_LOG                  1          0          0          1
CLINICS                    1          0          1          0
DOSE_RECORDS               1          4          1          3
GUARDIANS                  1          0          1          1
NEWBORNS                   1          2          0          2
TEST_LOG                   1          0          0          1
VACCINE_INVENTORY          1          2          1          2
VACCINE_MASTER             1          0          1          3

10 rows selected.

SQL> COLUMN constraint_name FORMAT A28
SQL> COLUMN search_condition_vc FORMAT A90
SQL> SELECT constraint_name, search_condition_vc AS check_condition
  2    FROM user_constraints
  3   WHERE constraint_type = 'C' AND constraint_name LIKE 'CK\_%' ESCAPE '\'
  4     AND table_name IN ('DOSE_RECORDS', 'VACCINE_MASTER', 'APP_USERS', 'VACCINE_INVENTORY')
  5   ORDER BY table_name, constraint_name;

CONSTRAINT_NAME              CHECK_CONDITION
---------------------------- --------------------------------------------------
CK_APP_USERS_ROLE            role IN ('ADMIN','NURSE','PARENT')
CK_APP_USERS_SCOPE           role = 'ADMIN'
                                                                  OR (role =
                             'NURSE'  AND clinic_id   IS NOT NULL)
                                                                  OR (role =
                             'PARENT' AND guardian_id IS NOT NULL)

CK_DOSE_DONE_DATE            (status = 'DONE' AND administered_date IS NOT NULL
                             )
                                                                 OR (status <
                             > 'DONE' AND administered_date IS NULL)

CK_DOSE_LATE                 is_late IN ('Y','N')
CK_DOSE_STATUS               status IN ('PENDING','DONE','OVERDUE')
CK_INVENTORY_QTY             quantity >= 0
CK_INVENTORY_THRESH          low_stock_threshold >= 0
CK_VACCINE_AGES              min_age_days >= 0
                                                                    AND min_age_da
                             ys <= due_age_days
                                                                    AND due_age_da
                             ys <= max_age_days

CK_VACCINE_DOSE_NO           dose_number >= 0
CK_VACCINE_REGIONAL          is_regional IN ('Y','N')

10 rows selected.
```

Figure 3.3: Primary, foreign, unique and check constraints per table

The schedule that `vaccine_master` holds is given in Figure 3.4. It follows the NIS: 3 rows at birth, 5 at 6 weeks, 3 at 10 weeks, 5 at 14 weeks, 4 at 9–11 months and 4 at 16–23 months, which makes 24, plus the two Japanese encephalitis rows marked `is_regional = 'Y'`.

```
SQL> SELECT dose_label(vaccine_name, dose_number) AS dose, age_label, min_age_days AS min_d, due_age_days AS due_d,
  2         max_age_days AS max_d, route, is_regional AS je_only
  3    FROM vaccine_master ORDER BY due_age_days, vaccine_id;

DOSE                   AGE_LABEL           MIN_D      DUE_D      MAX_D ROUTE          J
---------------------- -------------- ---------- ---------- ---------- -------------- -
Hepatitis B Birth dose At birth                0          0         14 Intramuscular  N
BCG                    At birth                0          0         14 Intradermal    N
bOPV-0                 At birth                0          0         14 Oral           N
PCV-1                  6 weeks                42         42         70 Intramuscular  N
fIPV-1                 6 weeks                42         42         70 Intradermal    N
Pentavalent-1          6 weeks                42         42         70 Intramuscular  N
RVV-1                  6 weeks                42         42         70 Oral           N
bOPV-1                 6 weeks                42         42         70 Oral           N
Pentavalent-2          10 weeks               70         70         98 Intramuscular  N
bOPV-2                 10 weeks               70         70         98 Oral           N
RVV-2                  10 weeks               70         70         98 Oral           N
Pentavalent-3          14 weeks               98         98        126 Intramuscular  N
RVV-3                  14 weeks               98         98        126 Oral           N
bOPV-3                 14 weeks               98         98        126 Oral           N
PCV-2                  14 weeks               98         98        126 Intramuscular  N
fIPV-2                 14 weeks               98         98        126 Intradermal    N
PCV Booster            9-11 months           270        270        359 Intramuscular  N
fIPV-3                 9-11 months           270        270        359 Intradermal    N
JE-1                   9-11 months           270        270        359 Subcutaneous   Y
MR-1                   9-11 months           270        270        359 Subcutaneous   N
Vitamin A (1st dose)   9-11 months           270        270        359 Oral           N
Vitamin A (2nd dose)   16-23 months          480        480        719 Oral           N
bOPV Booster           16-23 months          480        480        719 Oral           N
DPT Booster-1          16-23 months          480        480        719 Intramuscular  N
MR-2                   16-23 months          480        480        719 Subcutaneous   N
JE-2                   16-23 months          480        480        719 Subcutaneous   Y

26 rows selected.
```

Figure 3.4: The immunization schedule stored as data in vaccine_master

Two design points are worth stating. First, "date of birth is not in the future" cannot be a CHECK constraint because SYSDATE is not allowed in one, so a trigger enforces it. Second, the 14 indexes beyond the keys and unique constraints support the frequent access paths; the most important is `ix_inventory_fefo` on `(clinic_id, vaccine_id, expiry_date)`, which serves the first-expiry-first-out choice of a batch, and `ix_dose_status_sched` on `(status, scheduled_date)`, which serves the overdue and due-soon queries.

3.3 Schema Definition (DDL)

The complete script is `db/01_schema.sql` (Appendix A). Two of the tables are shown here to illustrate the constraints.

```
CREATE TABLE vaccine_master (
  vaccine_id      NUMBER GENERATED BY DEFAULT AS IDENTITY,
  vaccine_name    VARCHAR2(60)  NOT NULL,
  disease_covered VARCHAR2(150) NOT NULL,
  dose_number     NUMBER(2)     NOT NULL,
  age_label       VARCHAR2(30)  NOT NULL,
  min_age_days    NUMBER(5)     NOT NULL,
  due_age_days    NUMBER(5)     NOT NULL,
  max_age_days    NUMBER(5)     NOT NULL,
  route           VARCHAR2(30)  NOT NULL,
```

```
  is_regional     CHAR(1) DEFAULT 'N' NOT NULL,
  CONSTRAINT pk_vaccine_master   PRIMARY KEY (vaccine_id),
  CONSTRAINT uq_vaccine_dose     UNIQUE (vaccine_name, dose_number),
  CONSTRAINT ck_vaccine_dose_no  CHECK (dose_number >= 0),
  CONSTRAINT ck_vaccine_ages     CHECK (min_age_days >= 0
                                    AND min_age_days <= due_age_days
                                    AND due_age_days <= max_age_days),
  CONSTRAINT ck_vaccine_regional CHECK (is_regional IN ('Y','N'))
);

CREATE TABLE dose_records (
  record_id         NUMBER GENERATED BY DEFAULT AS IDENTITY,
  baby_id           NUMBER NOT NULL,
  vaccine_id        NUMBER NOT NULL,
  scheduled_date    DATE   NOT NULL,
  administered_date DATE,
  status            VARCHAR2(10) DEFAULT 'PENDING' NOT NULL,
  inventory_id      NUMBER,
  administered_by   NUMBER,
  is_late           CHAR(1) DEFAULT 'N' NOT NULL,
  remarks           VARCHAR2(500),
  CONSTRAINT pk_dose_records      PRIMARY KEY (record_id),
  CONSTRAINT uq_dose_baby_vaccine UNIQUE (baby_id, vaccine_id),
  CONSTRAINT ck_dose_status       CHECK (status IN ('PENDING','DONE','OVERDUE')),
  CONSTRAINT ck_dose_late         CHECK (is_late IN ('Y','N')),
  CONSTRAINT ck_dose_done_date    CHECK ((status = 'DONE' AND administered_date IS
NOT NULL)
                                     OR (status <> 'DONE' AND administered_date IS
NULL)),
  CONSTRAINT fk_dose_baby         FOREIGN KEY (baby_id)         REFERENCES newborns
(baby_id),
  CONSTRAINT fk_dose_vaccine      FOREIGN KEY (vaccine_id)      REFERENCES
vaccine_master (vaccine_id),
  CONSTRAINT fk_dose_inventory    FOREIGN KEY (inventory_id)    REFERENCES
vaccine_inventory (inventory_id),
  CONSTRAINT fk_dose_user         FOREIGN KEY (administered_by) REFERENCES
app_users (user_id)
);
```

3.4 Normalization

Unnormalized form. We started from the paper vaccination card. One sheet carries the baby (name, gender, date of birth, birth weight), the guardian (name, phone, e-mail, address), the clinic (name, city, phone) and then a repeating group of vaccine rows, each with vaccine name, disease, dose number, route, due date, given date, status, batch number, expiry date and the person who gave it:

```
CARD(baby_name, gender, dob, birth_weight,
     guardian_name, guardian_phone, guardian_email, address,
     clinic_name, clinic_city, clinic_phone,
     { vaccine_name, disease, dose_no, route, due_date, given_date,
       status, batch_no, expiry_date, given_by }*)
```

First normal form. The repeating group is removed: there is one row per baby per dose, with the key (baby, vaccine), and every column holds one value. The anomaly removed is the variable-length list inside one record.

Second normal form. The key is now composite, (baby_id, vaccine_id), and some columns depend on only part of it. baby_id → baby_name, gender, dob, weight, guardian_*, clinic_* moves to newborns. vaccine_id → vaccine_name, disease, dose_no, route, age windows moves to vaccine_master. What remains in dose_records (scheduled date, administered date, status, late flag, remarks, administering user) depends on the whole key. The update anomaly removed: a change to a vaccine's route or age window is made once and not on every baby's card.

Third normal form. Transitive dependencies are removed. baby_id → guardian_id → guardian name, phone, e-mail, address moves to guardians. baby_id → clinic_id → clinic name, city, phone moves to clinics. dose → inventory_id → batch number, expiry date, quantity moves to vaccine_inventory, one row per clinic, vaccine and batch. Login data forms app_users, with a role-dependent reference to a clinic or a guardian. The anomaly removed: a guardian's new phone number or a clinic's new address is stored once.

BCNF. We checked every non-trivial functional dependency. In each table the determinants are candidate keys, as listed below, so the schema is in BCNF.

Normal Form	Relations	Justification
1NF	card split into one row per baby and dose	Atomic values; no repeating groups
2NF	newborns, vaccine_master, dose_records	No partial dependency on the composite key (baby_id, vaccine_id)
3NF	guardians, clinics, vaccine_inventory, app_users	No transitive dependency: guardian, clinic and batch details depend on their own keys
BCNF	all ten tables	Every determinant is a candidate key: clinic_id/clinic_name; guardian_id/phone; user_id/username; baby_id; vaccine_id/(vaccine_name, dose_number); inventory_id/(clinic_id, vaccine_id, batch_no); record_id/(baby_id, vaccine_id); surrogate keys for alerts, audit_log and test_log

`is_regional` and `age_label` are stored in `vaccine_master`. Both depend only on `vaccine_id`, so they do not break third normal form. `age_label` is a descriptive label for the age band; the day counts remain the source of truth.

4. IMPLEMENTATION

All business rules live in the database: the scripts in `db/` are run in numeric order on an empty schema (`99_drop_all.sql` resets it, and every script can be run again) and the whole chain is run by `run_all.sql` (Figure 4.1). After compilation no object is invalid (Figure 4.2). The project contains five standalone functions, one standalone procedure, four packages, five triggers, one scheduler job and all four kinds of cursor. Errors are raised with `RAISE_APPLICATION_ERROR` in the range −20001 to −20099; the table of codes is at the top of `db/04_pkg_newborn.sql`. The language features are those of the Oracle PL/SQL Language Reference [3].



```
===== test_log =====

TC_CODE DESCRIPTION                                                    EXPECTED                                  ACTUAL                                    RESULT
------- ---------------------------------------------------------------- ----------------------------------------- ----------------------------------------- ------
TC-F1   calculate_due_date: DOB + 42-day vaccine (bOPV-1) gives DOB + 42 2026-08-14                               2026-08-14                                PASS
TC-F2   calculate_due_date with an unknown baby raises -20001          -20001                                    -20001                                    PASS
TC-F3   get_coverage_pct: 0 for a new baby, 3 of 24 doses done = 12.5  0 -> 12.5                                 0 -> 12.5                                 PASS
TC-F4   get_next_due_dose returns the earliest dose that is not DONE  2026-08-12|not DONE                       2026-08-12|not DONE                       PASS
TC-P1   register_newborn creates the baby and a full schedule (24; 26  24/26/baby=1                              24/26/baby=1                              PASS
        with 3E)

TC-P2   generate_schedule run twice more creates no duplicate doses    24 -> 24                                  24 -> 24                                  PASS
TC-P3   register_newborn with a future DOB fails and rolls back guardian -20003|guardians=0|babies=0             -20003|guardians=0|babies=0               PASS
        + baby

TC-P4   mark_dose_administered sets DONE and takes exactly 1 from stock DONE|stock 5 -> 4                        DONE|stock 5 -> 4                         PASS
TC-P5   marking an already DONE dose is rejected (-20005), stock       -20005|stock=4                            -20005|stock=4                            PASS
        unchanged

TC-P6   out-of-stock: error, dose stays PENDING, stock unchanged       -20009|PENDING|given=none|stock=0          -20009|PENDING|given=none|stock=0         PASS
TC-P7   administering before min_age_days (10-day-old, 6-week dose) is -20008|PENDING|stock=5                    -20008|PENDING|stock=5                    PASS
        rejected

TC-P8   dose given past max_age_days sets is_late = Y and raises a     DONE|late=Y|LATE_DOSE alerts=1 DONE|late=Y|LATE_DOSE alerts=1            PASS
        LATE_DOSE alert

TC-P9   FEFO: earliest non-expired batch is used, expired batches are  TEST-NEAR|expired batch               TEST-NEAR|expired batch                   PASS
        skipped                                                        untouched=10|expired only=-20009          untouched=10|expired only=-20009

TC-P10  reschedule_dose rejects DONE doses, past dates, out-of-window  DONE=-20010|PAST=-20011|WINDOW=-20012| DONE=-20010|PAST=-20011|WINDOW=-20012| PASS
        dates; accepts a valid one                                     OK=1                                      OK=1

TC-C1   refresh_overdue_status flips past-due PENDING doses to OVERDUE 8 -> 0; overdue=8                         8 -> 0; overdue=8                         PASS
TC-T1   guard trigger blocks changing a DONE dose back to PENDING      -20013|DONE                               -20013|DONE                               PASS
TC-T2   audit trigger records old status, new status and the acting user status=PENDING|status=DONE|nurse1       status=PENDING|status=DONE|nurse1         PASS
TC-T3   low-stock trigger fires once, when quantity first drops below  0,1,1                                    0,1,1                                     PASS
        the threshold

TC-T4   OVERDUE alerts: one per dose, none added by a second sweep     8,8,1                                     8,8,1                                     PASS
TC-U1   register_newborn creates the guardian's parent login (username test.guardian|1|PARENT                   test.guardian|1|PARENT                    PASS
        from the guardian name)

TC-U2   a second child of the same guardian does not create a second   test.guardian|none|logins=1               test.guardian|none|logins=1               PASS
        login

TC-U3   create_user adds a nurse for a clinic; usernames are unique    created NURSE at                          created NURSE at                          PASS
        ignoring case                                                  TEST_CLINIC|dup=-20018|next nurse name TEST_CLINIC|dup=-20018|next nurse name
                                                                       free=1                                    free=1

TC-U4   account rules: nurse needs a clinic, one login per guardian, no nurse-no-clinic=-20019|guardian-has-lo nurse-no-clinic=-20019|guardian-has-lo PASS
        admin via the app, valid hash                                  gin=-20017|admin-role=-20019|bad-hash= gin=-20017|admin-role=-20019|bad-hash=
                                                                       -20019                                    -20019

TC-U5   suggest_username: name-based, numbered when taken, accents     testu.zed|testu.zed2|testu.zed3|zoe.mu testu.zed|testu.zed2|testu.zed3|zoe.mu PASS
        folded, short names and length handled                         ller|parent|len=24                        ller|parent|len=24

TC-U6   guardian_exists answers Y/N for a phone number (same matching as before=N|known=Y|padded=Y|other=N       before=N|known=Y|padded=Y|other=N         PASS
        register_newborn)

TC-U7   a sibling joins the existing guardian and login; a new guardian sibling: no login, babies=2, logins=1, sibling: no login, babies=2, logins=1, PASS
        still needs a name and an address                              name kept=TEST_Guardian|new guardian   name kept=TEST_Guardian|new guardian
                                                                       without name=-20014|without           without name=-20014|without
                                                                       address=-20014                        address=-20014

TC-N1   update_newborn moves unfinished dose dates with the date of    late=N|bOPV-1 due +32|PENDING|dob         late=N|bOPV-1 due +32|PENDING|dob         PASS
        birth and recalculates late flags                              moved|name=TEST_N1_new                moved|name=TEST_N1_new

TC-N2   a date of birth that invalidates a given dose, or lies in the  -20020|dob unchanged|-20003             -20020|dob unchanged|-20003               PASS
        future, is refused

TC-N3   delete_newborn removes the baby, doses and alerts; stock comes restored=5|kept=4|left=0|audit=2         restored=5|kept=4|left=0|audit=2          PASS
        back only when asked

TC-N4   only an administrator can edit or delete a newborn; the change update=-20021|delete=-20021|intact=1|a update=-20021|delete=-20021|intact=1|a PASS
        is audited with the actor                                      udit by admin=1                           udit by admin=1

TC-R1   reporting REF cursors return the row counts the tables imply   overdue=31;upcoming=5;clinics=5;summar overdue=31;upcoming=5;clinics=5;summar PASS
                                                                       y=1;timeline=24                           y=1;timeline=24



RESULT     CASES
------ ----------
PASS           31

INFO
--------------------------------------------------------------------------------
leftover TEST_ rows (must be 0): 0
11_tests.sql complete.
```

Figure 4.1: Scripts 99 to 11 running in order on an empty schema

```
SQL> SELECT object_type, object_name, status
  2    FROM user_objects
  3   WHERE object_type IN ('FUNCTION', 'PROCEDURE', 'PACKAGE', 'PACKAGE BODY', 'TRIGGER')
  4     AND object_name NOT LIKE 'FIG\_%' ESCAPE '\'
  5   ORDER BY CASE object_type WHEN 'FUNCTION' THEN 1 WHEN 'PROCEDURE' THEN 2 WHEN 'PACKAGE' THEN 3 WHEN 'PACKAGE BODY' THEN 4 ELSE
5 END,
  6           object_name;

OBJECT_TYPE    OBJECT_NAME                    STATUS
-------------- ------------------------------ -------
FUNCTION       CALCULATE_DUE_DATE             VALID
FUNCTION       DOSE_LABEL                     VALID
FUNCTION       GET_AGE_TEXT                   VALID
FUNCTION       GET_COVERAGE_PCT               VALID
FUNCTION       GET_NEXT_DUE_DOSE              VALID
PROCEDURE      SEND_DUE_SOON_ALERTS           VALID
PACKAGE        ACCOUNT_PKG                    VALID
PACKAGE        NEWBORN_VACCINATION_PKG        VALID
PACKAGE        VACCINE_INVENTORY_PKG          VALID
PACKAGE        VAX_REPORTING_PKG              VALID
PACKAGE BODY   ACCOUNT_PKG                    VALID
PACKAGE BODY   NEWBORN_VACCINATION_PKG        VALID
PACKAGE BODY   VACCINE_INVENTORY_PKG          VALID
PACKAGE BODY   VAX_REPORTING_PKG              VALID
TRIGGER        TRG_DOSE_ALERTS                VALID
TRIGGER        TRG_DOSE_AUDIT                 VALID
TRIGGER        TRG_DOSE_STATUS_GUARD          VALID
TRIGGER        TRG_INVENTORY_LOW_STOCK        VALID
TRIGGER        TRG_NEWBORN_DOB_CHECK          VALID

19 rows selected.
```

Figure 4.2: PL/SQL functions, procedures, packages and triggers, all VALID

4.1 Functions

Five standalone functions (`db/03_functions.sql`) compute the values the rest of the system relies on.

Function	Returns	Purpose
`calculate_due_date(p_baby_id, p_vaccine_id)`	DATE	date of birth plus the vaccine's `due_age_days`; raises −20001 or −20002 if the baby or vaccine does not exist
`get_age_text(p_baby_id)`	VARCHAR2	readable age such as "3 months 12 days"
`get_coverage_pct(p_baby_id)`	NUMBER	doses given divided by doses scheduled, times 100
`get_next_due_dose(p_baby_id)`	NUMBER	`record_id` of the earliest dose that is not DONE, or NULL
`dose_label(p_vaccine_name, p_dose_number)`	VARCHAR2	display name such as "bOPV-1" or "Vitamin A (1st dose)"; deterministic, so it can be used inside SQL

`calculate_due_date` is called by `generate_schedule` for every dose of a new baby, and the other functions are called from the API queries that build the baby list, the profile and the dashboard.

```
CREATE OR REPLACE FUNCTION calculate_due_date (
  p_baby_id    IN NUMBER,
  p_vaccine_id IN NUMBER
) RETURN DATE
```

```
IS
  v_dob  newborns.date_of_birth%TYPE;
  v_days vaccine_master.due_age_days%TYPE;
BEGIN
  BEGIN
    SELECT date_of_birth INTO v_dob FROM newborns WHERE baby_id = p_baby_id;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20001, 'Baby not found: ' || NVL(TO_CHAR(p_baby_id),
'NULL'));
  END;
  BEGIN
    SELECT due_age_days INTO v_days FROM vaccine_master WHERE vaccine_id =
p_vaccine_id;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20002, 'Vaccine not found: ' ||
NVL(TO_CHAR(p_vaccine_id), 'NULL'));
  END;
  RETURN TRUNC(v_dob) + v_days;
END calculate_due_date;
```

Output of the four functions, including the error case, is in Figure 4.3.

```
SQL> DECLARE
  2    v_baby  newborns.baby_id%TYPE;
  3    v_vid   vaccine_master.vaccine_id%TYPE;
  4    v_next  NUMBER;
  5    v_label VARCHAR2(120);
  6    v_date  DATE;
  7  BEGIN
  8    SELECT baby_id INTO v_baby FROM newborns WHERE full_name = 'Kabir Nair';
  9    SELECT vaccine_id INTO v_vid FROM vaccine_master WHERE vaccine_name = 'bOPV' AND dose_number = 1;
 10    DBMS_OUTPUT.PUT_LINE('get_age_text(baby)               = ' || get_age_text(v_baby));
 11    DBMS_OUTPUT.PUT_LINE('calculate_due_date(baby, bOPV-1)   = ' || TO_CHAR(calculate_due_date(v_baby, v_vid), 'YYYY-MM-DD'));
 12    DBMS_OUTPUT.PUT_LINE('get_coverage_pct(baby)           = ' || get_coverage_pct(v_baby) || ' %');
 13    v_next := get_next_due_dose(v_baby);
 14    SELECT dose_label(v.vaccine_name, v.dose_number) || ', scheduled ' || TO_CHAR(d.scheduled_date, 'YYYY-MM-DD') || ', status ' |
| d.status
 15      INTO v_label
 16      FROM dose_records d JOIN vaccine_master v ON v.vaccine_id = d.vaccine_id WHERE d.record_id = v_next;
 17    DBMS_OUTPUT.PUT_LINE('get_next_due_dose(baby)          = record ' || v_next || '  (' || v_label || ')');
 18    BEGIN
 19      v_date := calculate_due_date(-1, v_vid);
 20    EXCEPTION WHEN OTHERS THEN
 21      DBMS_OUTPUT.PUT_LINE('calculate_due_date(-1, ...)       -> ' || SQLERRM);
 22    END;
 23  END;
 24  /
get_age_text(baby)               = 3 months 29 days
calculate_due_date(baby, bOPV-1)   = 2026-07-24
get_coverage_pct(baby)           = 45.8 %
get_next_due_dose(baby)          = record 1990  (Pentavalent-3, scheduled 2026-09-18, status OVERDUE)
calculate_due_date(-1, ...)       -> ORA-20001: Baby not found: -1

PL/SQL procedure successfully completed.
```

Figure 4.3: Output of the four standalone functions, including the error case

4.2 Procedures

`send_due_soon_alerts(p_days_ahead)` (`db/07_procedures.sql`) is the standalone procedure. It loops over the pending doses due from today up to `p_days_ahead` days ahead and inserts one `DUE_SOON` alert for each, skipping a dose that already has one, so the daily job never duplicates an alert. It sets a savepoint, commits on success and rolls back to the savepoint on failure; a negative number of days raises −20014.

```
CREATE OR REPLACE PROCEDURE send_due_soon_alerts (
  p_days_ahead IN NUMBER DEFAULT 7
```

```
)
IS
  CURSOR c_due (p_window NUMBER) IS
    SELECT d.record_id, d.baby_id, d.scheduled_date, n.clinic_id, n.full_name AS
baby_name,
           dose_label(v.vaccine_name, v.dose_number) AS dose_name
      FROM dose_records d
      JOIN newborns n       ON n.baby_id = d.baby_id
      JOIN vaccine_master v ON v.vaccine_id = d.vaccine_id
     WHERE d.status = 'PENDING'
       AND d.scheduled_date BETWEEN TRUNC(SYSDATE) AND TRUNC(SYSDATE) + p_window
     ORDER BY d.scheduled_date, d.record_id;
  v_created PLS_INTEGER := 0;
BEGIN
  SAVEPOINT sp_due_soon;
  IF p_days_ahead IS NULL OR p_days_ahead < 0 THEN
    RAISE_APPLICATION_ERROR(-20014, 'Days ahead must be zero or more');
  END IF;

  FOR r IN c_due(p_days_ahead) LOOP
    INSERT INTO alerts (baby_id, record_id, clinic_id, alert_type, message)
    SELECT r.baby_id, r.record_id, r.clinic_id, 'DUE_SOON',
           r.dose_name || ' for ' || r.baby_name || ' is due on ' ||
TO_CHAR(r.scheduled_date, 'YYYY-MM-DD')
      FROM dual
     WHERE NOT EXISTS (SELECT 1 FROM alerts a
                       WHERE a.record_id = r.record_id AND a.alert_type =
'DUE_SOON');
    v_created := v_created + SQL%ROWCOUNT;
  END LOOP;
  COMMIT;
  -- (exception handler: ROLLBACK TO sp_due_soon, re-raise -20xxx, wrap the rest as
-20099)
END send_due_soon_alerts;
```

The alerts raised by this procedure and by the triggers and sweeps are shown in Figure 4.4.

```
SQL> SELECT alert_type, COUNT(*) AS alerts, SUM(CASE WHEN is_read = 'N' THEN 1 END) AS unread
  2    FROM alerts GROUP BY alert_type ORDER BY alert_type;

ALERT_TYPE     ALERTS     UNREAD
---------- ---------- ----------
DUE_SOON            5          5
EXPIRY              1          1
LATE_DOSE           1          1
LOW_STOCK           1          1
OVERDUE            28         27

5 rows selected.
```

Figure 4.4: Alerts raised by triggers and sweeps

4.3 Packages

The application logic is grouped into four packages. The API calls only these packages.

Package (file)	Main members
newborn_vaccination_pkg (04)	register_newborn, generate_schedule, guardian_exists, is_dose_allowed, mark_dose_administered, reschedule_dose, refresh_overdue_status, update_newborn, delete_newborn, set_actor
vaccine_inventory_pkg (05)	restock, pick_batch, expiry_sweep
vax_reporting_pkg (06)	dashboard_summary, overdue_list, upcoming_list, clinic_coverage, baby_timeline (all return a SYS_REFCURSOR)
account_pkg (03b)	next_nurse_username, suggest_username, add_user, create_user

Registering a baby. register_newborn reuses a guardian found by phone number or inserts a new one (a new guardian needs a name and an address), inserts the baby, calls generate_schedule, and creates the guardian's parent login the first time, all in one transaction. If any step fails, including a date of birth in the future that the trigger rejects, the savepoint rolls everything back, so no half-registered family remains (Figure 4.5). The parent's username is built from the guardian's name by account_pkg.suggest_username (for example priya.sharma, with a number added if the name is taken). A second child of the same guardian creates no second login.

```
--- register_newborn: guardian + baby + schedule in one transaction
baby_id = 47, born 2026-08-02 (70 days ago), doses scheduled = 24
future date of birth  -> ORA-20003: Date of birth cannot be in the future
                         guardian rows left behind: 0  (whole transaction rolled back)
```

Figure 4.5: register_newborn creates 24 doses; a future date of birth rolls everything back

Recording a dose. mark_dose_administered is the core transaction of the system. It sets a savepoint first, so a failure at any step also releases the row lock, then proceeds as follows. It locks the dose row (FOR UPDATE) and reads the baby and the vaccine. It rejects a dose that is already DONE, a date before the date of birth or in the future, and a baby younger than the dose's min_age_days. It asks vaccine_inventory_pkg.pick_batch for the batch that expires first among those that are not expired and have stock (first-expiry-first-out), and raises −20009 if there is none. It reduces that batch by one, guarded by quantity > 0 so that a concurrent session taking the last dose makes this one fail cleanly. It marks the dose DONE with the batch and the nurse, sets is_late = 'Y' and inserts a LATE_DOSE alert when the baby is past max_age_days, and commits. Any error rolls back to the savepoint, so the dose and the stock can never disagree (Figure 4.6).

```
    -- 1. lock the dose row and read everything the checks need
    SELECT d.status, d.baby_id, d.vaccine_id, n.date_of_birth, n.full_name,
n.clinic_id,
           v.min_age_days, v.max_age_days, dose_label(v.vaccine_name,
v.dose_number)
      INTO v_status, v_baby_id, v_vaccine_id, v_dob, v_baby_name, v_baby_clinic,
           v_min_age, v_max_age, v_label
      FROM dose_records d
      JOIN newborns n       ON n.baby_id = d.baby_id
      JOIN vaccine_master v ON v.vaccine_id = d.vaccine_id
     WHERE d.record_id = p_record_id
       FOR UPDATE OF d.status;
    -- (checks: already DONE, before birth, in the future, younger than
min_age_days)

    -- 2. FEFO batch (raises -20009 when empty or expired)
    v_inventory := vaccine_inventory_pkg.pick_batch(p_clinic_id, v_vaccine_id);

    -- 3. deduct stock; the quantity > 0 guard catches a concurrent session taking
the last dose
    UPDATE vaccine_inventory
       SET quantity = quantity - 1
     WHERE inventory_id = v_inventory
       AND quantity > 0;
    IF SQL%ROWCOUNT = 0 THEN
      RAISE_APPLICATION_ERROR(c_err_no_stock, 'The last dose of the selected batch
was just used; please retry');
    END IF;

    UPDATE dose_records
       SET status = 'DONE', administered_date = v_date, inventory_id = v_inventory,
           administered_by = p_user_id, is_late = v_late
     WHERE record_id = p_record_id;
    -- (LATE_DOSE alert when v_age_days > v_max_age), then COMMIT;
  EXCEPTION
    WHEN OTHERS THEN
      v_code := SQLCODE;  v_msg := SQLERRM;
      ROLLBACK TO sp_administer;      -- 4. undo stock and dose together
      IF v_code BETWEEN -20999 AND -20000 THEN RAISE; END IF;
      RAISE_APPLICATION_ERROR(c_err_unexpected, 'Could not record the dose: ' ||
v_msg);
```

```
--- mark_dose_administered: BCG at Sunrise (FEFO batch, stock -1, late flag)
pick_batch (earliest valid expiry) = inventory 16, batch LOT-16A, stock before = 54
after: status = DONE, stock = 53 (change -1), is_late = Y (BCG max age is day 14, baby is 70 days old), LATE_DOSE alerts = 1
```

Figure 4.6: mark_dose_administered: FEFO batch, stock minus one, late flag, and the error paths

First-expiry-first-out. `pick_batch` returns the inventory row that expires first among the clinic's non-expired batches of that vaccine that still have stock; an expired batch is skipped even if it holds stock.

```
SELECT inventory_id
  INTO v_inv
  FROM (SELECT inventory_id
          FROM vaccine_inventory
         WHERE clinic_id = p_clinic_id
           AND vaccine_id = p_vaccine_id
           AND quantity > 0
           AND expiry_date >= TRUNC(SYSDATE)
```

```
        ORDER BY expiry_date, inventory_id)
WHERE ROWNUM = 1;
```

Other members. `reschedule_dose` moves a dose that is not DONE to a date that is today or later and inside its age window, and records the reason. `update_newborn` and `delete_newborn` are available to administrators only (error −20021 otherwise): a corrected date of birth moves the due dates of the unfinished doses and rebuilds the stale alerts, and it is refused (−20020) if a dose that was already given would fall before its minimum age; deleting a baby can return the given doses to stock. `restock` inserts a batch or adds to an existing one. `set_actor` stores the acting user in the session's client information so that the audit trigger knows who acted.

Usage example

```
EXEC newborn_vaccination_pkg.mark_dose_administered(:record_id, NULL, :clinic_id,
:user_id);
```

4.4 Triggers

Five triggers (`db/08_triggers.sql`) enforce the rules that must hold whichever program changes the data. None of them reads the table that fired it, which is how the mutating-table error (ORA-04091) is avoided; they read only other tables.

Trigger	Event, level	Rule
`trg_newborn_dob_check`	BEFORE INSERT OR UPDATE OF date_of_birth ON newborns, row	the date of birth must not be in the future (−20003)
`trg_dose_status_guard`	BEFORE UPDATE ON dose_records, row	a DONE dose can never go back to PENDING or OVERDUE (−20013); the administered date may not be before the date of birth or in the future (−20006, −20007); a PENDING dose whose date has passed becomes OVERDUE
`trg_dose_alerts`	AFTER UPDATE OF status ON dose_records, row, with a WHEN clause	inserts one OVERDUE alert when a dose becomes overdue, once per dose

Trigger	Event, level	Rule
trg_dose_audit	AFTER INSERT OR UPDATE OR DELETE ON dose_records, row	writes old and new values and the acting user to audit_log
trg_inventory_low_stock	AFTER UPDATE OF quantity ON vaccine_inventory, row, with a WHEN clause	inserts a LOW_STOCK alert when a batch crosses below its threshold, once while it stays low

```
CREATE OR REPLACE TRIGGER trg_dose_status_guard
BEFORE UPDATE ON dose_records
FOR EACH ROW
DECLARE
  v_dob newborns.date_of_birth%TYPE;
BEGIN
  IF :OLD.status = 'DONE' AND :NEW.status <> 'DONE' THEN
    RAISE_APPLICATION_ERROR(-20013, 'A dose that is DONE cannot be changed back to
' || :NEW.status);
  END IF;

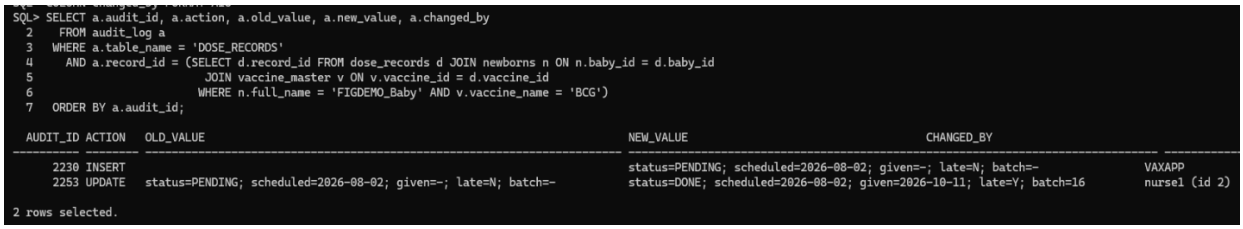
  IF :NEW.administered_date IS NOT NULL
     AND (:OLD.administered_date IS NULL OR :NEW.administered_date <>
:OLD.administered_date) THEN
    SELECT date_of_birth INTO v_dob FROM newborns WHERE baby_id = :NEW.baby_id;
    IF TRUNC(:NEW.administered_date) < TRUNC(v_dob) THEN
      RAISE_APPLICATION_ERROR(-20006, 'Administered date is before the date of
birth');
    ELSIF TRUNC(:NEW.administered_date) > TRUNC(SYSDATE) THEN
      RAISE_APPLICATION_ERROR(-20007, 'Administered date cannot be in the future');
    END IF;
  END IF;

  IF :NEW.status = 'PENDING' AND :NEW.scheduled_date < TRUNC(SYSDATE) THEN
    :NEW.status := 'OVERDUE';
  END IF;
END trg_dose_status_guard;
```

```
CREATE OR REPLACE TRIGGER trg_inventory_low_stock
AFTER UPDATE OF quantity ON vaccine_inventory
FOR EACH ROW
WHEN (NEW.quantity < NEW.low_stock_threshold AND OLD.quantity >=
OLD.low_stock_threshold)
DECLARE
  v_label VARCHAR2(100);
BEGIN
  SELECT dose_label(vaccine_name, dose_number) INTO v_label
    FROM vaccine_master WHERE vaccine_id = :NEW.vaccine_id;

  INSERT INTO alerts (clinic_id, alert_type, message)
  VALUES (:NEW.clinic_id, 'LOW_STOCK',
          'Low stock: batch ' || :NEW.batch_no || ' of ' || v_label || ' has ' ||
:NEW.quantity
          || ' dose(s) left (threshold ' || :NEW.low_stock_threshold || ')');
END trg_inventory_low_stock;
```

The guard trigger blocking a change from DONE to PENDING, and the audit row it leaves with the old status, the new status and the acting user, are shown in Figure 4.7; the audit rows are listed in Figure 4.8.

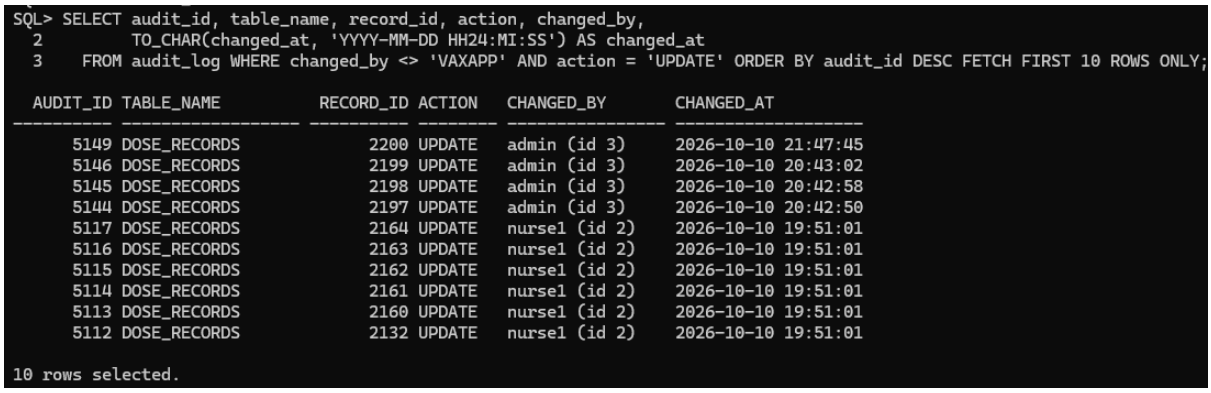


```
SQL> SELECT a.audit_id, a.action, a.old_value, a.new_value, a.changed_by
  2    FROM audit_log a
  3   WHERE a.table_name = 'DOSE_RECORDS'
  4     AND a.record_id = (SELECT d.record_id FROM dose_records d JOIN newborns n ON n.baby_id = d.baby_id
  5                          JOIN vaccine_master v ON v.vaccine_id = d.vaccine_id
  6                         WHERE n.full_name = 'FIGDEMO_Baby' AND v.vaccine_name = 'BCG')
  7   ORDER BY a.audit_id;

  AUDIT_ID ACTION   OLD_VALUE                                                      NEW_VALUE                                  CHANGED_BY
---------- -------- ------------------------------------------------------------ ------------------------------------------ ----------
      2230 INSERT                                                                status=PENDING; scheduled=2026-08-02; given=-; late=N; batch=-  VAXAPP
      2253 UPDATE   status=PENDING; scheduled=2026-08-02; given=-; late=N; batch=- status=DONE; scheduled=2026-08-02; given=2026-10-11; late=Y; batch=16  nurse1 (id 2)

2 rows selected.
```

Figure 4.7: Status-guard trigger blocking DONE to PENDING, and the audit row with the actor



```
SQL> SELECT audit_id, table_name, record_id, action, changed_by,
  2         TO_CHAR(changed_at, 'YYYY-MM-DD HH24:MI:SS') AS changed_at
  3    FROM audit_log WHERE changed_by <> 'VAXAPP' AND action = 'UPDATE' ORDER BY audit_id DESC FETCH FIRST 10 ROWS ONLY;

  AUDIT_ID TABLE_NAME          RECORD_ID ACTION   CHANGED_BY      CHANGED_AT
---------- ------------------ ---------- -------- --------------- -------------------
      5149 DOSE_RECORDS             2200 UPDATE   admin (id 3)    2026-10-10 21:47:45
      5146 DOSE_RECORDS             2199 UPDATE   admin (id 3)    2026-10-10 20:43:02
      5145 DOSE_RECORDS             2198 UPDATE   admin (id 3)    2026-10-10 20:42:58
      5144 DOSE_RECORDS             2197 UPDATE   admin (id 3)    2026-10-10 20:42:50
      5117 DOSE_RECORDS             2164 UPDATE   nurse1 (id 2)   2026-10-10 19:51:01
      5116 DOSE_RECORDS             2163 UPDATE   nurse1 (id 2)   2026-10-10 19:51:01
      5115 DOSE_RECORDS             2162 UPDATE   nurse1 (id 2)   2026-10-10 19:51:01
      5114 DOSE_RECORDS             2161 UPDATE   nurse1 (id 2)   2026-10-10 19:51:01
      5113 DOSE_RECORDS             2160 UPDATE   nurse1 (id 2)   2026-10-10 19:51:01
      5112 DOSE_RECORDS             2132 UPDATE   nurse1 (id 2)   2026-10-10 19:51:01

10 rows selected.
```

Figure 4.8: audit_log rows written by the trigger

4.5 Cursors

The project uses every kind of PL/SQL cursor.

Kind	Where	Why a cursor is needed
Explicit cursor with a parameter	`generate_schedule`: `CURSOR c_vaccines (p_regional VARCHAR2)`, opened, fetched in a loop and closed	the schedule is built dose by dose from `vaccine_master`; Japanese encephalitis rows are included only when requested; the cursor is closed in the exception handler as well
Parameterized cursor in a FOR loop	`expiry_sweep`: `CURSOR c_expiring (p_window NUMBER)`; `send_due_soon_alerts`: `c_due (p_window)`	each batch or dose needs its own alert, and the window (days) is passed in
Cursor FOR loop over an inline query	`refresh_overdue_status`	each past-due pending dose is touched individually so that the row triggers fire

Kind	Where	Why a cursor is needed
REF cursor (`SYS_REFCURSOR`)	the five procedures of `vax_reporting_pkg`	the API receives the result set from the package and sends it as JSON
Implicit cursor	`SQL%ROWCOUNT` after the stock update and the alert inserts	detects that no row was changed

```
-- refresh_overdue_status: cursor FOR loop
FOR r IN (SELECT record_id
            FROM dose_records
           WHERE status = 'PENDING'
             AND scheduled_date < TRUNC(SYSDATE)) LOOP
  UPDATE dose_records SET status = 'PENDING'
   WHERE record_id = r.record_id AND status = 'PENDING';
  v_flipped := v_flipped + SQL%ROWCOUNT;
END LOOP;
```

A row-level trigger fires only when its row is touched, and nothing touches a dose merely because a date has passed. The loop therefore updates each past-due pending dose to its own value: the update itself changes nothing, but `trg_dose_status_guard` turns the status into OVERDUE and `trg_dose_alerts` raises the alert. The `AND status = 'PENDING'` condition makes the loop skip a dose that a nurse marked DONE after the cursor read it (Figure 4.9).

```
-- expiry_sweep: parameterized cursor
CURSOR c_expiring (p_window NUMBER) IS
  SELECT i.clinic_id, i.batch_no, i.expiry_date,
         dose_label(v.vaccine_name, v.dose_number) AS vaccine_label
    FROM vaccine_inventory i
    JOIN vaccine_master v ON v.vaccine_id = i.vaccine_id
   WHERE i.quantity > 0
     AND i.expiry_date <= TRUNC(SYSDATE) + p_window
   ORDER BY i.expiry_date, i.inventory_id;
```

```
SQL> DECLARE
  2    v_before NUMBER; v_after NUMBER; v_over NUMBER; v_alerts1 NUMBER; v_alerts2 NUMBER;
  3    v_baby NUMBER;
  4  BEGIN
  5    SELECT baby_id INTO v_baby FROM newborns WHERE full_name = 'FIGDEMO_Baby';
  6    SELECT COUNT(*) INTO v_before FROM dose_records WHERE baby_id = v_baby AND status = 'PENDING' AND scheduled_date < TRUNC(SYSDATE);
  7    newborn_vaccination_pkg.refresh_overdue_status;
  8    SELECT COUNT(*) INTO v_after FROM dose_records WHERE baby_id = v_baby AND status = 'PENDING' AND scheduled_date < TRUNC(SYSDATE);
  9    SELECT COUNT(*) INTO v_over FROM dose_records WHERE baby_id = v_baby AND status = 'OVERDUE';
 10    SELECT COUNT(*) INTO v_alerts1 FROM alerts WHERE baby_id = v_baby AND alert_type = 'OVERDUE';
 11    newborn_vaccination_pkg.refresh_overdue_status;
 12    SELECT COUNT(*) INTO v_alerts2 FROM alerts WHERE baby_id = v_baby AND alert_type = 'OVERDUE';
 13    DBMS_OUTPUT.PUT_LINE('past-due PENDING doses before the sweep = ' || v_before || ', after = ' || v_after || ', now OVERDUE = ' || v_over)
;
 14    DBMS_OUTPUT.PUT_LINE('OVERDUE alerts after sweep 1 = ' || v_alerts1 || ', after sweep 2 = ' || v_alerts2 || '  (no duplicates)');
 15  END;
 16  /
refresh_overdue_status: 27 dose(s) checked
refresh_overdue_status: 0 dose(s) checked
past-due PENDING doses before the sweep = 7, after = 0, now OVERDUE = 7
OVERDUE alerts after sweep 1 = 7, after sweep 2 = 7  (no duplicates)

PL/SQL procedure successfully completed.
```

Figure 4.9: Overdue sweep flipping past-due doses; one alert per dose

The REF cursors of the reporting package return the dashboard summary and the clinic coverage (Figure 4.10) and a baby's timeline with its colour bucket and batch (Figure 4.11). `clinic_coverage` aggregates per clinic and then ranks the clinics with the analytic function `RANK() OVER (ORDER BY coverage_pct DESC)`.

```
SQL> EXEC vax_reporting_pkg.dashboard_summary(NULL, :cur)

PL/SQL procedure successfully completed.

SQL> PRINT cur

TOTAL_BABIES TOTAL_DOSES DOSES_DONE DOSES_PENDING DOSES_OVERDUE COVERAGE_PCT UNREAD_ALERTS
------------ ----------- ---------- ------------- ------------- ------------ -------------
          12         292         99           165            28         33.9            35

1 row selected.

SQL> PROMPT -- clinic_coverage (aggregate + RANK analytic)
-- clinic_coverage (aggregate + RANK analytic)
SQL> EXEC vax_reporting_pkg.clinic_coverage(:cur)

PL/SQL procedure successfully completed.

SQL> PRINT cur

 CLINIC_ID CLINIC_NAME                CITY        TOTAL_BABIES TOTAL_DOSES DOSES_DONE DOSES_OVERDUE COVERAGE_PCT COVERAGE_RANK
---------- -------------------------- ----------- ------------ ----------- ---------- ------------- ------------ -------------
         1 CityCare Pediatric Centre  Chennai                3          72         40             8         55.6             1
         3 Green Valley Immunization C Coimbatore             1          24          8             8         33.3             2
           entre

         4 Sunrise Child Health Clinic Vellore               7         172         48             7         27.9             3
         2 Lotus Mother and Baby Clini Bengaluru             1          24          3             5         12.5             4
           c


4 rows selected.

SQL> PROMPT -- upcoming_list(NULL, 7)
-- upcoming_list(NULL, 7)
SQL> EXEC vax_reporting_pkg.upcoming_list(NULL, 7, :cur)

PL/SQL procedure successfully completed.

SQL> PRINT cur

 RECORD_ID    BABY_ID BABY_NAME       GUARDIAN_NAME  GUARDIAN_PH CLINIC_NAME                DOSE_NAME             SCHEDULED DAYS_UNTIL_DUE
---------- ---------- --------------- -------------- ----------- -------------------------- --------------------- --------- --------------
      2054         89 Sara Khan       Farah Khan     9876500006  Sunrise Child Health Clinic PCV-1                 14-OCT-26              3
      2055         89 Sara Khan       Farah Khan     9876500006  Sunrise Child Health Clinic fIPV-1                14-OCT-26              3
      2056         89 Sara Khan       Farah Khan     9876500006  Sunrise Child Health Clinic Pentavalent-1         14-OCT-26              3
      2057         89 Sara Khan       Farah Khan     9876500006  Sunrise Child Health Clinic RVV-1                 14-OCT-26              3
      2058         89 Sara Khan       Farah Khan     9876500006  Sunrise Child Health Clinic bOPV-1                14-OCT-26              3

5 rows selected.

SQL> PROMPT -- overdue_list(Sunrise clinic)
-- overdue_list(Sunrise clinic)
SQL> VARIABLE clinic NUMBER
SQL> BEGIN SELECT clinic_id INTO :clinic FROM clinics WHERE clinic_name = 'Sunrise Child Health Clinic'; END;
  2  /

PL/SQL procedure successfully completed.

SQL> EXEC vax_reporting_pkg.overdue_list(:clinic, :cur)

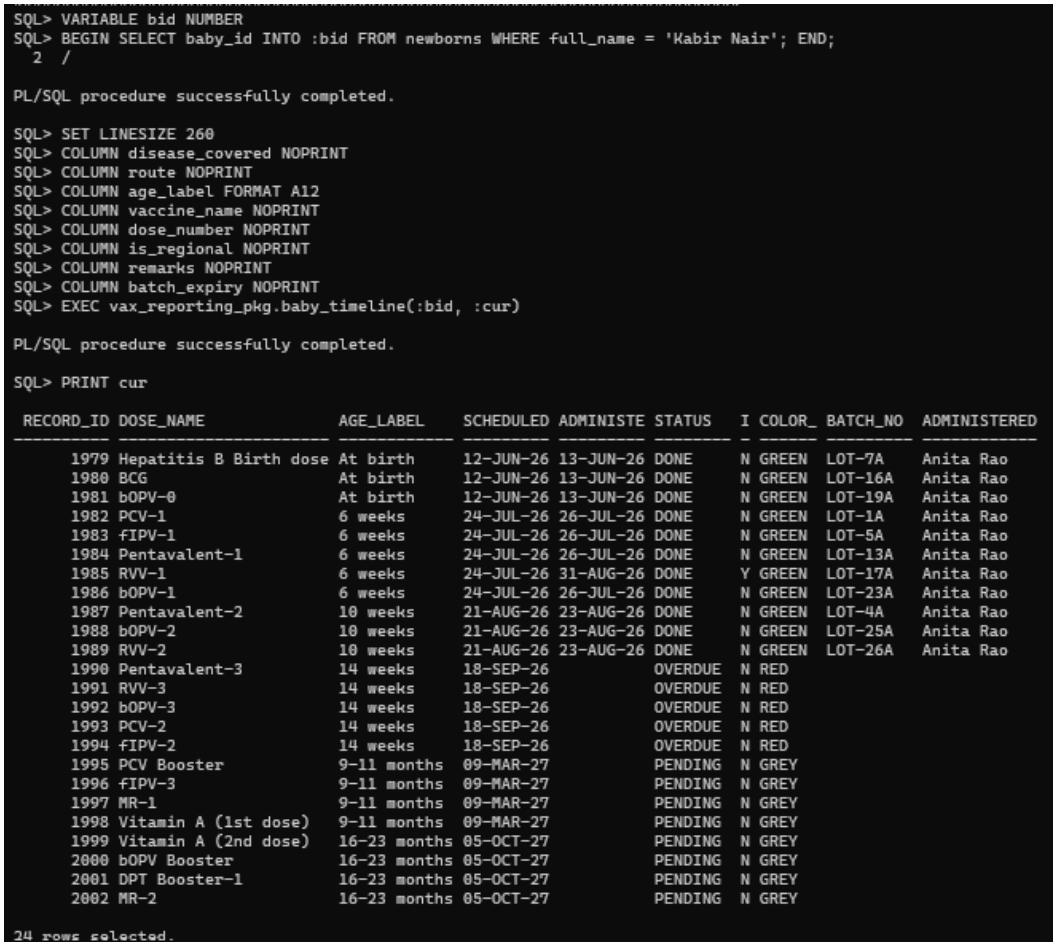
PL/SQL procedure successfully completed.

SQL> PRINT cur

 RECORD_ID    BABY_ID BABY_NAME       GUARDIAN_NAME  GUARDIAN_PH CLINIC_NAME                DOSE_NAME             SCHEDULED DAYS_OVERDUE
---------- ---------- --------------- -------------- ----------- -------------------------- --------------------- --------- ------------
      1990         86 Kabir Nair      Suresh Nair    9876500003  Sunrise Child Health Clinic Pentavalent-3         18-SEP-26           23
      1991         86 Kabir Nair      Suresh Nair    9876500003  Sunrise Child Health Clinic RVV-3                 18-SEP-26           23
      1992         86 Kabir Nair      Suresh Nair    9876500003  Sunrise Child Health Clinic bOPV-3                18-SEP-26           23
      1993         86 Kabir Nair      Suresh Nair    9876500003  Sunrise Child Health Clinic PCV-2                 18-SEP-26           23
      1994         86 Kabir Nair      Suresh Nair    9876500003  Sunrise Child Health Clinic fIPV-2                18-SEP-26           23
      2027         88 Rohan Das       Anil Das       9876500005  Sunrise Child Health Clinic Hepatitis B Birth dose 30-SEP-26           11
      2029         88 Rohan Das       Anil Das       9876500005  Sunrise Child Health Clinic bOPV-0                30-SEP-26           11

7 rows selected.
```

Figure 4.10: REF CURSOR output: dashboard summary and clinic coverage



```
SQL> VARIABLE bid NUMBER
SQL> BEGIN SELECT baby_id INTO :bid FROM newborns WHERE full_name = 'Kabir Nair'; END;
  2  /

PL/SQL procedure successfully completed.

SQL> SET LINESIZE 260
SQL> COLUMN disease_covered NOPRINT
SQL> COLUMN route NOPRINT
SQL> COLUMN age_label FORMAT A12
SQL> COLUMN vaccine_name NOPRINT
SQL> COLUMN dose_number NOPRINT
SQL> COLUMN is_regional NOPRINT
SQL> COLUMN remarks NOPRINT
SQL> COLUMN batch_expiry NOPRINT
SQL> EXEC vax_reporting_pkg.baby_timeline(:bid, :cur)

PL/SQL procedure successfully completed.

SQL> PRINT cur

 RECORD_ID DOSE_NAME              AGE_LABEL    SCHEDULED ADMINISTE STATUS   I COLOR_ BATCH_NO  ADMINISTERED
---------- ---------------------- ------------ --------- --------- -------- - ------ --------- ------------
      1979 Hepatitis B Birth dose At birth     12-JUN-26 13-JUN-26 DONE     N GREEN  LOT-7A    Anita Rao
      1980 BCG                    At birth     12-JUN-26 13-JUN-26 DONE     N GREEN  LOT-16A   Anita Rao
      1981 bOPV-0                 At birth     12-JUN-26 13-JUN-26 DONE     N GREEN  LOT-19A   Anita Rao
      1982 PCV-1                  6 weeks      24-JUL-26 26-JUL-26 DONE     N GREEN  LOT-1A    Anita Rao
      1983 fIPV-1                 6 weeks      24-JUL-26 26-JUL-26 DONE     N GREEN  LOT-5A    Anita Rao
      1984 Pentavalent-1          6 weeks      24-JUL-26 26-JUL-26 DONE     N GREEN  LOT-13A   Anita Rao
      1985 RVV-1                  6 weeks      24-JUL-26 31-AUG-26 DONE     Y GREEN  LOT-17A   Anita Rao
      1986 bOPV-1                 6 weeks      24-JUL-26 26-JUL-26 DONE     N GREEN  LOT-23A   Anita Rao
      1987 Pentavalent-2          10 weeks     21-AUG-26 23-AUG-26 DONE     N GREEN  LOT-4A    Anita Rao
      1988 bOPV-2                 10 weeks     21-AUG-26 23-AUG-26 DONE     N GREEN  LOT-25A   Anita Rao
      1989 RVV-2                  10 weeks     21-AUG-26 23-AUG-26 DONE     N GREEN  LOT-26A   Anita Rao
      1990 Pentavalent-3          14 weeks     18-SEP-26           OVERDUE  N RED
      1991 RVV-3                  14 weeks     18-SEP-26           OVERDUE  N RED
      1992 bOPV-3                 14 weeks     18-SEP-26           OVERDUE  N RED
      1993 PCV-2                  14 weeks     18-SEP-26           OVERDUE  N RED
      1994 fIPV-2                 14 weeks     18-SEP-26           OVERDUE  N RED
      1995 PCV Booster            9-11 months  09-MAR-27           PENDING  N GREY
      1996 fIPV-3                 9-11 months  09-MAR-27           PENDING  N GREY
      1997 MR-1                   9-11 months  09-MAR-27           PENDING  N GREY
      1998 Vitamin A (1st dose)   9-11 months  09-MAR-27           PENDING  N GREY
      1999 Vitamin A (2nd dose)   16-23 months 05-OCT-27           PENDING  N GREY
      2000 bOPV Booster           16-23 months 05-OCT-27           PENDING  N GREY
      2001 DPT Booster-1          16-23 months 05-OCT-27           PENDING  N GREY
      2002 MR-2                   16-23 months 05-OCT-27           PENDING  N GREY

24 rows selected.
```

Figure 4.11: REF CURSOR output: a baby's timeline with colour bucket and batch

4.6 Scheduled Job

`db/09_scheduler.sql` creates the DBMS_SCHEDULER job `JOB_DAILY_VAX_SWEEP`, which runs every day at 01:00 (Oracle Scheduler [5]). It calls `refresh_overdue_status`, `send_due_soon_alerts(7)` and `expiry_sweep(30)`. The API also runs `refresh_overdue_status` when the dashboard loads, so overdue doses are current even between two runs of the job (Figure 4.12).

```
DBMS_SCHEDULER.CREATE_JOB(
  job_name        => 'JOB_DAILY_VAX_SWEEP',
  job_type        => 'PLSQL_BLOCK',
  job_action      => 'BEGIN
                        newborn_vaccination_pkg.refresh_overdue_status;
                        send_due_soon_alerts(7);
                        vaccine_inventory_pkg.expiry_sweep(30);
                      END;',
  start_date      => SYSTIMESTAMP,
  repeat_interval => 'FREQ=DAILY;BYHOUR=1;BYMINUTE=0;BYSECOND=0',
  enabled         => TRUE);
```

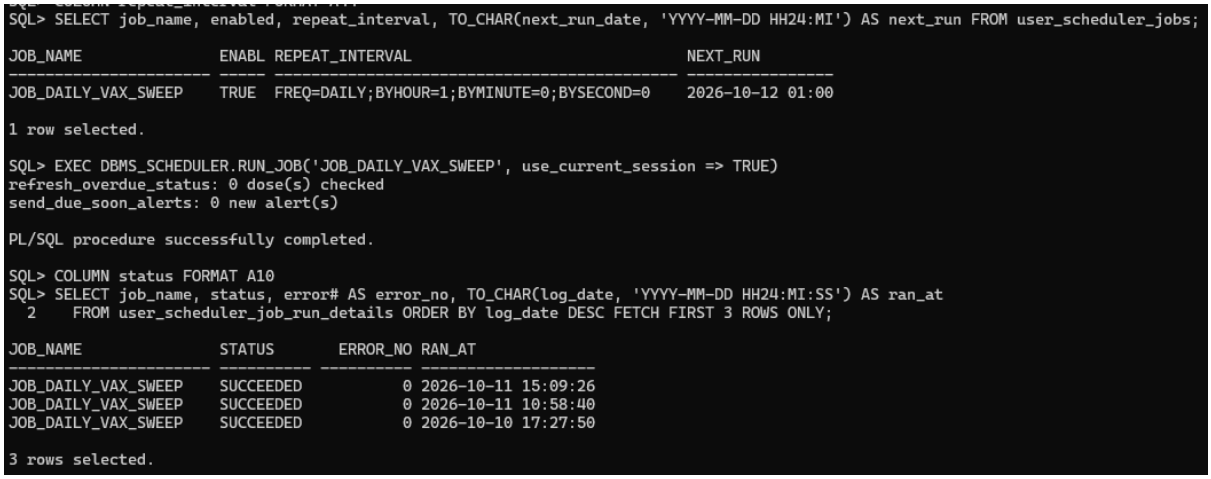


Figure 4.12: DBMS_SCHEDULER job definition and a successful run

4.7 Transactions and Error Handling

Every unit of work that changes data (`register_newborn`, `mark_dose_administered`, `reschedule_dose`, `refresh_overdue_status`, the two administrator procedures, the sweeps) has the same shape: set a savepoint, do the work, commit, and on any exception roll back to the savepoint, then re-raise a coded error (−20001 to −20021) or wrap an unexpected Oracle error as −20099 with the original message appended. `generate_schedule` does not commit, because it is a building block of `register_newborn`. The API turns each −20xxx code into a clean HTTP status and message for the user interface.

5. FRONT-END

The front end is a responsive web application written with React and Vite. It serves three roles: the **administrator** sees every clinic and the audit log and manages logins, a **nurse** works on the babies and the stock of one clinic, and a **parent** sees only their own children and their vaccination cards. Status colours are the same on every screen: green for a dose given, amber for a dose due soon, red for an overdue dose, grey for a dose in the future. Light and dark themes are both available.

5.1 User Interface

Sign-in. The sign-in page (Figure 5.1) lists the demo accounts with their passwords and a button that shows or hides what is typed in the password box.

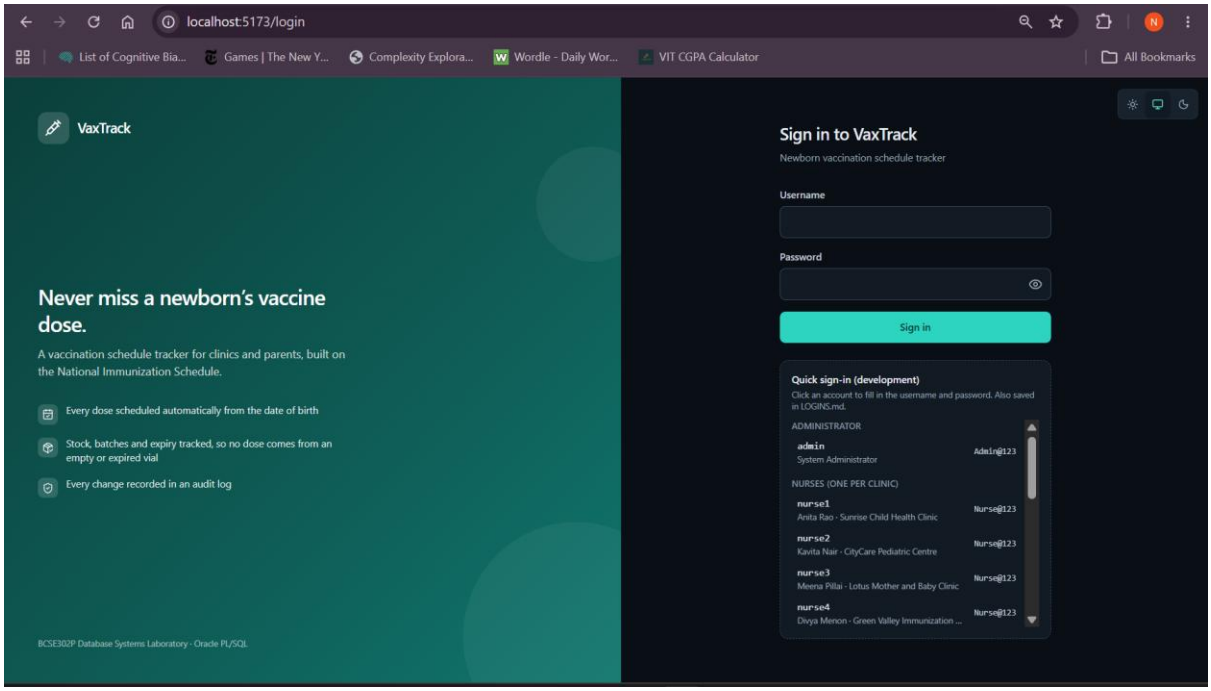


Figure 5.1: Sign-in page

Dashboard. The dashboard (Figure 5.2) shows the number of babies and of doses given, pending and overdue, a coverage chart, the overdue doses, the doses due within seven days, and the number of unread alerts. Figure 5.3 shows the same page in the light theme.

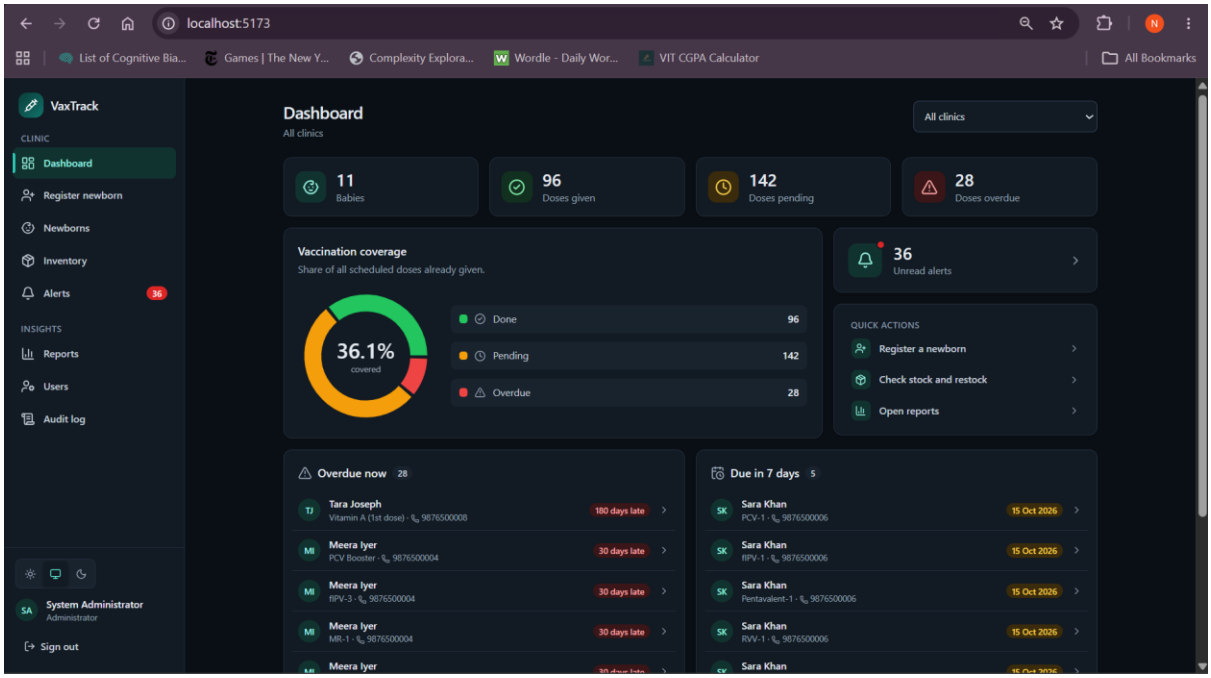


Figure 5.2: Dashboard with coverage chart, overdue list and unread alerts

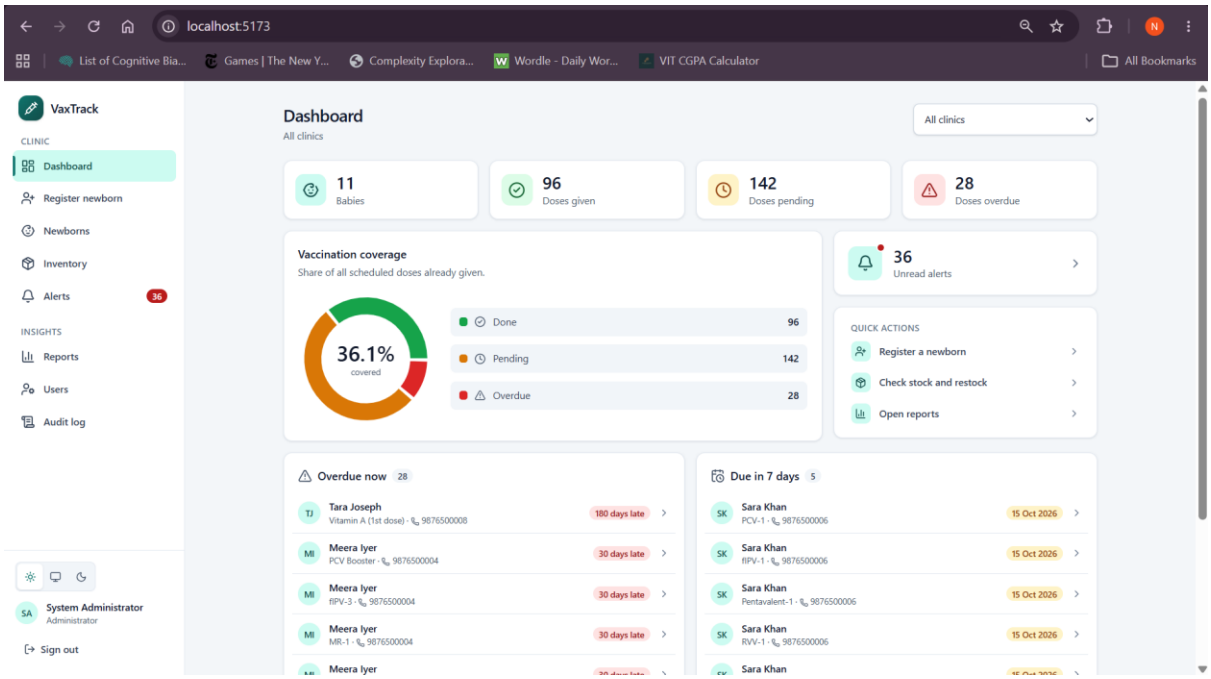


Figure 5.3: The same dashboard in light mode

Registering a newborn. The form (Figure 5.4) validates the input before it is sent and has an "Endemic district (JE)" box that adds the two Japanese encephalitis doses. The phone field tells the nurse whether the number already belongs to a registered guardian, answering only yes or no and showing no detail about the guardian (Figure 5.5). A new guardian needs a name and an address; for a known number the stored details are kept. When the baby is saved, the

generated schedule appears at once (Figure 5.7), and a card shows the parent login that was created, with a button to copy it.

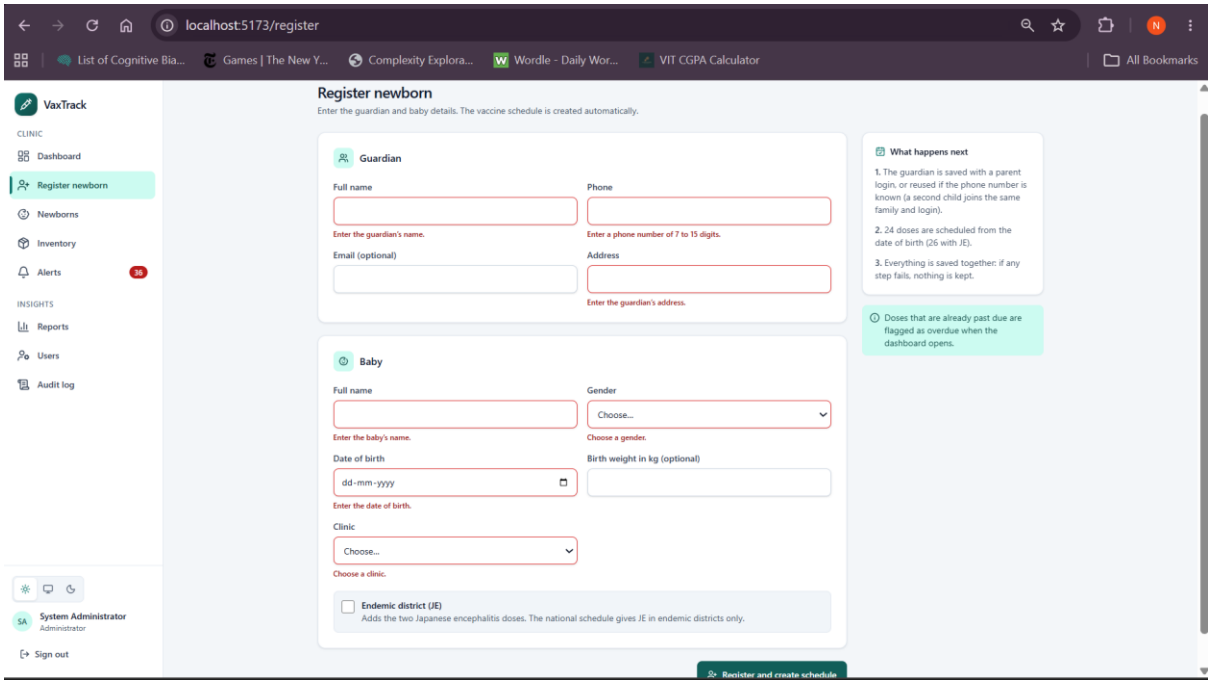


Figure 5.4: Register newborn form showing validation messages

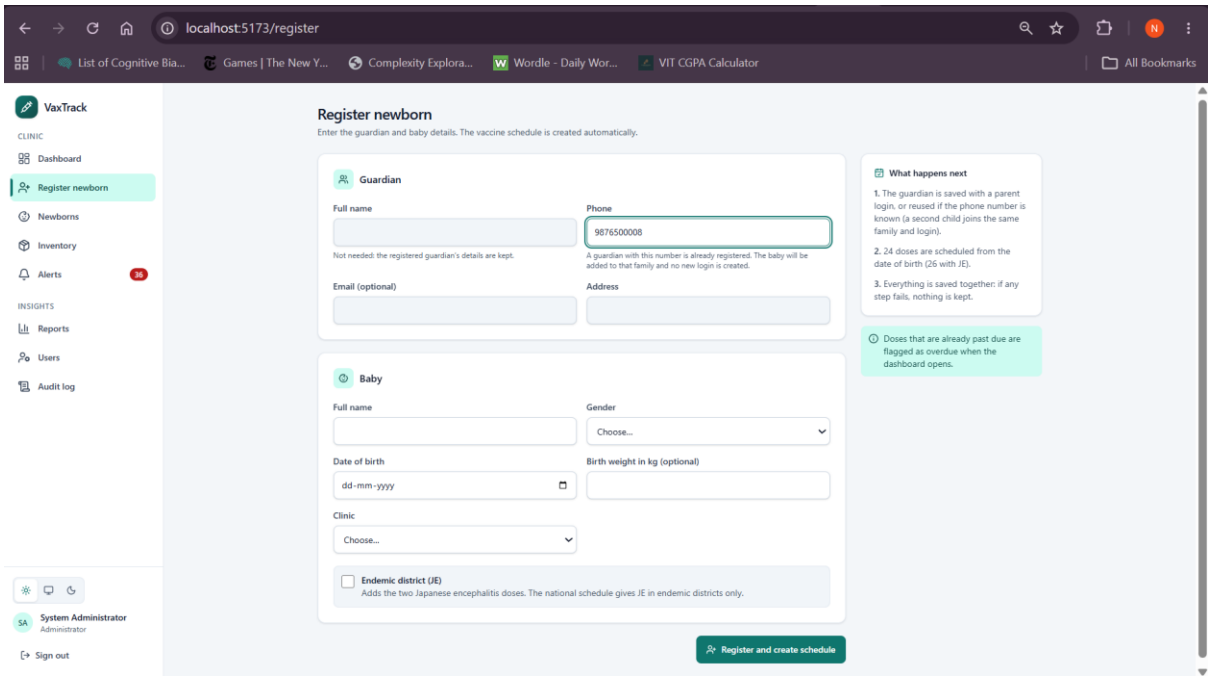


Figure 5.5: Register form: the phone number belongs to a registered guardian (yes/no only, guardian fields disabled)

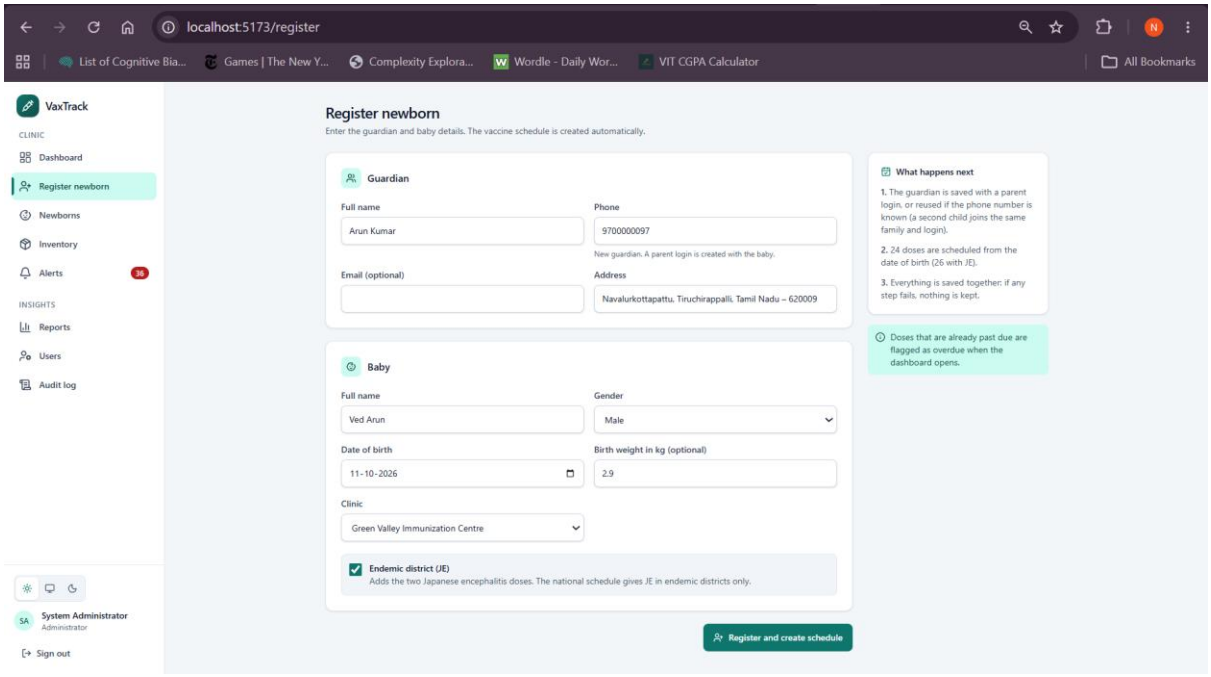


Figure 5.6: Registering a newborn

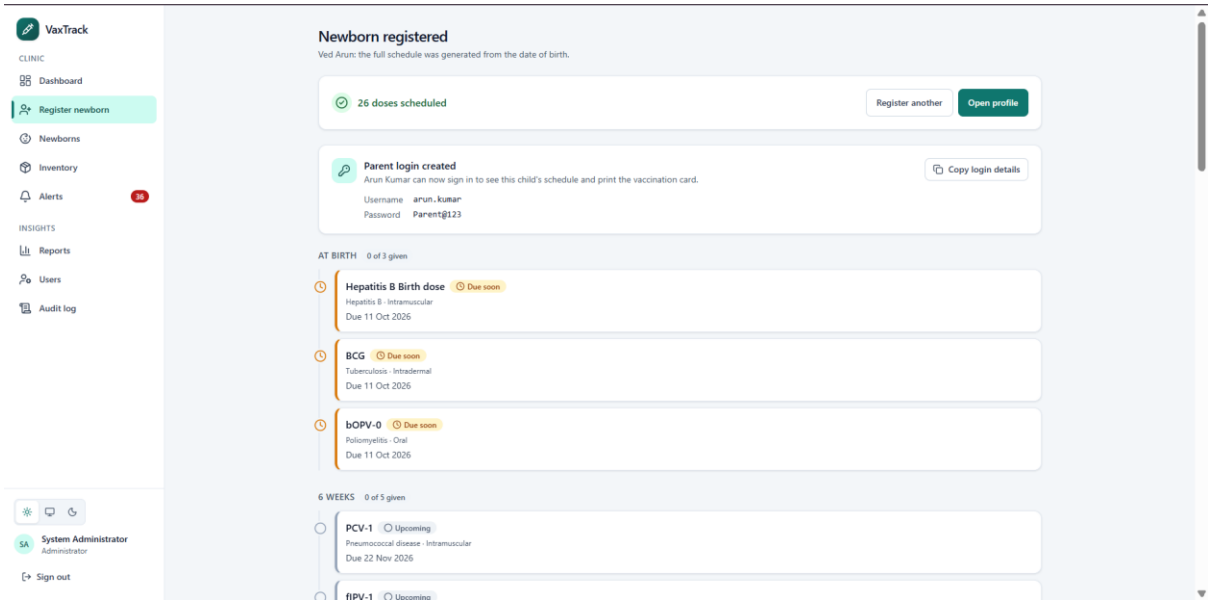


Figure 5.7: Generated 24-dose schedule shown right after registration

Baby profile. The profile (Figure 5.8) shows the age, the coverage, a bar of doses given, pending and overdue, and a vertical timeline grouped by age band, with a status filter. Each dose has actions for a nurse or an administrator. *Mark administered* opens a dialog (Figure 5.9) that shows which batch the database will use; if the clinic has no usable stock the dialog says so and the confirm button is disabled (Figure 5.10). *Reschedule* (Figure 5.11) asks for a new date and a reason.

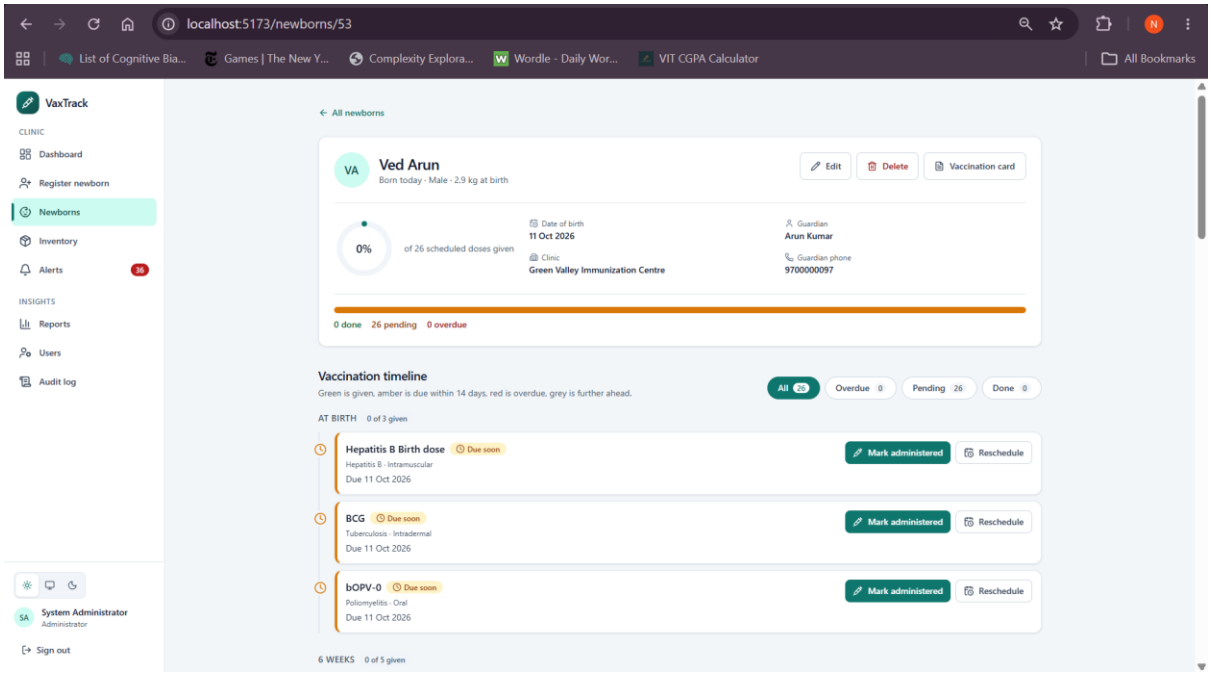


Figure 5.8: Baby profile: coverage, status bar and colour-coded timeline

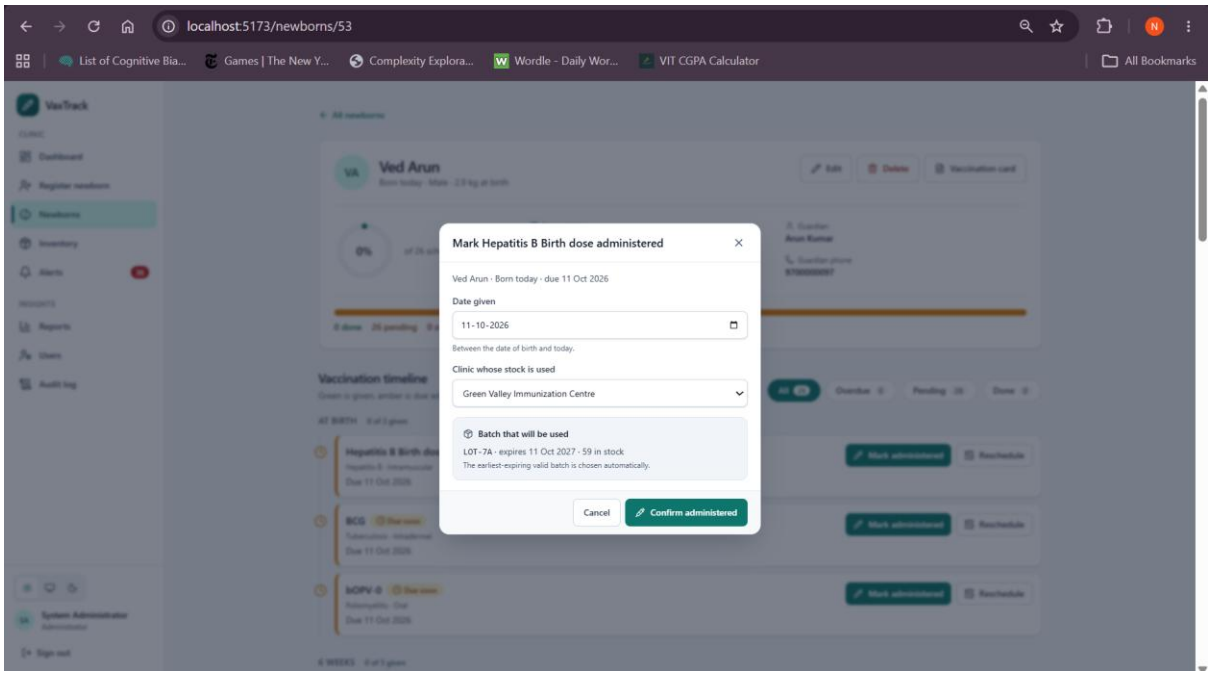


Figure 5.9: Mark administered dialog with the automatically chosen batch

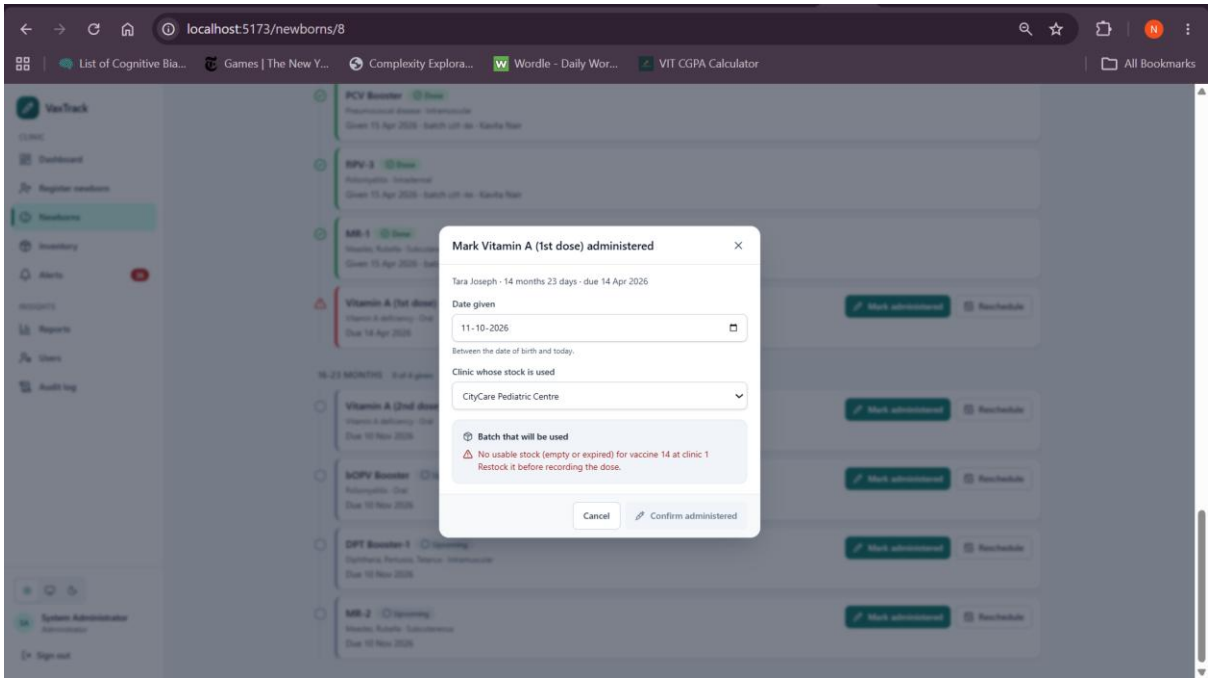


Figure 5.10: Mark administered dialog when the clinic has no stock (Confirm disabled)

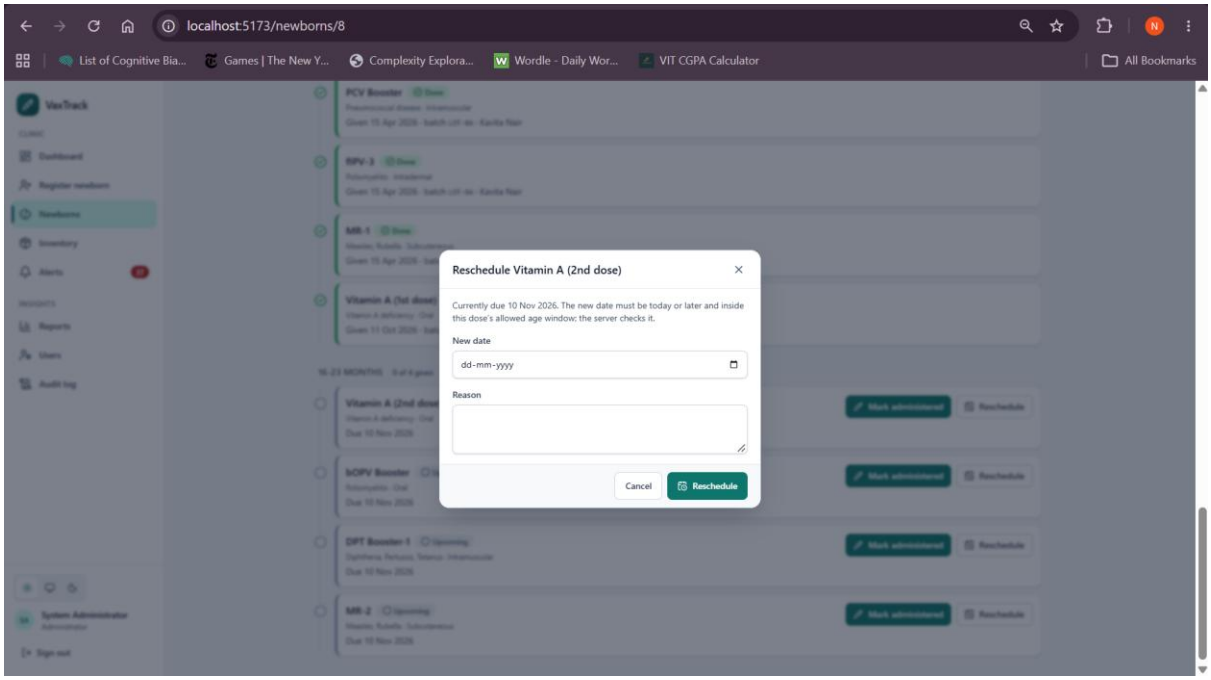


Figure 5.11: Reschedule dialog

Inventory and alerts. The inventory page (Figure 5.12) lists the batches with expiry and low-stock badges, and *Restock* (Figure 5.13) adds a batch. The alerts page (Figure 5.14) filters by type and marks alerts as read.

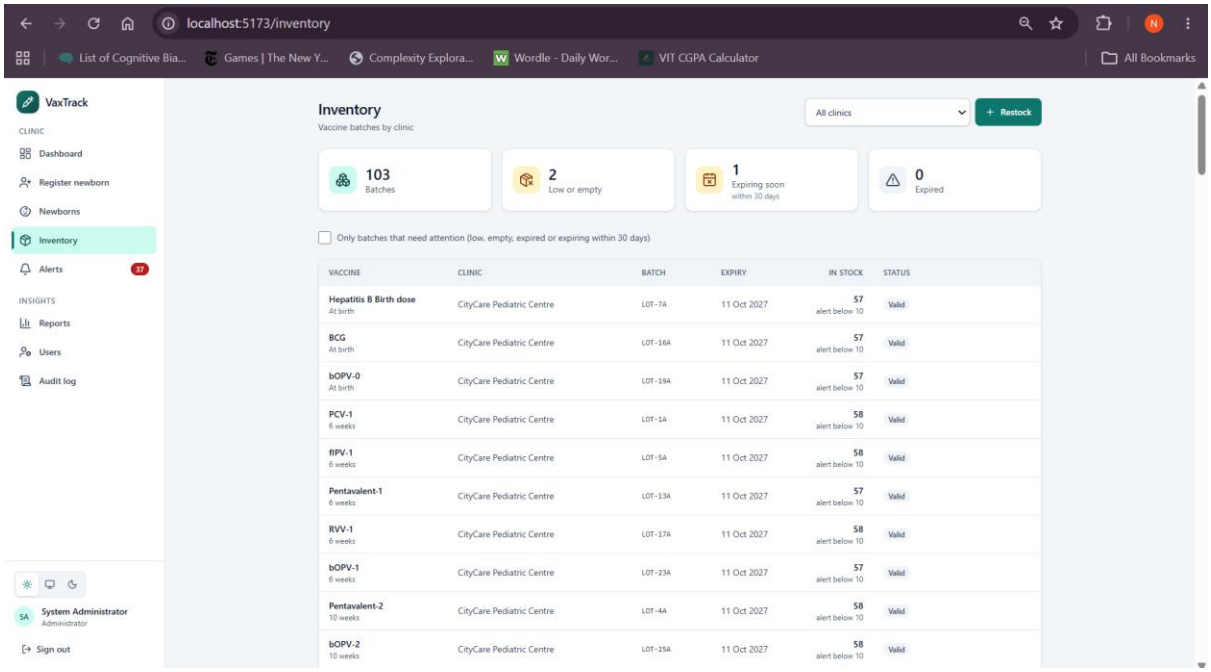


Figure 5.12: Inventory with a low-stock batch and expiry badges

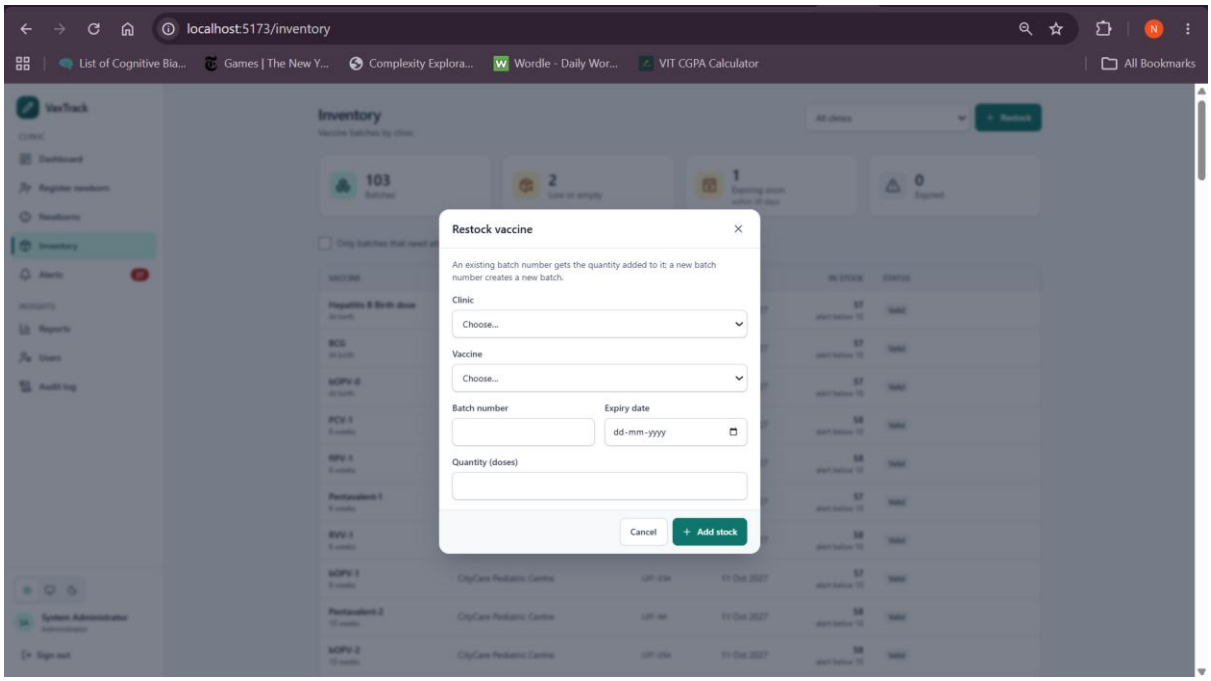


Figure 5.13: Restock dialog

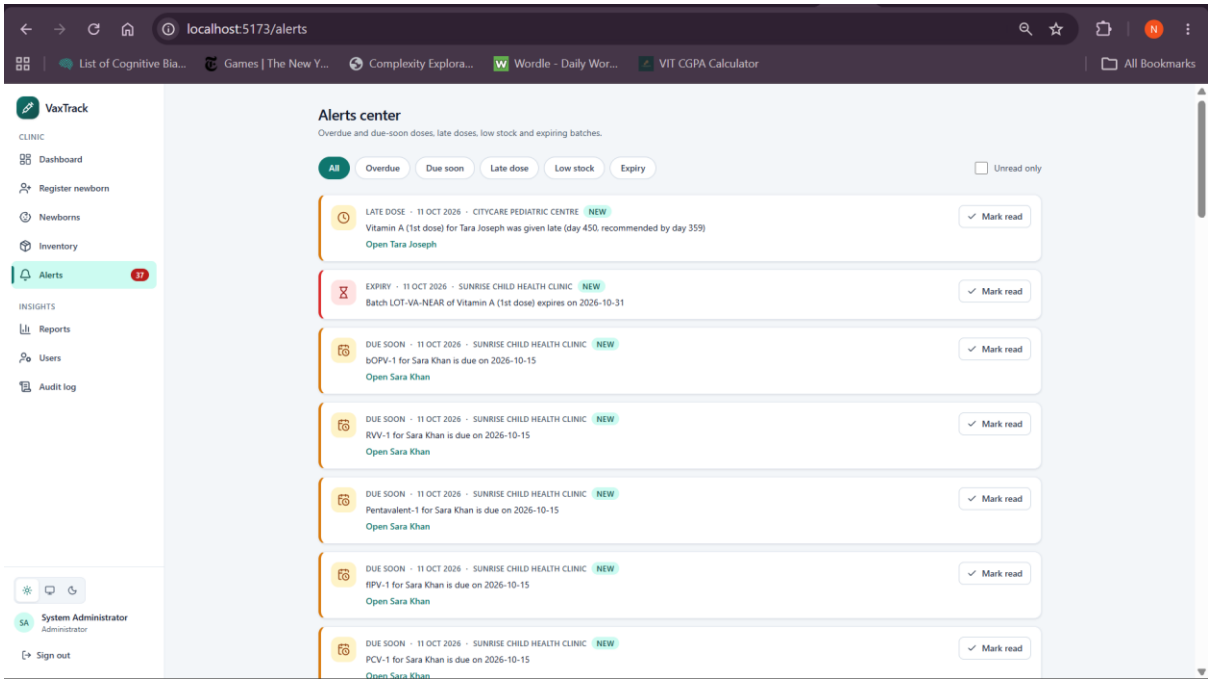


Figure 5.14: Alerts center filtered to low-stock alerts

Reports, vaccination card and parent view. Reports (Figure 5.15) compares the clinics' coverage and lists the overdue doses, with a CSV export. The vaccination card (Figure 5.16) is a printable page that a parent can open. A parent sees only their own children (Figure 5.17).

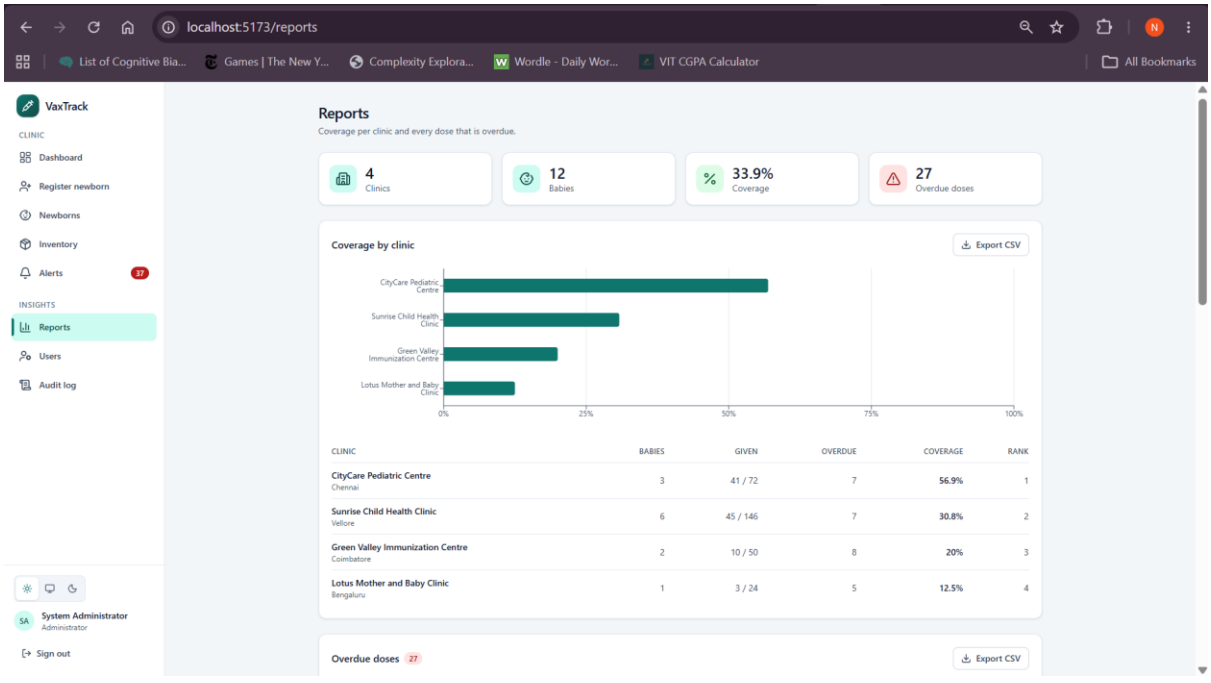


Figure 5.15: Reports: coverage by clinic chart, table and overdue list

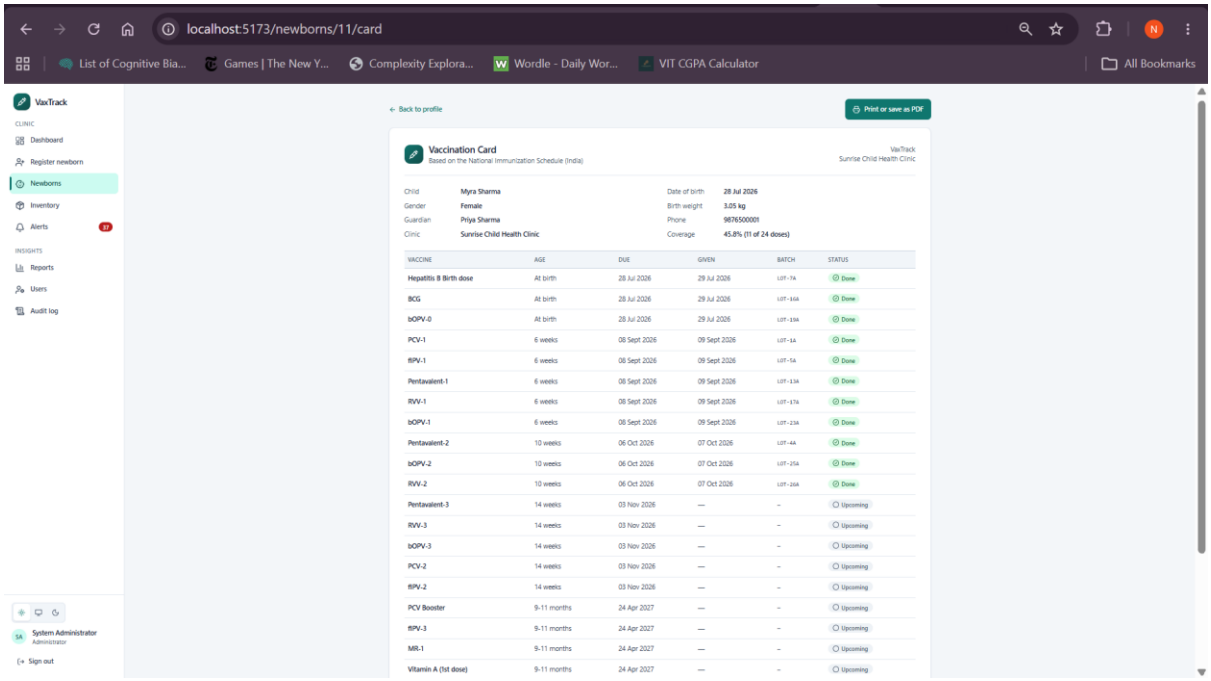


Figure 5.16: Printable vaccination card (parent view)

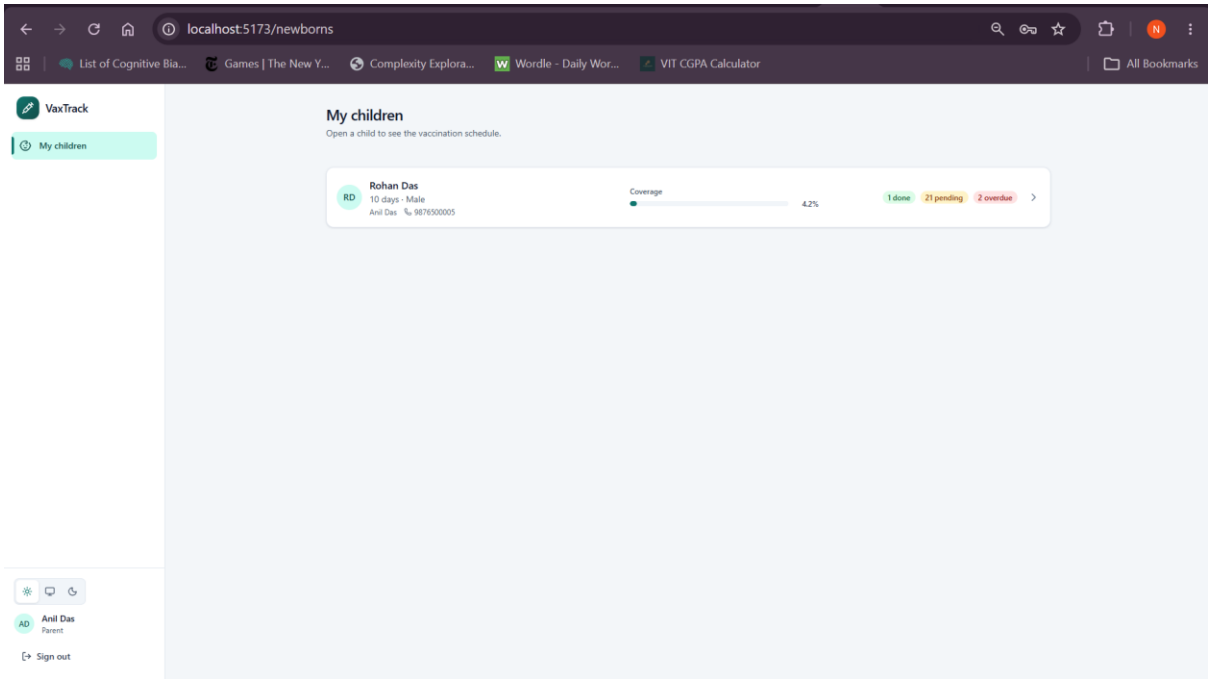


Figure 5.17: Parent sees only their own child

Administrator pages. The audit log (Figure 5.18) shows who changed which dose and when. The Users page (Figure 5.19) lists every login and adds nurses or parent logins. An administrator can also correct a newborn's date of birth (Figure 5.20), which moves the unfinished doses, or delete a newborn record after typing the baby's name (Figure 5.21).

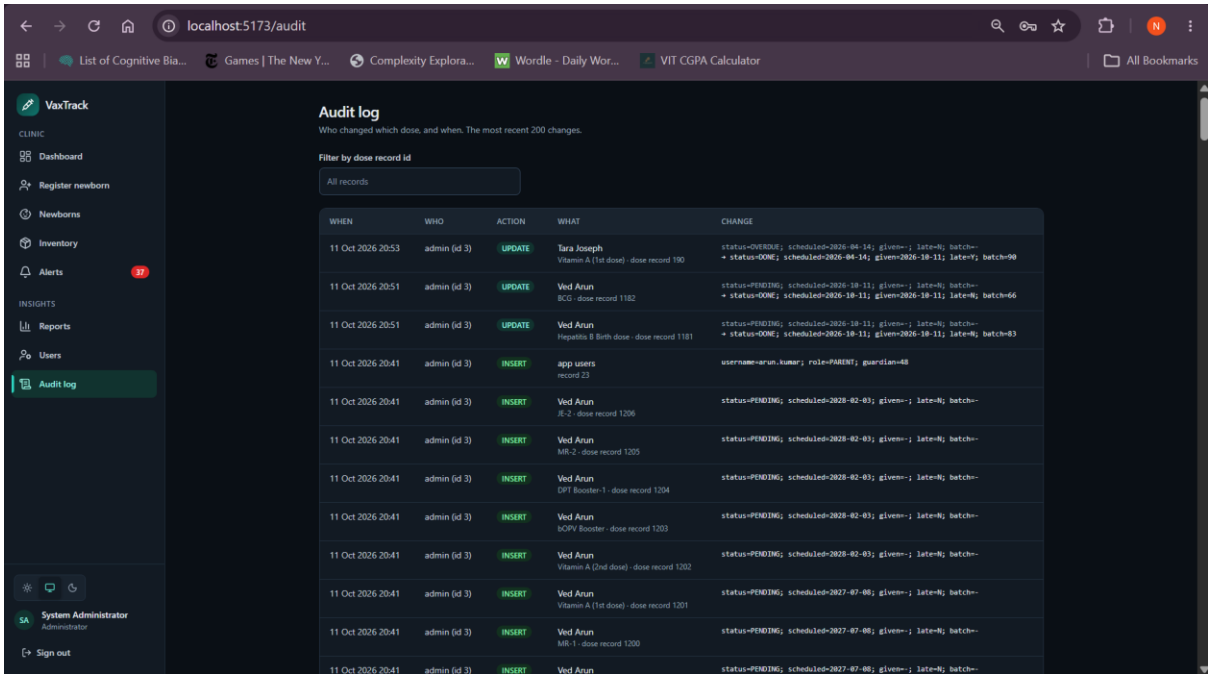


Figure 5.18: Admin audit log showing who changed the dose and when

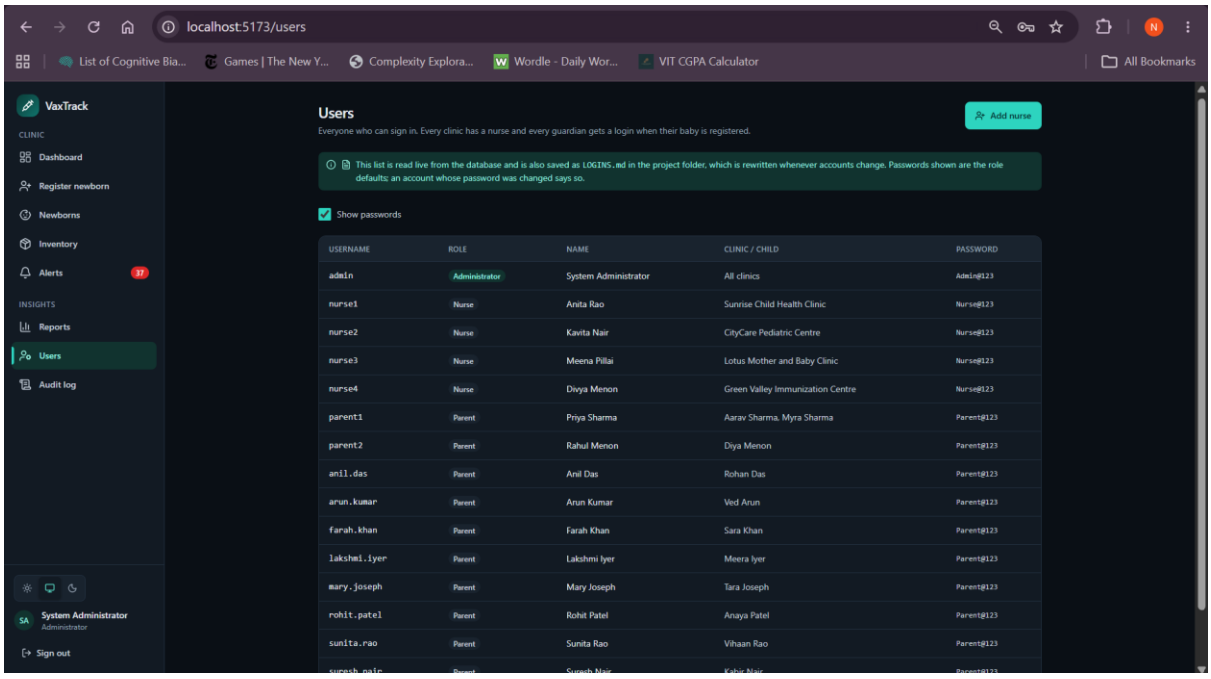


Figure 5.19: Admin Users page: every login, with the add-nurse and add-parent-login actions

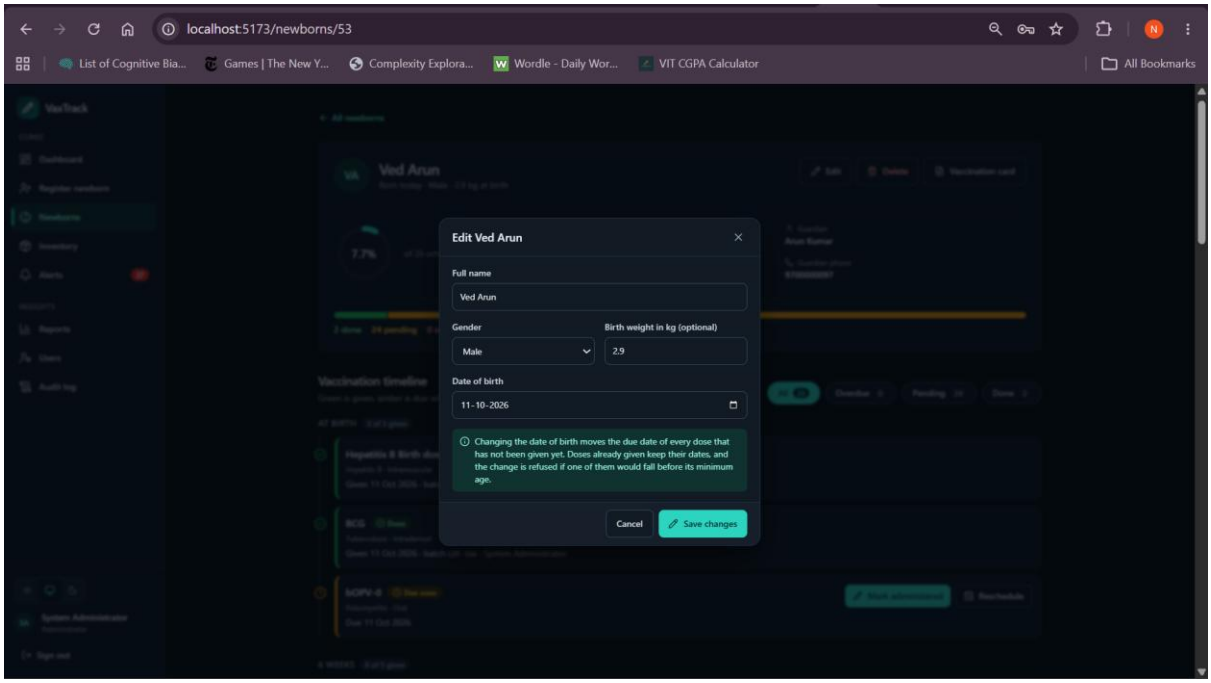


Figure 5.20: Administrator edit dialog: correcting a date of birth

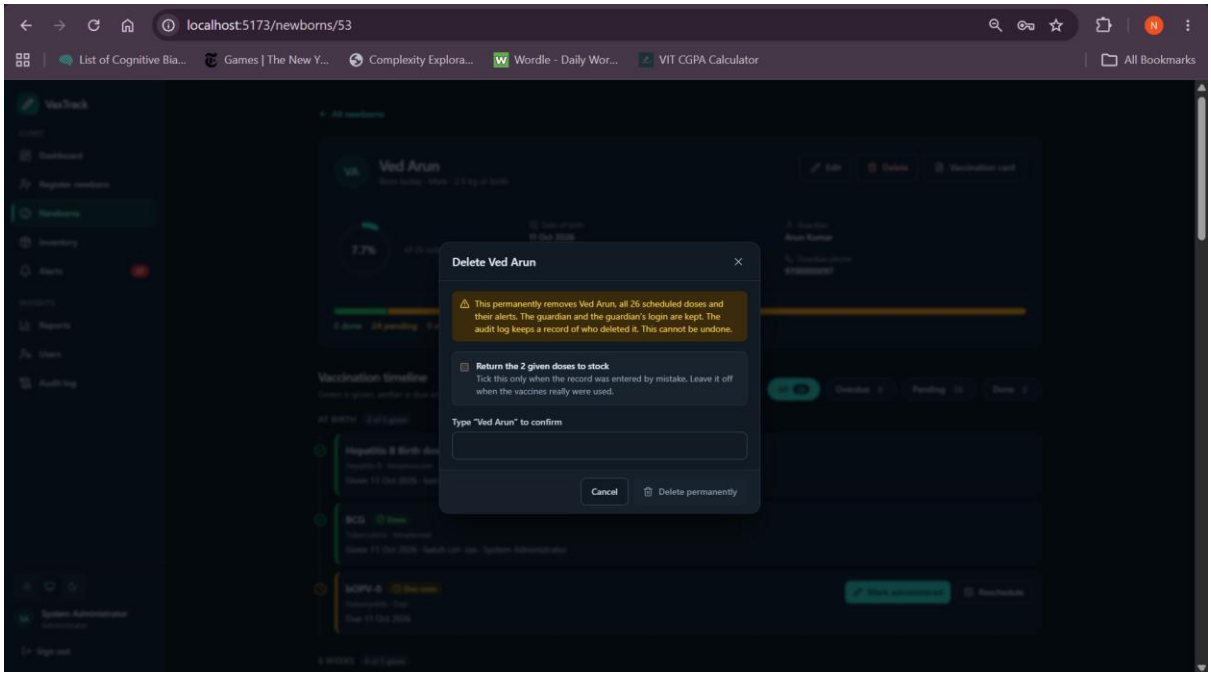


Figure 5.21: Administrator delete confirmation (type the name, optional stock return)

5.2 Integration with the DBMS

The application has three tiers (Figure 5.25). The React application [9] sends JSON over HTTP with a JSON Web Token [12]. The Express API validates the input, checks the role and the clinic, sets the acting user, calls one PL/SQL package procedure with bind variables, and turns the result into JSON. The Oracle database applies every rule. The API contains no business

logic: the schedule, the stock deduction, the late-dose flag and the alerts are all decided by the packages and triggers of Section 4.

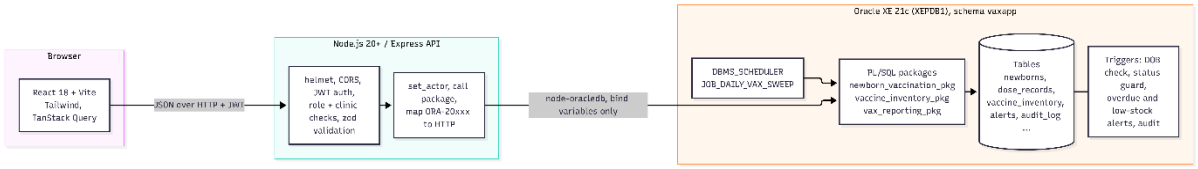


Figure 5.25: Architecture: React, Express, PL/SQL packages, Oracle

Connection. The API uses a `node-oracledb` connection pool in thin mode [7]. For each request it takes a connection, calls `newborn_vaccination_pkg.set_actor` so that the audit trigger records the right user, runs the work, commits or rolls back, and releases the connection.

```
export async function withConn(userId, fn) {
  const conn = await pool.getConnection();
  try {
    await conn.execute('BEGIN newborn_vaccination_pkg.set_actor(:id); END;', { id:
num(userId) });
    const result = await fn(conn);
    await conn.commit();
    return result;
  } catch (err) {
    try { await conn.rollback(); } catch { /* connection may already be gone */ }
    throw err;
  } finally {
    await conn.close();
  }
}
```

Calling a package. Registering a newborn is one call to the package; every value is a bind variable, and no SQL string is built from user input.

```
await call(conn,
  `BEGIN
     newborn_vaccination_pkg.register_newborn(
       p_guardian_name => :gname, p_guardian_phone => :gphone,
       p_guardian_email => :gemail, p_guardian_address => :gaddress,
       p_baby_name => :bname, p_gender => :gender,
       p_dob => TO_DATE(:dob, 'YYYY-MM-DD'), p_birth_weight_kg => :weight,
       p_clinic_id => :clinic, p_include_regional => :regional,
       p_baby_id => :baby_id, p_parent_hash => :phash, p_parent_username =>
:puser);
   END;`, { /* bind values */ });
```

Errors. A coded Oracle error such as `ORA-20009: No usable stock …` is mapped to an HTTP status (409 for no stock, 422 for a date that is not allowed, 403 for an action that only an administrator may take, and so on) and the readable message is shown to the user.

Security. Passwords are stored as bcrypt hashes [13]. Sign-in returns a signed token, and every route checks the role; a nurse can reach only the babies and the stock of their own clinic, and a parent only their own children. Input is validated with zod before it reaches the database, and the API sets the standard security headers with helmet. The known limitations are listed in Section 7.3.

Which package each screen calls.

Page (route)	Roles	API endpoints	PL/SQL objects reached
Login (`/login`)	all	`POST /auth/login`, `GET /me`	none (bcrypt + JWT in the API; reads `app_users`)
Dashboard (`/`)	A, N	`GET /dashboard/summary`, `/overdue`, `/upcoming?days=7`, `GET /clinics`	`newborn_vaccination_pkg.refresh_overdue_status`, `vax_reporting_pkg.dashboard_summary`, `overdue_list`, `upcoming_list`
Register newborn (`/register`)	A, N	`POST /newborns`, `GET /newborns/guardian-lookup?phone=` (yes/no only, 30 a minute), `GET /clinics`	`newborn_vaccination_pkg.guardian_exists`, `register_newborn` (calls `generate_schedule`, `calculate_due_date`, `account_pkg.suggest_username` and `add_user` for the guardian's parent login, all in one transaction; the guardian name is optional for a known phone), `vax_reporting_pkg.baby_timeline`, trigger `trg_newborn_dob_check`
Newborns (`/newborns`)	A, N, P	`GET /newborns?search=`	functions `get_age_text`, `get_coverage_pct`
Baby profile (`/newborns/:id`)	A, N, P	`GET /newborns/:id`	`vax_reporting_pkg.baby_timeline`, `get_age_text`, `get_coverage_pct`
Edit / Delete newborn (dialogs, admin only)	A	`PATCH /newborns/:id`, `DELETE /newborns/:id?restoreStock=1`	`newborn_vaccination_pkg.update_newborn` (recalculates unfinished due dates, late flags and stale alerts; refuses a birthday that invalidates a given dose), `delete_newborn` (optional stock return; guardian and login kept); both audited, both admin-checked in the database; triggers `trg_dose_status_guard`, `trg_dose_alerts`, `trg_dose_audit`
Mark administered (dialog)	A, N	`GET /doses/:id/batch-preview`, `POST /doses/:id/administer`	`newborn_vaccination_pkg.is_dose_allowed`, `vaccine_inventory_pkg.pick_batch`, `newborn_vaccination_pkg.mark_dose_administered`; triggers `trg_dose_status_guard`, `trg_dose_audit`, `trg_inventory_low_stock`
Reschedule (dialog)	A, N	`POST /doses/:id/reschedule`	`newborn_vaccination_pkg.reschedule_dose`

Page (route)	Roles	API endpoints	PL/SQL objects reached
Inventory (/inventory)	A, N	GET /inventory, GET /vaccines, POST /inventory/restock	dose_label, vaccine_inventory_pkg.restock
Alerts (/alerts)	A, N	GET /alerts, PATCH /alerts/:id/read	alerts are written by trg_dose_alerts, trg_inventory_low_stock, send_due_soon_alerts, expiry_sweep, mark_dose_administered
Reports (/reports)	A, N	GET /reports/clinic-coverage, GET /dashboard/overdue	vax_reporting_pkg.clinic_coverage (RANK analytic), overdue_list
Vaccination card (/newborns/:id/card)	A, N, P	GET /newborns/:id	vax_reporting_pkg.baby_timeline
Users (/users)	A	GET /users, GET /users/options, POST /users	account_pkg.create_user (nurse needs a clinic, one login per guardian, unique username, audit row); list is a SELECT
Sign-in login directory (/login)	public, development only	GET /auth/demo-logins (needs DEMO_LOGINS=true)	reads app_users; the API also rewrites LOGINS.md
Audit log (/audit)	A	GET /audit?recordId=	table audit_log, filled by trg_dose_audit and restock

The daily job JOB_DAILY_VAX_SWEEP runs the same sweeps without any page being open.

6. VERIFICATION & TESTING

6.1 Test Cases

The script `db/11_tests.sql` runs 31 test cases as anonymous blocks. Each block builds its own data (a test clinic and babies whose names start with `TEST_`), calls the object under test, compares the actual result with the expected one and writes a PASS or FAIL row to the table `test_log`; at the end the script deletes its own data, so it can be run repeatedly. The table below is taken from a run on a database built from scratch with scripts 99 to 10. Dates in the "Actual" column depend on the day of the run, and the counts in TC-R1 depend on the demo data.

ID	Description	Input	Expected	Actual	Result
TC-F1	`calculate_due_date` for a 42-day vaccine	baby DOB = today − 100, bOPV-1 (due day 42)	2026-08-09 (DOB + 42)	2026-08-09	PASS
TC-F2	`calculate_due_date` with an unknown baby	baby_id −999	error −20001	−20001	PASS
TC-F3	`get_coverage_pct` before and after doses	new baby; then 3 birth doses marked	0 → 12.5	0 → 12.5	PASS
TC-F4	`get_next_due_dose`	baby with BCG done	earliest non-DONE date, not DONE	2026-08-07, not DONE	PASS
TC-P1	`register_newborn` creates schedule	one baby; one baby with JE flag	24 / 26 doses, 1 baby	24/26/baby=1	PASS
TC-P2	`generate_schedule` idempotent	called twice more on a baby	24 → 24	24 → 24	PASS
TC-P3	future DOB rolls everything back	DOB = today + 5, new guardian phone	−20003, 0 guardians, 0 babies	−20003, guardians=0, babies=0	PASS
TC-P4	administer sets DONE, stock −1	BCG, batch qty 5	DONE, 5 → 4	DONE, 5 → 4	PASS
TC-P5	already DONE rejected	administer same dose twice	−20005, stock stays 4	−20005, stock=4	PASS
TC-P6	out-of-stock rolls back	fIPV-1 batch with qty 0	−20009, PENDING, no date, stock 0	−20009, PENDING, given=none, stock=0	PASS
TC-P7	too early	10-day-old baby, bOPV-1 (min day 42)	−20008, PENDING, stock 5	−20008, PENDING, stock=5	PASS
TC-P8	late dose	100-day-old baby, bOPV-1 (max day 70)	DONE, late=Y, 1 LATE_DOSE alert	DONE, late=Y, 1 alert	PASS
TC-P9	FEFO, expired skipped	batches +200 d, +50 d, expired −5 d	TEST-NEAR used, expired untouched (10), expired-only → −20009	as expected	PASS
TC-P10	reschedule rules	DONE dose; yesterday; before min age; DOB + 50	−20010, −20011, −20012, accepted	DONE=−20010, PAST=−20011, WINDOW=−20012, OK=1	PASS
TC-C1	overdue sweep	60-day-old baby (8 past-due doses)	8 → 0 PENDING past due, 8 OVERDUE	8 → 0; overdue=8	PASS

ID	Description	Input	Expected	Actual	Result
TC-T1	guard trigger	DONE dose updated to PENDING	−20013, status stays DONE	−20013, DONE	PASS
TC-T2	audit trigger	administer BCG as nurse1	old `status=PENDING`, new `status=DONE`, actor nurse1	status=PENDING | status=DONE | nurse1	PASS
TC-T3	low-stock trigger fires once	batch qty 11, threshold 10, three doses	alerts after each dose: 0, 1, 1	0,1,1	PASS
TC-T4	overdue alert once	sweep run twice on a 60-day-old baby	8, 8, max 1 per dose	8,8,1	PASS
TC-R1	REF cursor row counts	demo seed + one test baby	overdue=18; upcoming=5; clinics=5; summary=1; timeline=24	identical	PASS
TC-U1	register_newborn creates the guardian's parent login	new guardian TEST_Guardian, password hash of the default	username made from the guardian name, one PARENT login	test.guardian | 1 | PARENT	PASS
TC-U2	a second child of the same guardian creates no second login	same phone, second baby	no new username, still one login	test.guardian | none | logins=1	PASS
TC-U3	create_user adds a nurse; usernames unique ignoring case	nurse for TEST_CLINIC, then the same name in capitals	created, duplicate -20018, next nurse name free	created NURSE at TEST_CLINIC | dup=-20018 | next nurse name free=1	PASS
TC-U4	account rules	nurse without clinic; second login for a guardian; ADMIN role; bad hash	-20019, -20017, -20019, -20019	as expected	PASS
TC-U5	`suggest_username`: name-based, numbered when taken, accents folded, short names and length handled	"Testu Zed" three times (the first two get accounts), "Zoë Müller" with accents, "Li", a 120-character name	testu.zed | testu.zed2 | testu.zed3 | zoe.muller | parent | len=34	identical	PASS
TC-U6	`guardian_exists` answers Y/N (same matching as register_newborn)	unknown phone, then the same phone after registering (also with spaces around it), then another phone	before=N, known=Y, padded=Y, other=N	identical	PASS
TC-U7	a sibling joins the existing guardian and login; a new guardian still needs a name and an address	sibling registered with guardian name NULL and a known phone; a new phone with name NULL; a new phone with a name but no address	no new login, babies=2, logins=1, stored name kept; −20014 for both new-guardian cases	sibling: no login, babies=2, logins=1, name kept=TEST_Guardian | new guardian without name=-20014 | without address=-20014	PASS
TC-N1	update_newborn moves unfinished dose dates with the date of birth, recalculates late flags	60-day-old baby, BCG given, then date of birth set to 10 days ago (admin)	late=N, bOPV-1 due +32 days, PENDING, new name	as expected	PASS
TC-N2	a date of birth that invalidates a given dose, or lies in the future, is refused	bOPV-1 given at day 60, DOB moved to day 30; DOB in the future	-20020, date unchanged; -20003	-20020 | dob unchanged | -20003	PASS

ID	Description	Input	Expected	Actual	Result
TC-N3	delete_newborn removes baby, doses and alerts; stock returns only when asked	two babies, BCG given to each; delete one with restore, one without	stock 5 then 4, nothing left, 2 audit rows	restored=5 | kept=4 | left=0 | audit=2	PASS
TC-N4	only an administrator can edit or delete a newborn; the change is audited with the actor	nurse1 tries both; admin renames	-20021 twice, baby intact, 1 audit row by admin	update=-20021 | delete=-20021 | intact=1 | audit by admin=1	PASS

6.2 Output Screenshots

The full `test_log` after a run is shown in Figure 6.1, and close-ups of four key cases follow: the dose is recorded and the stock goes down by exactly one (Figure 6.2), an out-of-stock dose is rolled back (Figure 6.3), a late dose is flagged and raises an alert (Figure 6.4), and the low-stock trigger fires once (Figure 6.5).



```
===== test_log =====

TC_CODE DESCRIPTION                                                    EXPECTED                    ACTUAL                      RESULT
------- ------------------------------------------------------------- --------------------------- --------------------------- ------
TC-F1   calculate_due_date: DOB + 42-day vaccine (bOPV-1) gives DOB + 42 2026-08-14                2026-08-14                  PASS
TC-F2   calculate_due_date with an unknown baby raises -20001         -20001                      -20001                      PASS
TC-F3   get_coverage_pct: 0 for a new baby, 3 of 24 doses done = 12.5 0 -> 12.5                   0 -> 12.5                   PASS
TC-F4   get_next_due_dose returns the earliest dose that is not DONE  2026-08-12|not DONE         2026-08-12|not DONE         PASS
TC-P1   register_newborn creates the baby and a full schedule (24; 26 24/26/baby=1                24/26/baby=1                PASS
        with JE)

TC-P2   generate_schedule run twice more creates no duplicate doses   24 -> 24                    24 -> 24                    PASS
TC-P3   register_newborn with a future DOB fails and rolls back guardian -20003|guardians=0|babies=0 -20003|guardians=0|babies=0 PASS
        + baby

TC-P4   mark_dose_administered sets DONE and takes exactly 1 from stock DONE|stock 5 -> 4         DONE|stock 5 -> 4           PASS
TC-P5   marking an already DONE dose is rejected (-20005), stock       -20005|stock=4              -20005|stock=4              PASS
        unchanged

TC-P6   out-of-stock: error, dose stays PENDING, stock unchanged       -20009|PENDING|given=none|stock=0  -20009|PENDING|given=none|stock=0  PASS
TC-P7   administering before min_age_days (10-day-old, 6-week dose) is -20008|PENDING|stock=5      -20008|PENDING|stock=5      PASS
        rejected

TC-P8   dose given past max_age_days sets is_late = Y and raises a     DONE|late=Y|LATE_DOSE alerts=1 DONE|late=Y|LATE_DOSE alerts=1 PASS
        LATE_DOSE alert

TC-P9   FEFO: earliest non-expired batch is used, expired batches are  TEST-NEAR|expired batch    TEST-NEAR|expired batch     PASS
        skipped                                                        untouched=10|expired only=-20009  untouched=10|expired only=-20009

TC-P10  reschedule_dose rejects DONE doses, past dates, out-of-window  DONE=-20010|PAST=-20011|WINDOW=-20012| DONE=-20010|PAST=-20011|WINDOW=-20012| PASS
        dates; accepts a valid one                                     OK=1                        OK=1

TC-C1   refresh_overdue_status flips past-due PENDING doses to OVERDUE 8 -> 0; overdue=8           8 -> 0; overdue=8           PASS
TC-T1   guard trigger blocks changing a DONE dose back to PENDING      -20013|DONE                 -20013|DONE                 PASS
TC-T2   audit trigger records old status, new status and the acting user status=PENDING|status=DONE|nurse1  status=PENDING|status=DONE|nurse1  PASS
TC-T3   low-stock trigger fires once, when quantity first drops below  0,1,1                      0,1,1                       PASS
        the threshold

TC-T4   OVERDUE alerts: one per dose, none added by a second sweep     8,8,1                       8,8,1                       PASS
TC-U1   register_newborn creates the guardian's parent login (username test.guardian|1|PARENT      test.guardian|1|PARENT      PASS
        from the guardian name)

TC-U2   a second child of the same guardian does not create a second   test.guardian|none|logins=1  test.guardian|none|logins=1  PASS
        login

TC-U3   create_user adds a nurse for a clinic; usernames are unique    created NURSE at            created NURSE at            PASS
        ignoring case                                                  TEST_CLINIC|dup=-20018|next nurse name TEST_CLINIC|dup=-20018|next nurse name
                                                                       free=1                      free=1

TC-U4   account rules: nurse needs a clinic, one login per guardian, no nurse-no-clinic=-20019|guardian-has-lo nurse-no-clinic=-20019|guardian-has-lo PASS
        admin via the app, valid hash                                  gin=-20017|admin-role=-20019|bad-hash= gin=-20017|admin-role=-20019|bad-hash=
                                                                       -20019                      -20019

TC-U5   suggest_username: name-based, numbered when taken, accents     testu.zed|testu.zed2|testu.zed3|zoe.mu testu.zed|testu.zed2|testu.zed3|zoe.mu PASS
        folded, short names and length handled                         ller|parent|len=34          ller|parent|len=34

TC-U6   guardian_exists answers Y/N for a phone number (same matching as before=N|known=Y|padded=Y|other=N  before=N|known=Y|padded=Y|other=N  PASS
        register_newborn)

TC-U7   a sibling joins the existing guardian and login; a new guardian sibling: no login, babies=2, logins=1, sibling: no login, babies=2, logins=1, PASS
        still needs a name and an address                              name kept=TEST_Guardian|new guardian  name kept=TEST_Guardian|new guardian
                                                                       without name=-20014|without  without name=-20014|without
                                                                       address=-20014               address=-20014

TC-N1   update_newborn moves unfinished dose dates with the date of    late=N|bOPV-1 due +32|PENDING|dob  late=N|bOPV-1 due +32|PENDING|dob  PASS
        birth and recalculates late flags                              moved|name=TEST_N1_new      moved|name=TEST_N1_new

TC-N2   a date of birth that invalidates a given dose, or lies in the  -20020|dob unchanged|-20003  -20020|dob unchanged|-20003  PASS
        future, is refused

TC-N3   delete_newborn removes the baby, doses and alerts; stock comes restored=5|kept=4|left=0|audit=2  restored=5|kept=4|left=0|audit=2  PASS
        back only when asked

TC-N4   only an administrator can edit or delete a newborn; the change update=-20021|delete=-20021|intact=1|a update=-20021|delete=-20021|intact=1|a PASS
        is audited with the actor                                      udit by admin=1             udit by admin=1

TC-R1   reporting REF cursors return the row counts the tables imply   overdue=31;upcoming=5;clinics=5;summar overdue=31;upcoming=5;clinics=5;summar PASS
                                                                       y=1;timeline=24             y=1;timeline=24


RESULT      CASES
------ ----------
PASS          31

INFO
-----------------------------------------------------------------------
leftover TEST_ rows (must be 0): 0
11_tests.sql complete.
```

Figure 6.1: test_log: all 31 test cases PASS

TC-P4	mark_dose_administered sets DONE and takes exactly 1 from stock	DONE|stock 5 -> 4	DONE|stock 5 -> 4	PASS

Figure 6.2: TC-P4 close-up: DONE and stock decremented by exactly one

TC-P6	out-of-stock: error, dose stays PENDING, stock unchanged	-20009|PENDING|given=none|stock=0	-20009|PENDING|given=none|stock=0	PASS

Figure 6.3: TC-P6 close-up: out-of-stock rolls back

TC-P8	dose given past max_age_days sets is_late = Y and raises a LATE_DOSE alert	DONE|late=Y|LATE_DOSE alerts=1	DONE|late=Y|LATE_DOSE alerts=1	PASS

Figure 6.4: TC-P8 close-up: late dose flag and alert

TC-T3	low-stock trigger fires once, when quantity first drops below the threshold	0,1,1	0,1,1	PASS

Figure 6.5: TC-T3 close-up: low-stock trigger fires once

The API was tested separately. `api/scripts/smoke.mjs` calls every endpoint on success and failure paths (152 checks, Figure 6.11) and `api/scripts/authz-matrix.mjs` calls every route as an anonymous user, a parent, a nurse and an administrator, with forged and expired tokens as well (Figure 6.12). Examples of single responses are a login (Figure 6.6), a registration that returns 24 timeline rows (Figure 6.7), a recorded dose (Figure 6.8), an out-of-stock refusal with HTTP 409 and a clear message (Figure 6.9), and a parent who is refused a staff route with HTTP 403 (Figure 6.10).

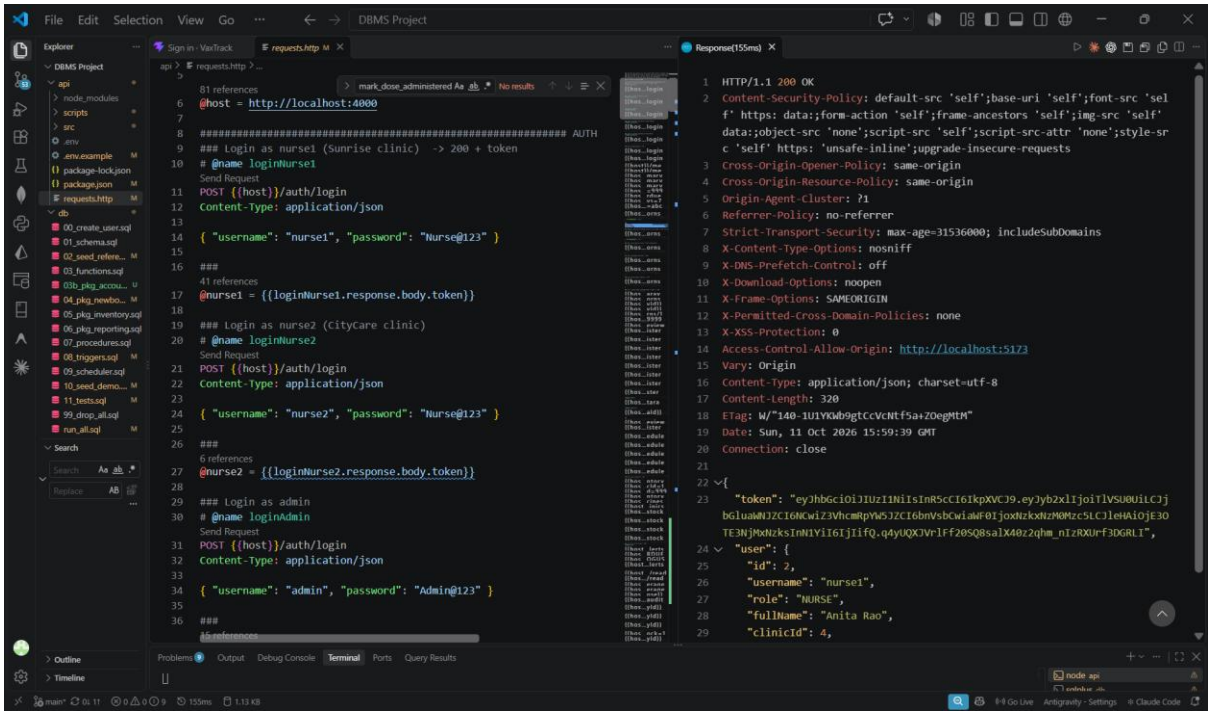


Figure 6.6: API: login returns a token and the user

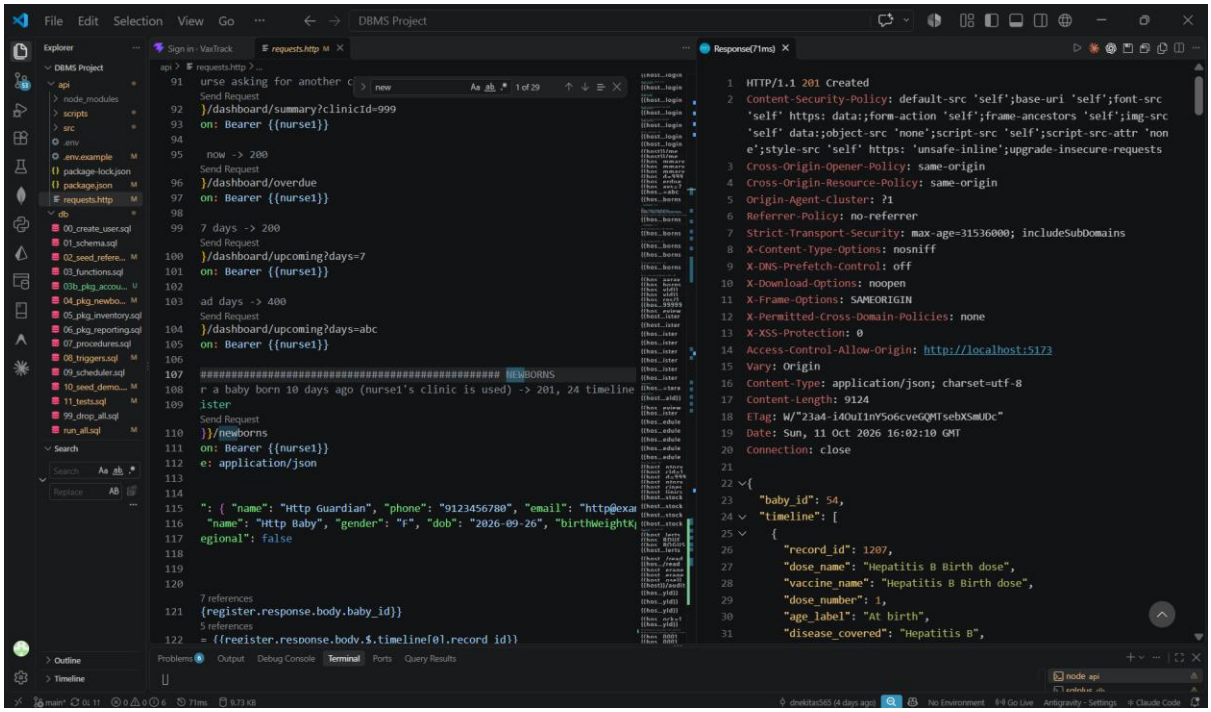


Figure 6.7: API: register a newborn returns 24 timeline rows

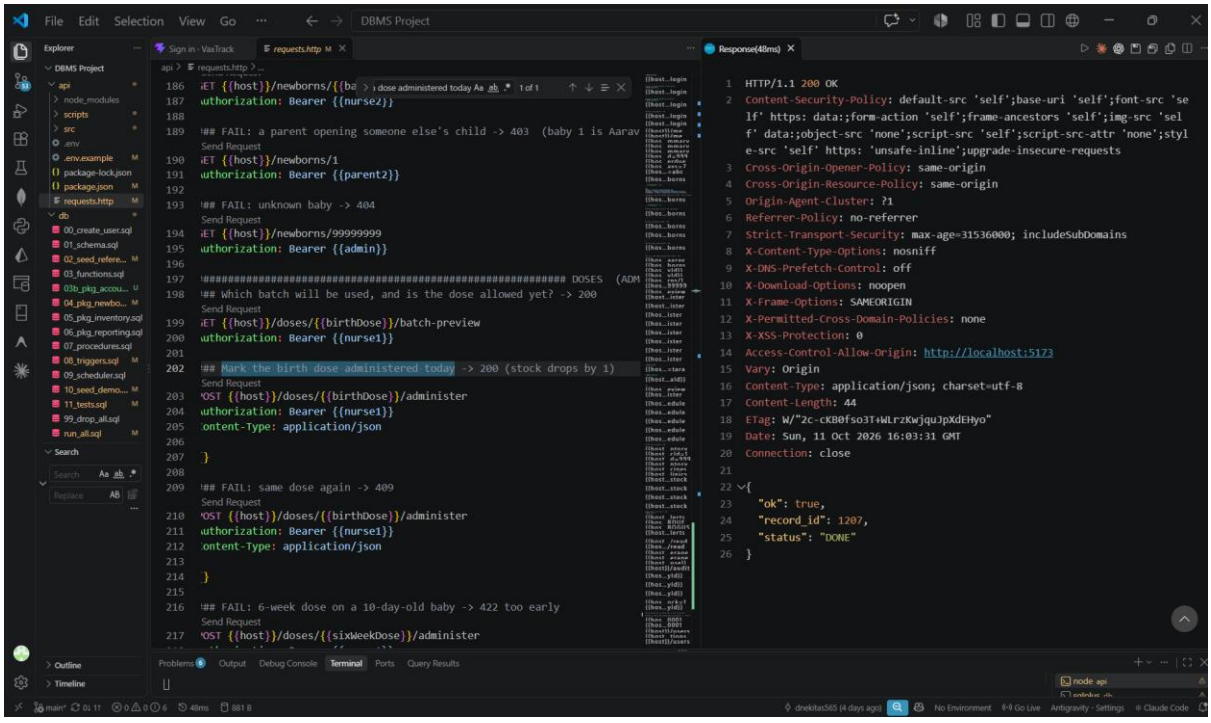


Figure 6.8: API: administer a dose

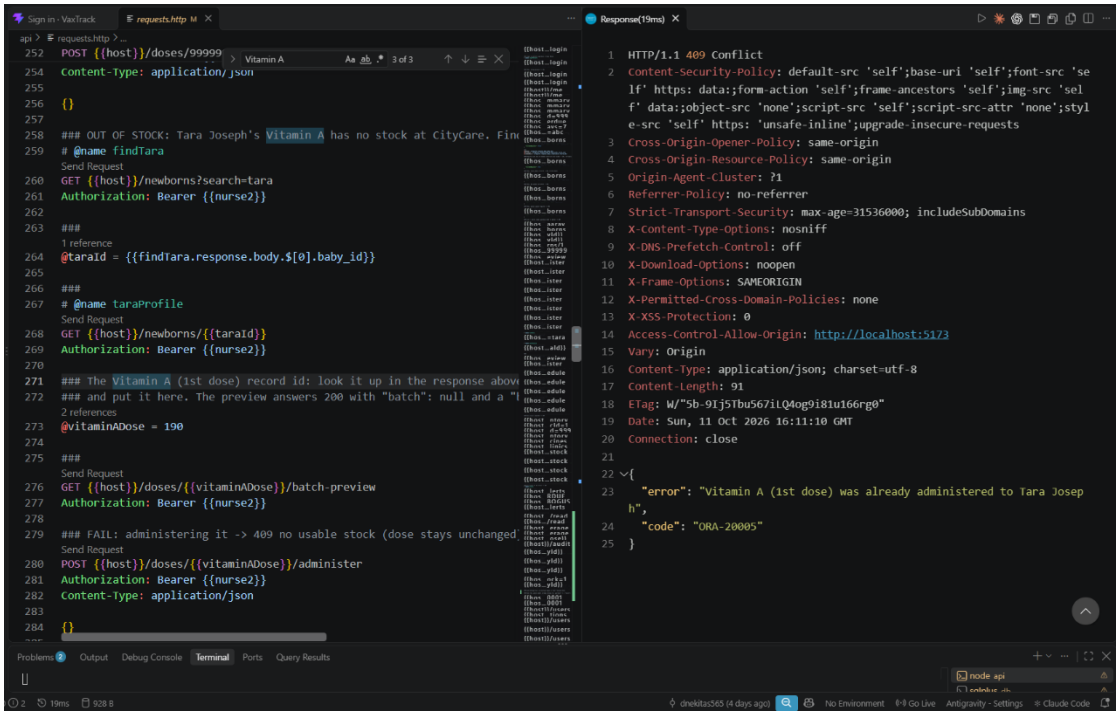


Figure 6.9: API: out of stock returns 409 with a clear message

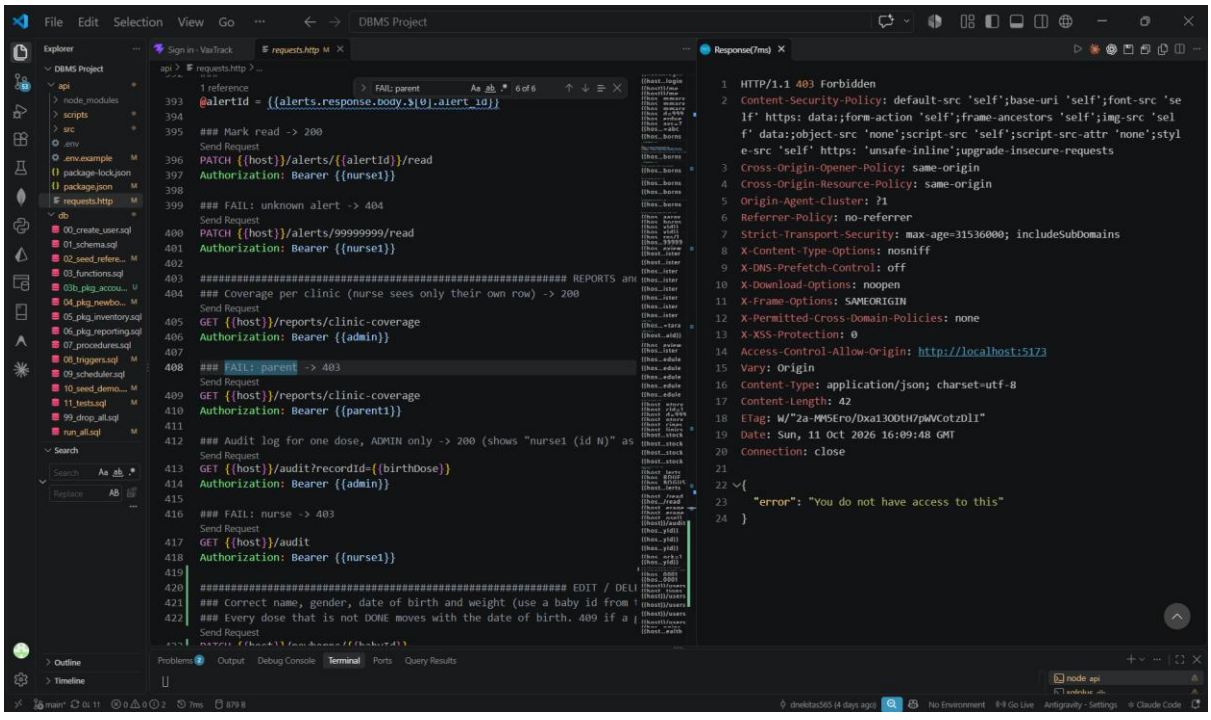
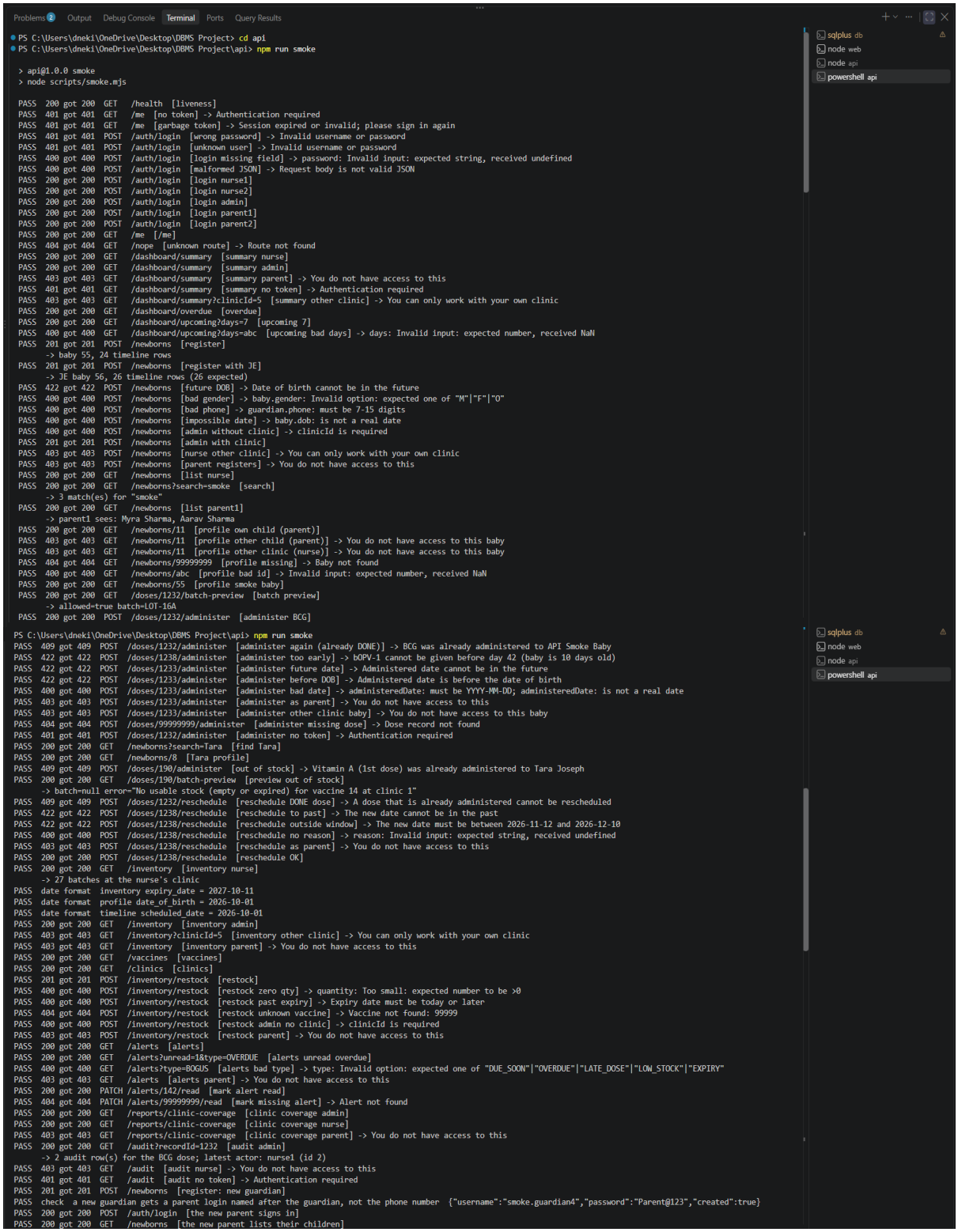


Figure 6.10: API: a parent calling a staff route gets 403


Problems 2
Output
Debug Console
Terminal
Ports
Query Results
sqlplus db
node web
node api
powershell api
PS C:\Users\dneki\OneDrive\Desktop\DBMS Project> cd api
PS C:\Users\dneki\OneDrive\Desktop\DBMS Project\api> npm run smoke
> api@1.0.0 smoke
> node scripts/smoke.mjs
PASS 200 got 200 GET /health [liveness]
PASS 401 got 401 GET /me [no token] -> Authentication required
PASS 401 got 401 GET /me [garbage token] -> Session expired or invalid; please sign in again
PASS 401 got 401 POST /auth/login [wrong password] -> Invalid username or password
PASS 401 got 401 POST /auth/login [unknown user] -> Invalid username or password
PASS 400 got 400 POST /auth/login [login missing field] -> password: Invalid input: expected string, received undefined
PASS 400 got 400 POST /auth/login [malformed JSON] -> Request body is not valid JSON
PASS 200 got 200 POST /auth/login [login nurse1]
PASS 200 got 200 POST /auth/login [login nurse2]
PASS 200 got 200 POST /auth/login [login admin]
PASS 200 got 200 POST /auth/login [login parent1]
PASS 200 got 200 POST /auth/login [login parent2]
PASS 200 got 200 GET /me [/me]
PASS 404 got 404 GET /nope [unknown route] -> Route not found
PASS 200 got 200 GET /dashboard/summary [summary nurse]
PASS 200 got 200 GET /dashboard/summary [summary admin]
PASS 403 got 403 GET /dashboard/summary [summary parent] -> You do not have access to this
PASS 401 got 401 GET /dashboard/summary [summary no token] -> Authentication required
PASS 403 got 403 GET /dashboard/summary?clinicId=5 [summary other clinic] -> You can only work with your own clinic
PASS 200 got 200 GET /dashboard/overdue [overdue]
PASS 200 got 200 GET /dashboard/upcoming?days=7 [upcoming 7]
PASS 400 got 400 GET /dashboard/upcoming?days=abc [upcoming bad days] -> days: Invalid input: expected number, received NaN
PASS 201 got 201 POST /newborns [register]
-> baby 55, 24 timeline rows
PASS 201 got 201 POST /newborns [register with JE]
-> JE baby 56, 26 timeline rows (26 expected)
PASS 422 got 422 POST /newborns [future DOB] -> Date of birth cannot be in the future
PASS 400 got 400 POST /newborns [bad gender] -> baby.gender: Invalid option: expected one of "M"|"F"|"O"
PASS 400 got 400 POST /newborns [bad phone] -> guardian.phone: must be 7-15 digits
PASS 400 got 400 POST /newborns [impossible date] -> baby.dob: is not a real date
PASS 400 got 400 POST /newborns [admin without clinic] -> clinicId is required
PASS 201 got 201 POST /newborns [admin with clinic]
PASS 403 got 403 POST /newborns [nurse other clinic] -> You can only work with your own clinic
PASS 403 got 403 POST /newborns [parent registers] -> You do not have access to this
PASS 200 got 200 GET /newborns [list nurse]
PASS 200 got 200 GET /newborns?search=smoke [search]
-> 3 match(es) for "smoke"
PASS 200 got 200 GET /newborns [list parent1]
-> parent1 sees: Myra Sharma, Aarav Sharma
PASS 200 got 200 GET /newborns/11 [profile own child (parent)]
PASS 403 got 403 GET /newborns/11 [profile other child (parent)] -> You do not have access to this baby
PASS 403 got 403 GET /newborns/11 [profile other clinic (nurse)] -> You do not have access to this baby
PASS 404 got 404 GET /newborns/99999999 [profile missing] -> Baby not found
PASS 400 got 400 GET /newborns/abc [profile bad id] -> Invalid input: expected number, received NaN
PASS 200 got 200 GET /newborns/55 [profile smoke baby]
PASS 200 got 200 GET /doses/1232/batch-preview [batch preview]
-> allowed=true batch=LOT-16A
PASS 200 got 200 POST /doses/1232/administer [administer BCG]
PS C:\Users\dneki\OneDrive\Desktop\DBMS Project\api> npm run smoke
PASS 409 got 409 POST /doses/1232/administer [administer again (already DONE)] -> BCG was already administered to API Smoke Baby
PASS 422 got 422 POST /doses/1238/administer [administer too early] -> bOPV-1 cannot be given before day 42 (baby is 10 days old)
PASS 422 got 422 POST /doses/1233/administer [administer future date] -> Administered date cannot be in the future
PASS 422 got 422 POST /doses/1233/administer [administer before DOB] -> Administered date is before the date of birth
PASS 400 got 400 POST /doses/1233/administer [administer bad date] -> administeredDate: must be YYYY-MM-DD; administeredDate: is not a real date
PASS 403 got 403 POST /doses/1233/administer [administer as parent] -> You do not have access to this
PASS 403 got 403 POST /doses/1233/administer [administer other clinic baby] -> You do not have access to this baby
PASS 404 got 404 POST /doses/99999999/administer [administer missing dose] -> Dose record not found
PASS 401 got 401 POST /doses/1232/administer [administer no token] -> Authentication required
PASS 200 got 200 GET /newborns?search=Tara [find Tara]
PASS 200 got 200 GET /newborns/8 [Tara profile]
PASS 409 got 409 POST /doses/190/administer [out of stock] -> Vitamin A (1st dose) was already administered to Tara Joseph
PASS 200 got 200 GET /doses/190/batch-preview [preview out of stock]
-> batch=null error="No usable stock (empty or expired) for vaccine 14 at clinic 1"
PASS 409 got 409 POST /doses/1232/reschedule [reschedule DONE dose] -> A dose that is already administered cannot be rescheduled
PASS 422 got 422 POST /doses/1238/reschedule [reschedule to past] -> The new date cannot be in the past
PASS 422 got 422 POST /doses/1238/reschedule [reschedule outside window] -> The new date must be between 2026-11-12 and 2026-12-10
PASS 400 got 400 POST /doses/1238/reschedule [reschedule no reason] -> reason: Invalid input: expected string, received undefined
PASS 403 got 403 POST /doses/1238/reschedule [reschedule as parent] -> You do not have access to this
PASS 200 got 200 POST /doses/1238/reschedule [reschedule OK]
PASS 200 got 200 GET /inventory [inventory nurse]
-> 27 batches at the nurse's clinic
PASS date format inventory expiry_date = 2027-10-11
PASS date format profile date_of_birth = 2026-10-01
PASS date format timeline scheduled_date = 2026-10-01
PASS 200 got 200 GET /inventory [inventory admin]
PASS 403 got 403 GET /inventory?clinicId=5 [inventory other clinic] -> You can only work with your own clinic
PASS 403 got 403 GET /inventory [inventory parent] -> You do not have access to this
PASS 200 got 200 GET /vaccines [vaccines]
PASS 200 got 200 GET /clinics [clinics]
PASS 201 got 201 POST /inventory/restock [restock]
PASS 400 got 400 POST /inventory/restock [restock zero qty] -> quantity: Too small: expected number to be >0
PASS 400 got 400 POST /inventory/restock [restock past expiry] -> Expiry date must be today or later
PASS 404 got 404 POST /inventory/restock [restock unknown vaccine] -> Vaccine not found: 99999
PASS 400 got 400 POST /inventory/restock [restock admin no clinic] -> clinicId is required
PASS 403 got 403 POST /inventory/restock [restock parent] -> You do not have access to this
PASS 200 got 200 GET /alerts [alerts]
PASS 200 got 200 GET /alerts?unread=1&type=OVERDUE [alerts unread overdue]
PASS 400 got 400 GET /alerts?type=BOGUS [alerts bad type] -> type: Invalid option: expected one of "DUE_SOON"|"OVERDUE"|"LATE_DOSE"|"LOW_STOCK"|"EXPIRY"
PASS 403 got 403 GET /alerts [alerts parent] -> You do not have access to this
PASS 200 got 200 PATCH /alerts/142/read [mark alert read]
PASS 404 got 404 PATCH /alerts/99999999/read [mark missing alert] -> Alert not found
PASS 200 got 200 GET /reports/clinic-coverage [clinic coverage admin]
PASS 200 got 200 GET /reports/clinic-coverage [clinic coverage nurse]
PASS 403 got 403 GET /reports/clinic-coverage [clinic coverage parent] -> You do not have access to this
PASS 200 got 200 GET /audit?recordId=1232 [audit admin]
-> 2 audit row(s) for the BCG dose; latest actor: nurse1 (id 2)
PASS 403 got 403 GET /audit [audit nurse] -> You do not have access to this
PASS 401 got 401 GET /audit [audit no token] -> Authentication required
PASS 201 got 201 POST /newborns [register: new guardian]
PASS check a new guardian gets a parent login named after the guardian, not the phone number {"username":"smoke.guardian4","password":"Parent@123","created":true}
PASS 200 got 200 POST /auth/login [the new parent signs in]
PASS 200 got 200 GET /newborns [the new parent lists their children]

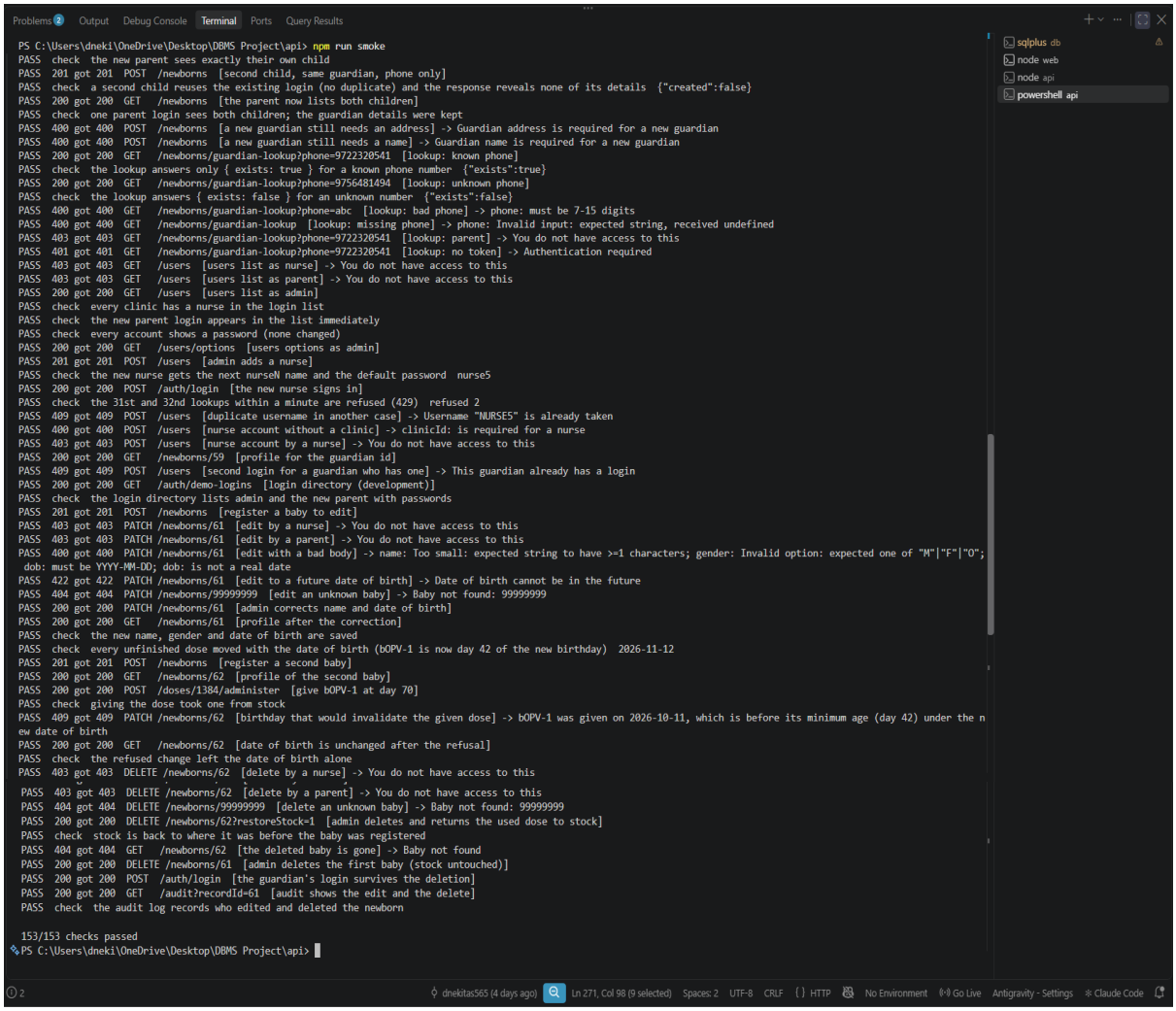


Figure 6.11: npm run smoke: 152 of 152 checks

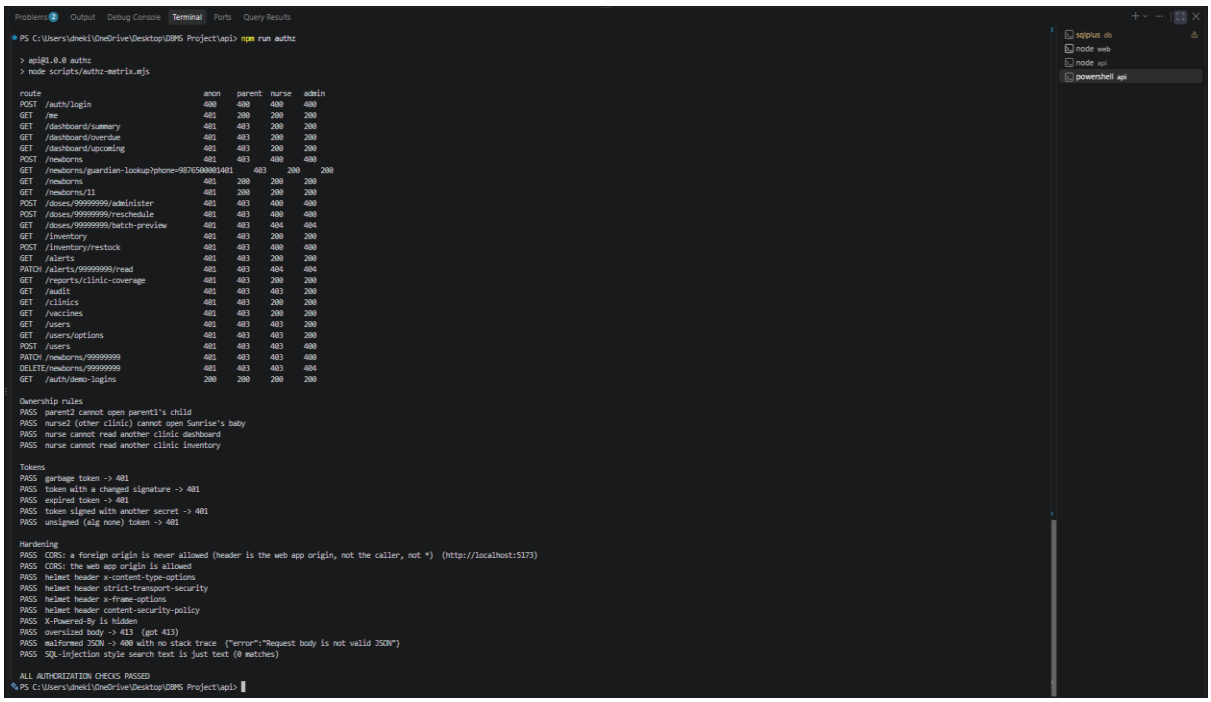


Figure 6.12: npm run authz: role by route matrix and token checks

6.3 Validation of Functionality

Constraints and triggers. The negative cases show that the database defends itself. A date of birth in the future is rejected and the registration rolls back completely (TC-P3). A DONE dose cannot be changed back to PENDING (TC-T1). Stock cannot go below zero, and a failed administration leaves both the dose and the stock unchanged (TC-P6). The audit trigger records the old and new status and the acting user (TC-T2), the low-stock alert is raised once and not on every dose (TC-T3), and the overdue alert is raised once per dose even when the sweep runs twice (TC-T4).

Procedures and packages. The registration creates 24 doses, or 26 with the Japanese encephalitis box ticked, and running `generate_schedule` again adds none (TC-P1, TC-P2). Recording a dose rejects a dose that is already given, a baby that is too young, and an empty or expired batch, flags a late dose, and picks the batch that expires first while skipping an expired one (TC-P4 to TC-P9). Rescheduling rejects a given dose, a past date and a date outside the age window (TC-P10). The overdue sweep flips exactly the past-due doses (TC-C1). The reporting cursors return the number of rows that plain SQL counts at the same moment (TC-R1).

Accounts and administrator tools. A new guardian gets exactly one parent login named after the guardian, a second child adds no second login, and a sibling can be registered with the phone number alone (TC-U1, TC-U2, TC-U7). Usernames are unique ignoring case and are numbered when a name is taken (TC-U3, TC-U5). The account rules reject a nurse without a clinic, a second login for a guardian, an administrator account created through the application, and a bad password hash (TC-U4). The date-of-birth correction moves the unfinished doses, is refused when it would invalidate a given dose, and only an administrator can edit or delete a newborn (TC-N1 to TC-N4).

Front end. A scripted browser run of 25 steps (sign-in, restock, registration, administering a dose, the low-stock alert, rescheduling, the failure case, the parent's view and the audit log) passes in the light and in the dark theme. Further browser checks cover the account and administrator features, theme switching, and the absence of horizontal scrolling at 360, 390, 768 and 1280 px.

Bugs the tests found. The first run failed TC-R1 with `ORA-00937: not a single-group group function`, because `dashboard_summary` combined aggregates and scalar subqueries in one SELECT. We rewrote the query with inline views joined together and ran

the whole chain again from an empty schema. When the account tests were added, repeated runs left some test guardians behind, because the cleanup matched them by phone pattern; it now matches them by name, and the final check counts any leftover rows.

7. RESULTS & DISCUSSION

7.1 Achievements

Measured against the objectives of Section 1.2:

1. **Schema.** A ten-table schema in BCNF with primary, foreign, unique and check constraints, identity keys and 14 supporting indexes. The immunization schedule is data: changing a due age is an UPDATE of `vaccine_master`.
2. **Business rules in the database.** All five kinds of object are implemented and compile without errors: five functions, one procedure, four packages, five triggers and a scheduler job, together with explicit, parameterized, FOR-loop and REF cursors. Dose and stock can never disagree because recording a dose is one transaction with a savepoint, and the stock update is guarded against two nurses taking the last dose.
3. **Web application.** Three roles use a responsive web application with light and dark themes that reaches the database only through the packages. A baby's full schedule appears at registration, overdue and due-soon doses are flagged, and a parent can print the vaccination card.
4. **Verification.** 31 of 31 automated PL/SQL test cases pass and leave no test data behind; 152 of 152 API checks pass; the authorization matrix shows that every route refuses the roles that may not use it; and the scripted browser journey passes in both themes.
5. **Accessibility.** Lighthouse accessibility scores were 100 on every page and device profile that we audited in the production build, in both themes (Figure 7.1, Figure 7.2), measured with Lighthouse [15] against the accessibility guidelines [14]. The target was 90. Lighthouse covers only part of the accessibility guidelines, so this does not replace a manual keyboard and screen-reader test.

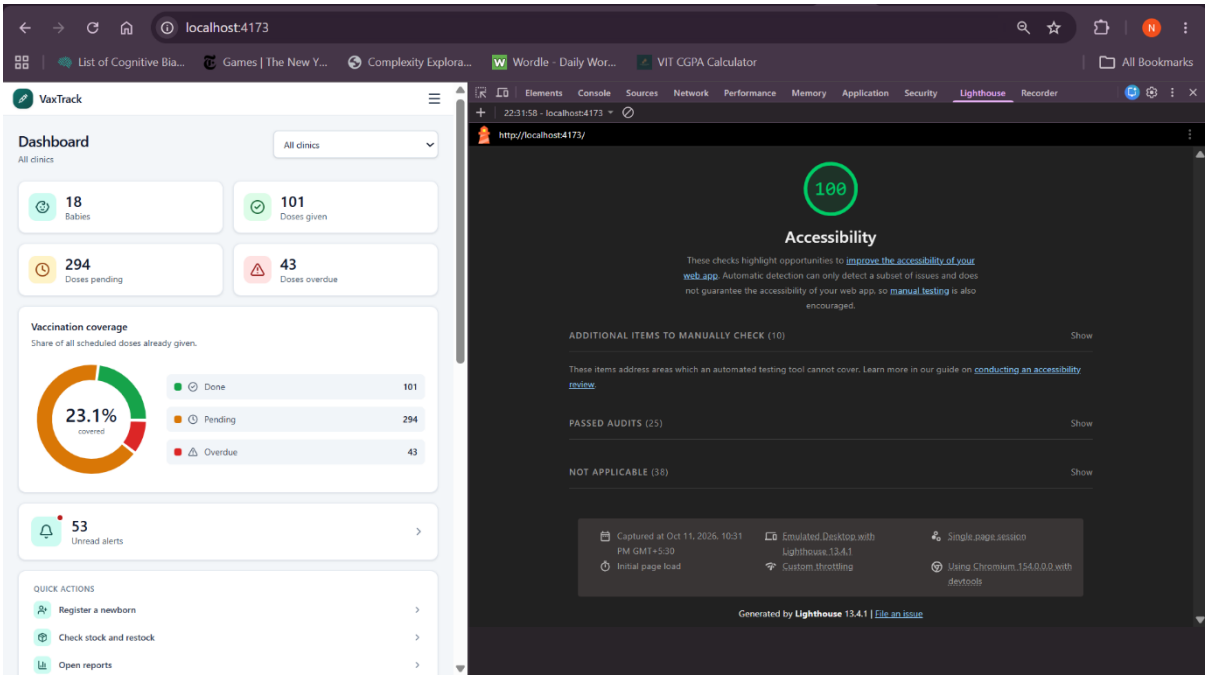


Figure 7.1: Lighthouse accessibility score for the dashboard (production build)

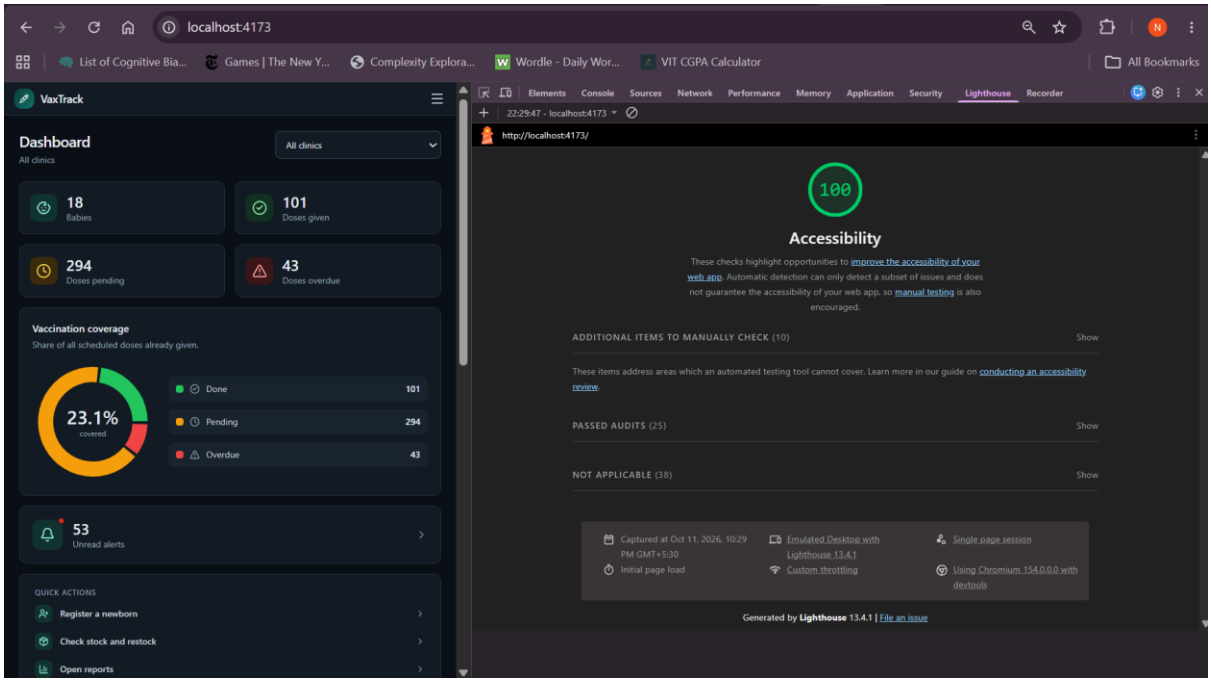


Figure 7.2: Lighthouse accessibility score in dark mode

7.2 Challenges Faced

- **Mutating-table errors.** A row-level trigger that reads the table it fires on raises ORA-04091. We avoided it by design: no trigger reads its own table (the status guard reads `newborns`, the alert trigger reads `newborns` and `vaccine_master`). Because a row trigger fires only when its row is touched and nothing touches a dose merely because

time has passed, the overdue sweep updates each past-due pending dose to its own value so that the triggers fire.

- **Keeping stock and doses consistent.** Recording a dose changes two tables. We put both changes in one procedure with a savepoint, locked the dose row, and added the guard `quantity > 0` to the stock update, so a failure at any step, or a second nurse taking the last dose, leaves nothing half done.
- **A sweep that could abort.** While reviewing, we saw that `refresh_overdue_status` touched each past-due row with an unguarded update. If a nurse marked a dose DONE just after the sweep read it, the guard trigger would have rejected the update and aborted the whole sweep. The update now has `AND status = 'PENDING'`, so such a row is skipped. Forty-eight concurrent dashboard requests, each of which runs the sweep, all succeeded.
- **A correct date of birth.** After an administrator corrects a date of birth, an overdue dose gets a new date but keeps its status, so the alert trigger did not fire. We changed its WHEN clause to fire also when an overdue dose's date changes, which still gives one alert per dose.
- **PL/SQL restrictions.** A function declared inside a block cannot be called from a SQL statement (PLS-00231); we assigned its value to a variable first. The first version of the dashboard summary query failed with ORA-00937 and was rewritten (Section 6.3).
- **Application issues.** Oracle DATE values reached the browser as JavaScript date strings and every page showed "Invalid Date"; a fetch-type handler now returns `YYYY-MM-DD`. User names were case sensitive, so "Nurse1" was refused on phones that capitalize the first letter; sign-in is now case-insensitive. An asynchronous session callback stalled the connection pool and was removed. On a 390 px phone the page was 425 px wide because grid columns would not shrink; the grids now have a one-column base. The overflow check that had missed this compared the page with `window.innerWidth`, which a phone browser widens; it now compares with the real device width.

7.3 Possible Improvements

- **Performance.** The tables are small, but paging of the long lists and a materialized view for the clinic coverage report would be needed for many clinics.
- **Security.** The token is kept in the browser's local storage, so a cross-site-scripting flaw would expose it; a production system should use an HttpOnly cookie. The login limiter

is in memory and is lost when the API restarts. The demo accounts share one default password per role; a real deployment would force a change at first sign-in and provide a change-password page. A web server in front of the application should add a content security policy and HSTS. The application schema owner is used by the API; a production system would use a smaller set of privileges.

- **Data.** Phone numbers are compared exactly as typed, so "98765 43210" and "9876543210" would be two guardians; a normalisation step would remove this.
- **Features.** SMS or WhatsApp reminders to parents, a multi-language interface, digital vaccination certificates, and the vaccines that are outside our scope.

8. CONCLUSION

8.1 Summary of Learning Outcomes

Through this project we applied the main ideas of a database course to one working application. We moved from an unnormalized vaccination card to a BCNF schema and saw how each step removes an anomaly. We used constraints for the rules that fit them and triggers for those that do not, such as a date of birth that cannot be in the future. We wrote functions, a procedure and packages in PL/SQL, and used explicit, parameterized, FOR-loop and REF cursors where row-by-row processing or a result set for the application was needed. We learned to design a transaction as a unit: savepoints, row locks and guarded updates keep a dose and its stock consistent, and an error is raised with a code that the application can translate. We learned that a row-level trigger must not read its own table, and why a trigger needs a row to be touched before it fires. We also learned to test a database systematically: a script that builds its own data, compares expected with actual output, and removes what it created found real defects that manual clicking had not.

8.2 Future Scope

- The vaccines beyond newborn tracking: Td for pregnant women, the DPT booster at 5–6 years, Td at 10 and 16 years, and vitamin A doses 3 to 9.
- Reminders to parents by SMS or WhatsApp, a multi-language interface, and digital vaccination certificates.
- A progressive web app or native mobile application, and deployment to a cloud database and web server with proper session cookies, a content security policy and a forced password change at first sign-in.
- Analytics across clinics: coverage by district, drop-out between doses, and forecasting of the stock each clinic needs.
- More than one login per family, so that both parents can have their own.

9. REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] Oracle Corporation, "Database PL/SQL Language Reference, 21c," Oracle Help Center. [Online].

[4] Oracle Corporation, "Oracle Database 21c Express Edition Documentation," Oracle Help Center. [Online].

[5] Oracle Corporation, "Oracle Database 21c: Administrator's Guide, Scheduling Jobs with Oracle Scheduler (DBMS_SCHEDULER)," Oracle Help Center. [Online].

[6] Ministry of Health and Family Welfare, Government of India, "National Immunization Schedule (NIS)," document supplied for the project (file *National Immunization Schedule (NIS).pdf*).

[7] Oracle Corporation, "node-oracledb: The Node.js Driver for Oracle Database, documentation, version 7." [Online].

[8] OpenJS Foundation, "Express 5.x API reference." [Online].

[9] Meta Platforms, "React documentation, version 18." [Online].

[10] Vite contributors, "Vite documentation." [Online].

[11] Tailwind Labs, "Tailwind CSS documentation, version 4." [Online].

[12] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015. [Online].

[13] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conf., FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[14] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation. [Online].

[15] Google, "Lighthouse accessibility scoring," Chrome for Developers. [Online].

10. APPENDICES

Appendix A – Additional Code

The complete source is in the project folder and is to be pasted into this appendix in the order below (monospaced, 9–10 pt). Run in this order on an empty schema, each script can be run again.

Order	File	Contents
1	`db/00_create_user.sql`	creates the application user `vaxapp` and its privileges
2	`db/01_schema.sql`	all tables, constraints, identity keys and indexes
3	`db/02_seed_reference.sql`	clinics, the vaccine schedule, the administrator and nurse logins
4	`db/03_functions.sql`	the five standalone functions
5	`db/03b_pkg_accounts.sql`	`account_pkg`
6	`db/04_pkg_newborn.sql`	`newborn_vaccination_pkg` with the table of error codes
7	`db/05_pkg_inventory.sql`	`vaccine_inventory_pkg`
8	`db/06_pkg_reporting.sql`	`vax_reporting_pkg`
9	`db/07_procedures.sql`	`send_due_soon_alerts`
10	`db/08_triggers.sql`	the five triggers
11	`db/09_scheduler.sql`	the daily scheduler job
12	`db/10_seed_demo.sql`	demonstration babies, doses and stock
13	`db/11_tests.sql`	the 31 automated test cases
–	`db/run_all.sql`, `report/run_all.log`	the whole chain and a log of one clean run
–	`api/src/`	the API: `db.js`, `app.js`, `errors.js`, `validate.js` and the routes
–	`web/src/`	the React application

The API and web sources are long; if the page limit is a problem, include `db.js`, `errors.js`, `routes/newborns.js` and `pages/RegisterNewborn.jsx` and refer to the repository for the rest.

Appendix B – Additional Screenshots

Screenshots that are not placed in the main body, each with a figure caption in the same numbering style, go here (for example further API responses, or additional pages in the dark theme).